

**Optimasi Rute Distribusi Rendah Emisi pada Green Capacitated Vehicle
Routing Problem menggunakan Variable Neighborhood Search dengan
Clarke-Wright Savings sebagai solusi awal**



LORANYOH OEMATAN

H011221003



**PROGRAM STUDI MATEMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM**

UNIVERSITAS HASANUDDIN

MAKASSAR 2026

**Optimasi Rute Distribusi Rendah Emisi pada Green Capacitated Vehicle
Routing Problem menggunakan Variable Neighborhood Search dengan
Clarke-Wright Savings sebagai solusi awal**

LORANYOH OEMATAN

H011221003



PROGRAM STUDI MATEMATIKA

FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM

UNIVERSITAS HASANUDDIN

MAKASSAR

2026

**Optimasi Rute Distribusi Rendah Emisi pada Green Capacitated Vehicle
Routing Problem menggunakan Variable Neighborhood Search dengan
Clarke-Wright Savings sebagai solusi awal**

LORANYOH OEMATAN

H011221003

Skripsi

sebagai salah satu syarat untuk mencapai gelar sarjana

Program Studi Matematika

pada

**PROGRAM STUDI MATEMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS HASANUDDIN
MAKASSAR
2026**

SKRIPSI**OPTIMASI RUTE DISTRIBUSI RENDAH EMISI PADA
GREEN CAPACITATED VEHICLE ROUTING PROBLEM
MENGUNAKAN VARIABLE NEIGHBORHOOD SEARCH
DENGAN CLARKE-WRIGHT SAVINGS SEBAGAI SOLUSI AWAL****LORANYOH OEMATAN****H011221003**

Skripsi,

telah dipertahankan di depan Panitia Ujian Sarjana Sains pada tanggal 11 Agustus
2026 dan dinyatakan telah memenuhi syarat kelulusan pada

Program Studi Matematika
Departemen Matematika
Fakultas Matematika dan Ilmu Pengetahuan Alam
Universitas Hasanuddin
Makassar

Mengesahkan:
Pembimbing tugas akhir,

Dr. Khaeruddin, M.Sc
NIP. 19650914 199103 1 001



Mengetahui:
Ketua Program Studi,

Dr. Firman, S.Si., M.Si
NIP. 19680429 200212 1 001

PERNYATAAN KEASLIAN SKRIPSI DAN PELIMPAHAN HAK CIPTA

Dengan ini saya menyatakan bahwa, skripsi berjudul "*Optimasi Rute Distribusi Pemadatan Emisi pada Green Capacitated Vehicle Routing Problem menggunakan Variable Neighborhood Search dengan Clarke-Wright Savings sebagai solusi awal*" adalah benar karya saya dengan arahan dari Bapak Dr. Khaeruddin, M.Sc. sebagai Pembimbing. Karya ilmiah ini belum diajukan dan tidak sedang diajukan dalam bentuk apa pun kepada perguruan tinggi mana pun. Sumber informasi yang berasal atau dikutip dari karya yang diterbitkan maupun tidak diterbitkan dari penulis lain telah disebutkan dalam teks dan dicantumkan dalam Daftar Pustaka skripsi ini. Apabila di kemudian hari terbukti atau dapat dibuktikan bahwa sebagian atau keseluruhan skripsi ini adalah karya orang lain, maka saya bersedia menerima sanksi atas perbuatan tersebut berdasarkan aturan yang berlaku.

Dengan ini saya melimpahkan hak cipta (hak ekonomis) dari karya tulis saya berupa skripsi ini kepada Universitas Hasanuddin.



UCAPAN TERIMA KASIH

Segala puji bagi Tuhan Yang Maha Esa yang telah melimpahkan berkat-Nya kepada penulis sehingga skripsi dengan judul *“Optimasi Rute Distribusi Rendah Emisi pada Green Capacitated Vehicle Routing Problem menggunakan Variable Neighborhood Search dengan Clarke-Wright Savings sebagai solusi awal”* ini dapat selesai. Penulis mengucapkan terima kasih khususnya kepada kedua orang tua, Bapak dan Mama yang telah membesarkan dan mendidik penulis, begitu pula kepada adik-adik penulis yang telah memberikan dukungan pada penulis. Penulis menyadari bahwa penyelesaian skripsi ini tidak lepas dari dukungan dan bantuan banyak pihak baik secara langsung maupun tidak langsung. Untuk itu pada kesempatan ini penulis mengucapkan terima kasih kepada:

1. Bapak Prof. Dr. Ir. Jamaluddin Jompa, M.Sc., selaku Rektor Universitas Hasanuddin, Dr. Sci. Muhammad Zakir, M.Si. selaku Dekan Fakultas Matematika dan Ilmu Pengetahuan Alam, serta Bapak Dr. Firman, S.Si., M.Si., selaku Ketua Departemen Matematika.
2. Bapak Dr. Khaeruddin, M.Sc., selaku dosen pembimbing utama atas kesediaan dan kesabarannya dalam membimbing dan memberikan arahan kepada penulis, serta meluangkan banyak waktu sehingga penulis dapat menyelesaikan skripsi ini.
3. Bapak Prof. Agustinus Ribal, S.Si., M.Sc. PhD, selaku penasihat akademik sekaligus dosen penguji serta Bapak Prof. Dr. Syamsuddin Toaha, M.Sc. selaku dosen penguji yang telah memberikan saran dan arahan kepada penulis dalam penyusunan skripsi ini.
4. Seluruh dosen dan staf Departemen Matematika yang telah memberikan ilmunya kepada penulis selama menjadi mahasiswa di Departemen Matematika.
5. Teman-teman seperjuangan penulis dalam pelayanan di Rumah Biru GMKI, UTMUH, Kawan-kawan Rill 2022, teman-teman Program studi Matematika angkatan 2022, sahabat Zalacca, serta saudara-saudara KKT yang senantiasa memberikan dukungan, kebersamaan, motivasi, dan semangat kepada penulis selama menjalani perkuliahan maupun dalam proses penyusunan skripsi ini.
6. Seluruh pihak yang tidak dapat penulis sebutkan satu per satu, yang telah membantu dan memberikan dukungan dalam bentuk apapun.

Penulis menyadari bahwa skripsi ini masih jauh dari kesempurnaan, kritik dan saran yang membangun diharapkan oleh penulis untuk perbaikan dan pengembangan penelitian lebih lanjut. Akhir kata, semoga skripsi ini dapat bermanfaat bagi siapa saja yang membacanya.

ABSTRAK

LORANYOH OEMATAN. **Optimasi Rute Distribusi Rendah Emisi pada Green Capacitated Vehicle Routing Problem menggunakan Variable Neighborhood Search dengan Clarke-Wright Savings sebagai solusi awal** (dibimbing oleh Dr. Khaeruddin, M.Sc)

Peningkatan aktivitas distribusi barang menyebabkan bertambahnya emisi gas rumah kaca dari sektor transportasi sehingga diperlukan optimasi rute yang tidak hanya memperhatikan efisiensi distribusi, tetapi juga meminimalkan dampak lingkungan. Permasalahan tersebut dimodelkan dalam Green Capacitated Vehicle Routing Problem (GCVRP), yaitu pengembangan dari Capacitated Vehicle Routing Problem (CVRP) yang mempertimbangkan kapasitas kendaraan sekaligus emisi yang dihasilkan selama proses distribusi. Penelitian ini bertujuan mengoptimalkan rute distribusi pada GCVRP menggunakan metode Clarke-Wright Savings (CWS) sebagai pembentuk solusi awal dan Variable Neighborhood Search (VNS) sebagai metode optimasi untuk meminimalkan total emisi kendaraan. Dataset yang digunakan berasal dari CVRPLIB, yaitu tiga instance Augerat, A-n32-k5, A-n60-k9, dan A-n80-k10, yang dikembangkan menjadi GCVRP dengan menambahkan parameter emisi berupa tahun kendaraan, kapasitas kendaraan, koefisien emisi dasar, dan koefisien emisi berdasarkan beban. Pengujian dilakukan pada tiga skenario, yaitu kendaraan homogen, kendaraan dengan koefisien emisi berbeda, serta kendaraan dengan kapasitas dan koefisien emisi berbeda. Metode CWS digunakan untuk membentuk rute awal berdasarkan nilai penghematan, sedangkan VNS memperbaiki solusi melalui struktur *neighborhood relocate*, *swap*, *reverse*, dan *two-opt* dengan fungsi tujuan meminimalkan total emisi. Hasil penelitian menunjukkan bahwa seluruh solusi yang dihasilkan memenuhi kendala kapasitas kendaraan dan setiap pelanggan dilayani tepat satu kali. Dibandingkan dengan solusi awal CWS, penerapan VNS berhasil menurunkan total emisi pada seluruh dataset dan skenario pengujian dengan persentase penurunan berkisar antara 7,31% hingga 20,61%, serta rata-rata sebesar 13,49%. Hasil tersebut menunjukkan bahwa kombinasi metode Clarke-Wright Savings dan Variable Neighborhood Search efektif dalam menghasilkan rute distribusi yang lebih rendah emisi pada permasalahan Green Capacitated Vehicle Routing Problem, sehingga dapat menjadi pendekatan yang baik dalam mendukung distribusi yang lebih ramah lingkungan.

Kata kunci: *relocate*, efisiensi, skenario, ramah lingkungan.

ABSTRACT

Increased goods distribution activity has led to a rise in greenhouse gas emissions from the transport sector, necessitating route optimization that considers not only distribution efficiency but also the minimization of environmental impact. This issue is modeled as the Green Capacitated Vehicle Routing Problem (GCVRP), an extension of the Capacitated Vehicle Routing Problem (CVRP) that accounts for both vehicle capacity and emissions generated during distribution. This study aims to optimize distribution routes for the GCVRP by employing the Clarke-Wright Savings (CWS) method to generate an initial solution and Variable Neighborhood Search (VNS) as the optimization method to minimize total vehicle emissions. The datasets utilized are sourced from CVRPLIB—specifically three Augerat instances (A-n32-k5, A-n60-k9, and A-n80-k10)—adapted into GCVRP instances by incorporating emission parameters such as vehicle model year, capacity, base emission coefficient, and load-dependent emission coefficient. Testing was conducted across three scenarios: a homogeneous vehicle fleet, vehicles with varying emission coefficients, and vehicles with differing capacities and emission coefficients. The CWS method was used to construct initial routes based on savings values, while VNS refined the solution using relocate, swap, reverse, and two-opt neighborhood structures, with the objective of minimizing total emissions. The results demonstrate that all generated solutions satisfy vehicle capacity constraints and ensure each customer is served exactly once. Compared to the initial CWS solutions, the application of VNS successfully reduced total emissions across all datasets and test scenarios, with reduction percentages ranging from 7.31% to 20.61% and an average reduction of 13.49%. These findings indicate that combining the Clarke-Wright Savings method with Variable Neighborhood Search is effective in generating lower-emission distribution routes for the GCVRP, offering a viable approach to support more environmentally friendly distribution practices.

Keywords: relocationt, efficiency, scenario, environmentally friendly

DAFTAR ISI

PERNYATAAN KEASLIAN.....	viii
UCAPAN TERIMA KASIH	ix
ABSTRAK.....	vii
<i>ABSTRACT</i>	viii
DAFTAR ISI.....	ix
DAFTAR TABEL.....	xi
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Tujuan Penelitian.....	2
1.4 Batasan Masalah.....	3
1.5 Manfaat Penelitian.....	3
1.6 Landasan Teori	3
BAB II METODOLOGI PENELITIAN	16
2.1 Metode penelitian	16
2.2 Jenis dan Sumber Data	16
2.3 Transformasi Dataset CVRPLIB menjadi GCVRP.....	16
2.4 Prosedur Penelitian	16
2.5 Pseudocode Algoritma	17
2.5 Alur penelitian.....	19
BAB III HASIL DAN PEMBAHASAN	20
3.1 Hasil	21
3.1.1 Deskripsi data.....	21
3.1.2 Implementasi Algoritma.....	23

3.2 Pembahasan	45
BAB IV PENUTUP.....	46
DAFTAR PUSTAKA	48
LAMPIRAN	49

DAFTAR TABEL

Nomor Urut	Halaman
1. Data permintaan pelanggan.....	8
2. Data jarak pelanggan.....	8
3. Urutan nilai saving	8
4. Hasil akhir rute	9
5. Permintaan pelanggan.....	12
6. Matriks jarak pelanggan.....	12
7. Emisi rute 1	13
8. Emisi rute 2.....	13
9. Perhitungan emisi rute 1 baru.....	14
10. Perhitungan emisi rute 2 baru	14
11. Karakteristik data	22
12. Parameter Eksperimen	22
13. Hasil Clarke-Wright savings pada data A-n32-k5 dengan skenario 1 dan 2.....	24
14. Data Kendaraan A-n32-k5 dengan skenario 3	25
15. Hasil Clarke-Wright Savings data A-n32-k5 dengan skenario 3.....	25
16. Hasil Clarke-Wright Savings data A-n60-k9 dengan skenario 1 dan 2.....	26
17. Hasil Clarke-Wright Savings Skenario 3 pada Data A-n60-k9.....	27
18. Hasil Clarke-Wright Savings Skenario 1 dan 2 pada Data A-n80-k10.....	28
19. Hasil Clarke-Wright Savings Data 80 Skenario 3.....	29
20. Hasil Implementasi Variable Neighborhood Search	31
21. Hasil Optimasi Variable Neighborhood Search	32
22. Hasil Optimasi VNS Data 32 Skenario 3	33
23. Hasil Implementasi Variable Neighborhood Search A-n60-k9.....	34
24. Hasil Optimasi VNS Berbasis Emisi pada Data A-n60-k9	35
25. Hasil Optimasi Variable Neighborhood Search Data A-n60-k9 Skenario 3	37
26. Hasil Optimasi VNS Berbasis Emisi pada Data A-n80-k10	38
27. Hasil Optimasi Rute Distribusi Data A-n80-k10.....	40
28. Hasil Optimasi Variable Neighborhood Search Data 80 Skenario 3.....	41
29. Perbandingan Jarak hasil CWS dan VNS	43
30. Perbandingan Emisi hasil CWS dan VNS	44
31. <i>Gap</i> hasil dengan BKS dataset.....	45

DAFTAR LAMPIRAN

No Urut	Halaman
1. Diagram Implementasi Algoritma	49
2. Gambar Rute hasil tiap Dataset	50
3. Grafik Konvergensi VNS	57
4. Kode Program.....	62

BAB I

PENDAHULUAN

1.1 Latar Belakang

Distribusi barang merupakan salah satu aktivitas utama dalam sistem logistik karena berperan penting dalam memastikan produk dapat sampai kepada pelanggan secara tepat waktu, tepat jumlah dan dengan biaya yang efisien. Dalam pelaksanaannya, proses distribusi tidak hanya menuntut pengiriman barang sampai kepada pelanggan, tetapi juga memerlukan perencanaan rute yang tepat agar kendaraan dapat melayani seluruh titik tujuan secara efisien. Rute distribusi yang tidak dirancang dengan baik dapat menimbulkan pemborosan jarak tempuh, waktu perjalanan, dan penggunaan bahan bakar (Toth & Vigo 2014).

Selain berkaitan dengan efisiensi operasional, kegiatan distribusi juga memiliki hubungan erat dengan isu lingkungan. Transportasi jalan masih menjadi salah satu penyumbang emisi karbon yang signifikan, terutama karena sebagian besar kendaraan distribusi menggunakan bahan bakar fosil. International Energy Agency melaporkan bahwa emisi sektor transportasi jalan masih menjadi perhatian utama dalam upaya penurunan emisi global. Hal ini menunjukkan bahwa pengelolaan distribusi modern tidak cukup hanya berorientasi pada efisiensi biaya, tetapi juga perlu memperhatikan pengurangan emisi kendaraan (International Energy Agency, 2025).

Salah satu permasalahan optimasi yang banyak digunakan dalam perencanaan rute distribusi adalah *Vehicle Routing Problem* atau VRP. VRP merupakan permasalahan penentuan rute kendaraan dari suatu depot menuju sejumlah pelanggan dengan tujuan memperoleh rute terbaik berdasarkan kriteria tertentu, seperti jarak, waktu, atau biaya. Dalam pengembangannya, VRP memiliki beberapa variasi sesuai dengan kendala yang digunakan, salah satunya adalah *Capacitated Vehicle Routing Problem* (CVRP), yaitu VRP yang mempertimbangkan batas kapasitas kendaraan dalam melayani permintaan pelanggan (Toth & Vigo, 2014). Seiring berkembangnya kebutuhan distribusi yang lebih berkelanjutan, CVRP kemudian berkembang ke arah *Green Vehicle Routing Problem* (GVRP) yaitu penyusunan rute yang tidak hanya mempertimbangkan efisiensi jarak, tetapi juga aspek lingkungan seperti konsumsi bahan bakar dan emisi karbon. Selanjutnya, apabila konsep GVRP dikombinasikan dengan batasan kapasitas kendaraan sebagaimana terdapat dalam CVRP, maka permasalahan tersebut dapat diarahkan menjadi *Green Capacitated Vehicle Routing Problem* atau GCVRP.

GCVRP menjadi relevan karena emisi kendaraan tidak hanya dipengaruhi oleh jarak tempuh, tetapi juga oleh faktor lain seperti beban kendaraan, kecepatan, konsumsi bahan bakar, karakteristik perjalanan. Oleh karena itu, rute dengan jarak terpendek belum tentu selalu menghasilkan emisi paling rendah. Bektas dan Laporte menjelaskan bahwa permasalahan rute yang mempertimbangkan polusi perlu memperhitungkan emisi, konsumsi bahan bakar, dan waktu perjalanan. Dengan

demikian, optimasi rute distribusi rendah emisi memerlukan pendekatan yang tidak hanya berfokus pada jarak, tetapi juga pada nilai emisi yang dihasilkan kendaraan. (Bektas & Laporte, 2011).

Salah satu metode yang dapat digunakan untuk membentuk solusi awal dalam penyelesaian masalah rute kendaraan adalah *Clarke-Wright Savings*. Metode ini bekerja dengan menghitung nilai penghematan jarak apabila dua pelanggan digabungkan ke dalam satu rute dibandingkan apabila pelanggan tersebut dilayani secara terpisah dari depot. Meskipun *Clarke-Wright Savings* efektif dalam membentuk rute awal, metode ini pada dasarnya masih berorientasi pada penghematan jarak, sehingga diperlukan metode perbaikan solusi agar rute yang terbentuk tidak berhenti pada susunan awal yang belum tentu optimal. Salah satu metode yang dapat digunakan adalah *Variable Neighborhood Search* yang merupakan metode metaheuristic yang bekerja dengan mengubah struktur ketetangaan secara sistematis untuk menemukan solusi yang lebih baik. Dalam konteks GCVRP, VNS relevan digunakan untuk memperbaiki rute awal melalui pertukaran atau pemindahan pelanggan sehingga dapat diperoleh rute distribusi yang lebih rendah emisi (Moreno Pérez; Garside dan Rahmat., 2024).

Data emisi menunjukkan bahwa sektor transportasi masih menjadi sumber pencemaran yang signifikan. Pada tahun 2023, emisi CO₂ sektor transportasi Indonesia mencapai 149.538 ribu ton CO₂ atau sekitar 22% dari total emisi nasional, dengan transportasi jalan menyumbang sekitar 90% dari emisi sektor transportasi (Asian Transport Observatory, 2024). Selain itu, IESR mencatat bahwa 80% emisi sektor transportasi berasal dari transportasi jalan, dengan kontribusi angkutan barang sebesar 30% dari emisi transportasi jalan (IESR, 2025). Kondisi ini menunjukkan bahwa aktivitas distribusi barang perlu dikelola melalui rute yang lebih efisien dan rendah emisi. Oleh karena itu, penulis mengangkat tugas akhir dengan judul “ **Optimasi Rute Distribusi Rendah Emisi pada Green Capacitated Vehicle Routing Problem menggunakan Variable Neighborhood Search dengan Clarke & Wright Savings sebagai solusi awal**”.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, maka rumusan masalah yang akan dikaji dalam penelitian ini sebagai berikut

1. Bagaimana penerapan metode Clarke-Wright Savings sebagai solusi awal pada Green Capacitated Vehicle Routing Problem (GCVRP)
2. Bagaimana penerapan metode Variable Neighborhood Search (VNS) mengoptimalkan rute distribusi pada Green Capacitated Vehicle Routing Problem (GCVRP)

1.3 Tujuan Penelitian

Tujuan dari penelitian ini adalah untuk :

1. Menerapkan metode Clarke-Wright Savings sebagai solusi awal dalam mengoptimalkan rute distribusi pada Green Capacitated Vehicle Routing Problem

2. Menerapkan metode Variable Neighborhood Search dalam mengoptimalkan rute distribusi pada Green Capacitated Vehicle Routing Problem

1.4 Batasan Masalah

Adapun batasan masalah dalam penelitian ini adalah sebagai berikut:

1. Penelitian ini hanya membahas permasalahan Green Capacitated Vehicle Routing Problem pada kasus distribusi barang menggunakan tiga dataset CVRPLIB, yaitu A-n32-k5, A-n60-k9, dan A-n80-k10.
2. Data yang digunakan meliputi koordinat depot, koordinat pelanggan, jumlah *Demand* pelanggan, jumlah kendaraan, kapasitas kendaraan, serta menambahkan parameter emisi kendaraan yang ditentukan berdasarkan skenario penelitian dan tidak merepresentasikan hasil pengukuran emisi aktual.
3. Perhitungan jarak antar titik menggunakan jarak Euclidean berdasarkan koordinat depot dan pelanggan, sehingga tidak mempertimbangkan kondisi jalan sebenarnya, kemacetan, cuaca, waktu tempuh, maupun hambatan operasional lainnya.
4. Penelitian ini menggunakan tiga skenario kendaraan, yaitu kendaraan homogen, kendaraan dengan koefisien emisi berbeda, serta kendaraan dengan kapasitas dan koefisien emisi berbeda.
5. Metode Clarke-Wright Savings digunakan hanya untuk memperoleh solusi awal berdasarkan nilai penghematan jarak, sedangkan optimasi rute untuk meminimalkan emisi dilakukan menggunakan algoritma Variable Neighborhood Search.
6. Hasil penelitian difokuskan pada perbandingan total emisi kendaraan sebelum dan sesudah penerapan Variable Neighborhood Search, tanpa membahas biaya distribusi, waktu pelayanan, konsumsi bahan bakar secara langsung, maupun perbandingan dengan algoritma optimasi lain.

1.5 Manfaat Penelitian

Manfaat penelitian ini adalah sebagai berikut:

- 1 Hasil penelitian ini dapat digunakan sebagai sumber pembelajaran dalam program akademik yang berkaitan
- 2 Model GCVRP yang digunakan dapat menjadi solusi bagi suatu perusahaan dalam perencanaan rute di masa yang akan datang.

1.6 Landasan Teori

1.6.1 Distribusi

Distribusi merupakan kegiatan penyaluran barang atau jasa dari pihak produsen, pemasok atau sumber barang menuju konsumen akhir melalui saluran distribusi (Subagyo 2000). Dalam konteks logistik, Bowersox (2006) menjelaskan bahwa distribusi berkaitan dengan pengelolaan strategis terhadap pemindahan dan penyimpanan barang, mulai dari pemasok, fasilitas perusahaan, hingga sampai kepada pelanggan. Oleh karena itu distribusi memiliki hubungan erat dengan transportasi, penyimpanan, penjadwalan, serta penentuan rute pengiriman.

1.6.2 Optimasi

Optimasi merupakan proses pencarian hasil terbaik yang dapat dicapai secara efektif berdasarkan tujuan tertentu. Dalam bidang matematika, optimasi berkaitan dengan kajian untuk menentukan nilai minimum atau maksimum dari suatu fungsi riil. Untuk memperoleh nilai optimal tersebut, diperlukan pemilihan nilai variabel baik dalam bentuk bilangan bulat maupun bilangan riil, secara sistematis sehingga menghasilkan solusi terbaik (Mindaputra 2009).

1.6.3 Emisi

Emisi dalam optimasi rute merupakan gas buang yang dihasilkan kendaraan selama proses distribusi dari depot menuju pelanggan. Emisi umumnya berasal dari pembakaran bahan bakar kendaraan, terutama emisi karbon dioksida (CO_2). Dalam kegiatan distribusi, besarnya emisi dipengaruhi oleh jarak tempuh, jenis kendaraan, konsumsi bahan bakar, kecepatan, serta beban muatan kendaraan. Menurut Úbeda, Faulin, Serrano, dan Arcelus (2014), total emisi CO_2 dapat dihitung berdasarkan jarak antar titik dan faktor emisi kendaraan dengan rumus:

$$E_{ij} = \varepsilon(q_j) \times d_{ij}$$

Sedangkan apabila kendaraan yang digunakan memiliki karakteristik emisi berbeda, maka rumus emisi dapat dikembangkan dengan memasukkan indeks kendaraan k sebagai berikut. (Xiao, Zhao, Kaku, & Xu 2012)

$$E_{ij}^k = (\alpha_k + \beta_k L_{ij}^k) d_{ij}$$

Faktor emisi $\varepsilon(q_j)$ dimodelkan secara linear berdasarkan karakteristik kendaraan dan muatan kendaraan. Dengan demikian, semakin jauh jarak tempuh dan semakin besar muatan kendaraan, maka emisi yang dihasilkan juga semakin besar.

Keterangan:

E_{ij} = emisi dari titik i ke titik j ;

d_{ij} = jarak dari titik i ke titik j ;

$\varepsilon(q_j)$ = faktor emisi yang dipengaruhi oleh muatan atau permintaan.

α_k = koefisien emisi dasar kendaraan k .

β_k = koefisien emisi tambahan berdasarkan muatan.

L_{ij}^k = sisa muatan kendaraan pada ruas i ke j .

1.6.4 Vehicle Routing Problem

Vehicle Routing Problem (VRP) merupakan permasalahan optimasi yang berkaitan dengan penentuan rute terbaik bagi sejumlah kendaraan untuk melayani sejumlah pelanggan dari suatu depot. Menurut Toth dan Vigo (2002), VRP bertujuan merancang sekumpulan rute optimal bagi armada kendaraan dalam melayani

pelanggan tertentu. Tujuan utama dari VRP adalah meminimalkan jarak tempuh, waktu perjalanan, atau biaya distribusi yang dikeluarkan perusahaan.

1.6.4.1 Capacitated Vehicle Routing Problem

Capacitated Vehicle Routing Problem (CVRP) merupakan permasalahan optimasi rute kendaraan yang bertujuan menentukan rute distribusi terbaik dari depot menuju sejumlah pelanggan dengan mempertimbangkan batas kapasitas kendaraan. Dalam CVRP, setiap kendaraan memiliki kapasitas angkut tertentu, sedangkan setiap pelanggan memiliki jumlah permintaan yang harus dipenuhi. Oleh karena itu, rute yang dibentuk harus mampu melayani seluruh pelanggan tanpa melebihi kapasitas kendaraan yang digunakan (Supardi dan Sianturi 2020)

1.6.4.2 Green Vehicle Routing Problem

Green Vehicle Routing Problem (GVRP) merupakan pengembangan dari Vehicle Routing Problem yang tidak hanya mempertimbangkan efisiensi rute distribusi, tetapi juga memperhatikan aspek lingkungan. GVRP muncul karena aktivitas transportasi merupakan salah satu bagian penting dalam logistik, yang juga menjadi salah satu penyumbang emisi polutan. Tujuan utama dari GVRP adalah meminimalkan total jarak tempuh dengan tetap memperhatikan keterbatasan operasional kendaraan, seperti kapasitas bahan bakar, waktu perjalanan, dan kebutuhan pengisian bahan bakar di sepanjang rute. (Erdoğan & Miller-Hooks, 2012).

1.6.4.2 Green Capacitated Vehicle Routing Problem

Green Capacitated Vehicle Routing Problem (GCVRP) merupakan pengembangan dari CVRP yang tidak hanya mempertimbangkan kapasitas kendaraan dan rute distribusi, tetapi juga memperhatikan aspek lingkungan. Pada GCVRP, tujuan optimasi diperluas dengan mempertimbangkan emisi yang dihasilkan selama proses distribusi. Menurut Úbeda, Faulin, Serrano, dan Arcelus (2014), GCVRP dapat dipahami sebagai permasalahan CVRP klasik yang ditambahkan dengan kriteria minimisasi emisi lingkungan. Emisi yang dimaksud terutama berupa emisi karbon dioksida (CO_2) dari kendaraan distribusi. Besarnya emisi tersebut dipengaruhi oleh beberapa faktor seperti jarak tempuh, beban kendaraan, kecepatan, kondisi cuaca, jenis kendaraan, serta konsumsi bahan bakar. Pada GCVRP, emisi kendaraan dipengaruhi oleh beberapa faktor, seperti jarak tempuh, beban kendaraan, jenis kendaraan, konsumsi bahan bakar, dan karakteristik operasional kendaraan. Semakin jauh jarak tempuh dan semakin besar muatan yang dibawa, maka emisi yang dihasilkan kendaraan cenderung semakin tinggi. Oleh karena itu, penentuan rute pada GCVRP tidak hanya diarahkan untuk memperoleh jarak terpendek, tetapi juga untuk menurunkan emisi kendaraan selama proses distribusi.

Didefinisikan variabel Keputusan sebagai berikut

$$x_{ij}^k = \begin{cases} 1, & \text{jika kendaraan } k \text{ melakukan perjalanan dari titik } v_i \text{ ke titik } v_j. \\ 0, & \text{jika kendaraan } k \text{ tidak melakukan perjalanan dari titik } v_i \text{ ke titik } v_j. \end{cases}$$

Sedangkan variabel muatan kendaraan didefinisikan sebagai berikut

$$L_{ij}^k$$

Dengan L_{ij}^k menyatakan sisa muatan kendaraan k ketika melakukan perjalanan dari titik v_i ke titik v_j .

Emisi kendaraan pada setiap ruas perjalanan dipengaruhi oleh jarak tempuh dan sisa muatan kendaraan. Emisi kendaraan pada ruas perjalanan dari titik v_i ke titik v_j oleh kendaraan k dirumuskan sebagai berikut.

$$E_{ij}^k = (\alpha_k + \beta_k L_{ij}^k) d_{ij}$$

Jika Z adalah fungsi tujuan, maka fungsi tujuan GCVRP yaitu:

$$\text{Min } Z = \sum_{k \in K} \sum_{i \in V} \sum_{j \in V, i \neq j} (\alpha_k x_{ij}^k + \beta_k L_{ij}^k) d_{ij}$$

Dengan kendala sebagai berikut

- a. Setiap pelanggan dikunjungi tepat satu kali

$$\sum_{k \in K} \sum_{i \in V, i \neq j} x_{ij}^k = 1, \quad \forall j \in N$$

- b. Keseimbangan aliran kendaraan pada setiap pelanggan

$$\sum_{\substack{i \in V \\ i \neq h}} x_{ih}^k = \sum_{\substack{j \in V \\ j \neq h}} x_{hj}^k, \quad \forall h \in N, \forall k \in K$$

- c. Total permintaan dalam satu rute tidak melebihi kapasitas kendaraan

$$\sum_{i \in N} q_i \sum_{\substack{j \in V \\ j \neq i}} x_{ij}^k \leq Q, \quad \forall k \in K$$

- d. Sisa muatan pada setiap ruas tidak melebihi kapasitas kendaraan

$$0 \leq L_{ij}^k \leq Q x_{ij}^k, \quad \forall i, j \in V, i \neq j, \forall k \in K$$

- e. Kendaraan yang berangkat harus kembali ke depot

$$\boxed{\sum_{j \in N} x_{0j}^k = \sum_{i \in N} x_{i0}^k, \quad \forall k \in K}$$

Kendala ini memastikan bahwa jumlah keberangkatan kendaraan k dari depot sama dengan jumlah kepulangannya ke depot.

Dengan :

K = himpunan kendaraan.

V = himpunan seluruh titik, terdiri dari depot dan pelanggan.

N = himpunan pelanggan.

- x_{ij}^k = variabel keputusan perjalanan
 d_{ij} = jarak dari titik v_i ke titik v_j .
 q_i = permintaan pelanggan i .
 Q = kapasitas kendaraan.
 α_k = koefisien emisi dasar kendaraan k .
 β_k = koefisien emisi tambahan berdasarkan muatan.
 L_{ij}^k = sisa muatan kendaraan pada ruas i ke j .
 Z = total emisi kendaraan yang diminimumkan

1.6.5 Clarke-Wright Savings

Clarke-Wright Savings pertama kali diperkenalkan oleh Clarke dan Wright (1964) dalam jurnal berjudul *Scheduling of Vehicles from a Central Depot to a Number of Delivery Points* yang diterbitkan dalam *Operations Research*. Dalam jurnal tersebut Metode Clarke-Wright Savings dideskripsikan merupakan salah satu metode heuristik yang digunakan untuk menyelesaikan permasalahan rute kendaraan dengan cara menghitung nilai penghematan jarak (*saving*) dari penggabungan dua pelanggan ke dalam satu rute. Pada tahap awal, setiap pelanggan dilayani secara terpisah dari depot. Selanjutnya pasangan pelanggan dengan nilai penghematan terbesar akan digabungkan secara bertahap selama tidak melanggar batasan yang ditentukan, seperti kapasitas kendaraan dan ketentuan bahwa setiap pelanggan hanya dikunjungi satu kali. Rumus umum nilai saving pada metode Clarke-Wright Savings adalah sebagai berikut.

$$S_{ij} = d_{0i} + d_{0j} - d_{ij}$$

Keterangan :

- S_{ij} = Nilai saving antara pelanggan i dan j
 d_{0i} = Jarak depot ke pelanggan i
 d_{0j} = Jarak depot ke pelanggan j
 d_{ij} = Jarak antara pelanggan i dan pelanggan j

Hal tersebut menunjukkan bahwa nilai saving diperoleh dari selisih antara jarak ketika pelanggan dilayani secara terpisah dan jarak ketika pelanggan digabungkan dalam satu rute. Setelah seluruh nilai saving dihitung, nilai tersebut diurutkan dari yang terbesar sampai terkecil. Proses penggabungan rute kemudian dilakukan mulai dari pasangan pelanggan dengan nilai saving terbesar. Setelah nilai saving dihitung, seluruh nilai dikumpulkan dalam daftar saving, kemudian diurutkan secara menurun, dan penggabungan rute dimulai dari nilai saving terbesar. Pichpibul, T., & Kawtummachai, R. (2013).

Berikut contoh studi kasus

Pada sebuah perusahaan yang bergerak di bidang distribusi barang, proses pengiriman dilakukan dari satu depot menuju empat pelanggan tetap. Setiap pelanggan memiliki jumlah permintaan yang berbeda, sedangkan kendaraan yang digunakan memiliki kapasitas maksimum sebesar 10 satuan muatan.

Tabel 1 Data permintaan pelanggan

Pelanggan	Demand
1	4
2	3
3	5
4	2

Tabel 2 Data jarak pelanggan

Dari/Ke	Depot	1	2	3	4
Depot	0	10	12	8	9
1	10	0	7	9	11
2	12	7	0	10	6
3	8	9	10	0	7
4	9	11	6	7	0

Penyelesaian

1. Setiap pelanggan dilayani terpisah

$$0 \rightarrow 1 \rightarrow 0 = 10 + 10 = 20$$

$$0 \rightarrow 2 \rightarrow 0 = 12 + 12 = 24$$

$$0 \rightarrow 3 \rightarrow 0 = 8 + 8 = 16$$

$$0 \rightarrow 4 \rightarrow 0 = 9 + 9 = 18$$

$$\text{Total jarak awal} = 20 + 24 + 16 + 18 = \mathbf{78}$$

2. Menghitung nilai saving

$$S_{12} = 10 + 12 - 7 = 15$$

$$S_{13} = 10 + 8 - 9 = 9$$

$$S_{14} = 10 + 9 - 11 = 8$$

$$S_{23} = 12 + 8 - 10 = 10$$

$$S_{24} = 12 + 9 - 6 = 15$$

$$S_{34} = 8 + 9 - 7 = 10$$

3. Mengurutkan nilai saving

Tabel 3 Urutan nilai saving

Urutan	Pasangan	Nilai Saving
1	1-2	15
2	2-4	15
3	2-3	10
4	3-4	10
5	1-3	9
6	1-4	8

4. Menggabungkan rute

Pasangan dengan nilai saving terbesar diprioritaskan untuk digabungkan terlebih dahulu.

Pasangan 1 – 2 memiliki nilai saving terbesar yaitu 15 dan untuk *Demand* pelanggan 1 dan 2 adalah

$4+3 = 7$, karena $7 \leq 10$ maka pelanggan 1 dan 2 dapat digabungkan menjadi :

$$0 \rightarrow 1 \rightarrow 2 \rightarrow 0$$

Selanjutnya pasangan 2 – 4 juga memiliki nilai saving 15

Demand rute sementara adalah

$4+3+2 = 9$, karena $9 \leq 10$ maka pelanggan 4 dapat digabungkan ke dalam rute tersebut

$$0 \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 0$$

Pelanggan 3 tidak dapat digabungkan ke rute tersebut karena total *Demand* menjadi 14 dan nilai tersebut melebihi kapasitas kendaraan. Oleh karena itu, pelanggan 3 tetap membentuk rute sendiri

$$0 \rightarrow 3 \rightarrow 0$$

Tabel 4 Hasil akhir rute

Rute	Urutan Rute	<i>Demand</i>	Jarak
1	$0 \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 0$	9	$10 + 7 + 6 + 9 = 32$
2	$0 \rightarrow 3 \rightarrow 0$	5	$8 + 8 = 16$

5. Menghitung hasil penghematan jarak

Diperoleh total jarak setelah Clarke-Wright Savings

$$32 + 16 = 48$$

Total penghematan jarak

$$78 - 48 = 30$$

Presentase penghematan jarak setelah dilakukan Clarke-Wright Savings

$$\frac{78-48}{78} \times 100 \% = 38,46 \%$$

1.6.6 Variable Neighborhood Search

Variable Neighborhood Search atau VNS adalah algoritma metaheuristic yang digunakan untuk menyelesaikan masalah optimasi, terutama pada permasalahan yang memiliki ruang solusi besar dan sulit diselesaikan dengan metode eksak. Mladenović dan Hansen menjelaskan bahwa VNS bekerja dengan cara mengubah struktur lingkungan pencarian secara sistematis untuk memperoleh solusi yang lebih baik (Mladenović & Hansen, 1997). Prinsip utama VNS adalah melakukan pencarian solusi dengan memanfaatkan beberapa struktur lingkungan atau *neighborhood* secara sistematis.

Gagasan dasar VNS adalah bahwa solusi optimum lokal pada satu lingkungan belum tentu menjadi optimum lokal pada lingkungan lainnya. Oleh karena itu, algoritma ini tidak hanya mencari solusi pada satu bentuk lingkungan, tetapi berpindah dari satu lingkungan ke lingkungan lain untuk menghindari jebakan optimum lokal. Dengan cara tersebut, VNS

memiliki peluang lebih besar untuk menemukan solusi yang lebih baik atau mendekati optimum global. Secara umum, tahapan VNS dimulai dari pembentukan solusi awal, penentuan beberapa struktur lingkungan, proses *shaking* untuk menghasilkan solusi baru, kemudian dilanjutkan dengan *local search* untuk memperbaiki solusi tersebut. Jika solusi baru lebih baik daripada solusi sebelumnya, maka solusi tersebut diterima dan pencarian dimulai kembali dari lingkungan pertama. Namun, jika tidak ditemukan perbaikan, algoritma akan berpindah ke lingkungan berikutnya sampai kriteria penghentian terpenuhi.

Menurut penelitian yang dilakukan oleh Hansen, P., & Mladenović, N. (2001) dan Hansen. (2017) metode VNS langkah-langkah sebagai berikut.

1. Menentukan Solusi Awal

Langkah pertama dalam VNS adalah membentuk solusi awal. Pada kasus GCVRP, solusi awal berupa susunan rute kendaraan yang mengunjungi seluruh pelanggan dengan tetap memperhatikan kapasitas kendaraan.

Contoh solusi awal:

$$S = \{R_1, R_2, R_3, \dots, R_m\}$$

dengan R_m menyatakan rute kendaraan ke- m .

2. Menentukan Struktur Neighborhood

Setelah solusi awal diperoleh, langkah berikutnya adalah menentukan beberapa struktur lingkungan atau *neighborhood*. Struktur ini digunakan untuk menghasilkan solusi baru dari solusi sebelumnya.

Contoh *neighborhood* yang dapat digunakan:

$$\begin{aligned} N_1 &= \text{swap} \\ N_2 &= \text{relocate} \\ N_3 &= \text{2-opt} \\ N_4 &= \text{exchange} \end{aligned}$$

Keterangan:

- Swap: menukar posisi dua pelanggan.
- Relocate: memindahkan satu pelanggan ke posisi lain.
- 2-opt: membalik urutan sebagian rute.
- Exchange: menukar pelanggan antar-rute.

3. Melakukan Proses Shaking

Tahap *shaking* dilakukan untuk menghasilkan solusi baru secara acak dari struktur *neighborhood* tertentu. Tujuannya adalah agar pencarian tidak terjebak pada solusi optimum lokal.

Misalnya, dari solusi awal S , dibentuk solusi baru:

$$S' \in N_k(S)$$

Artinya, solusi baru S' diperoleh dari lingkungan ke- k terhadap solusi S .

4. Melakukan Local Search

Solusi hasil *shaking* kemudian diperbaiki menggunakan *local search*. Pada tahap ini, algoritma mencari solusi yang lebih baik di sekitar solusi baru tersebut.

Hasil dari proses ini adalah solusi:

$$S''$$

Solusi S'' merupakan hasil perbaikan dari S' .

5. Mengevaluasi Solusi Baru

Setelah diperoleh solusi hasil *local search*, nilai fungsi tujuan dihitung. Pada GCVRP, fungsi tujuan biasanya berupa minimasi total biaya perjalanan, jarak, konsumsi bahan bakar, atau emisi karbon.

$$f(S) = \sum_{(i,j) \in S} c_{ij}$$

Solusi baru diterima apabila memiliki nilai fungsi tujuan yang lebih kecil dan tetap memenuhi kendala kapasitas kendaraan.

$$S = \begin{cases} S'', & \text{jika } f(S'') < f(S) \\ S, & \text{jika } f(S'') \geq f(S) \end{cases}$$

6. Mengulang dari Neighborhood Pertama Jika Ada Perbaikan

Jika solusi baru lebih baik daripada solusi sebelumnya, maka solusi tersebut diterima sebagai solusi terbaik sementara. Setelah itu, pencarian kembali dimulai dari struktur *neighborhood* pertama.

$$k = 1$$

Hal ini dilakukan agar pencarian kembali dimulai dari perubahan paling sederhana.

Berpindah ke Neighborhood berikutnya jika tidak ada Perbaikan

Jika solusi baru tidak lebih baik, maka algoritma berpindah ke struktur *neighborhood* berikutnya.

$$k = k + 1$$

Proses ini terus dilakukan sampai seluruh struktur *neighborhood* diperiksa.

7. Menghentikan Proses Berdasarkan Kriteria Penghentian

Algoritma berhenti apabila kriteria penghentian telah terpenuhi. Kriteria penghentian dapat berupa:

- jumlah iterasi maksimum tercapai;
- batas waktu komputasi tercapai;
- tidak ada perbaikan solusi dalam beberapa iterasi;
- seluruh struktur *neighborhood* telah diperiksa.

Berikut contoh studi kasus

Misalkan terdapat 1 depot dan 5 pelanggan. Depot diberi simbol 0, sedangkan pelanggan diberi simbol 1 sampai 5. Jumlah kendaraan yang digunakan adalah 2 kendaraan dengan kapasitas masing-masing sebesar 10 satuan.

Tabel 5 Permintaan pelanggan

Pelanggan	Demand
1	4
2	3
3	2
4	5
5	4

Koefisien emisi kendaraan diskenarioikan:

$$\alpha = 1$$

$$\beta = 0,05$$

Rumus emisi yang digunakan:

$$E_{ij} = d_{ij} (\alpha + \beta q_{ij})$$

Tabel 6 Matriks jarak pelanggan

Titik	0	1	2	3	4
0	0	4	6	5	7
1	4	0	3	4	6
2	6	3	0	2	5
3	5	4	2	0	4
4	7	6	5	4	0

Penyelesaian

1. Misalkan solusi awal yang diperoleh

$$S = R1, R2$$

dengan:

$$R_1 = 0 - 1 - 4 - 0$$

$$R_2 = 0 - 2 - 3 - 5 - 0$$

Kapasitas

$$R_1 = q_1 + q_4 = 4 + 5 = 9 \leq 10$$

$$R_2 = q_2 + q_3 + q_5 = 3 + 2 + 4 = 9 \leq 10$$

Hitung emisi

- Rute 1

Tabel 7 Emisi rute 1

Ruas Perjalanan	Jarak (d_{ij})	Muatan (q_{ij})	Emisi
0 – 1	4	9	$(4(1+0,05(9))) = 5,80$
1 – 4	6	5	$(6(1+0,05(5))) = 7,50$
4 – 0	7	0	$(7(1+0,05(0))) = 7$
Total Emisi Rute 1			20,30

- Rute 2

Tabel 8 Emisi rute 2

Ruas Perjalanan	Jarak (d_{ij})	Muatan (q_{ij})	Emisi
0 – 2	6	9	$(6(1+0,05(9))) = 8,70$
2 – 3	2	6	$(2(1+0,05(6))) = 2,60$
3 – 5	5	4	$(5(1+0,05(4))) = 6$
5 – 0	8	0	$(8(1+0,05(0))) = 8$
Total Emisi Rute 2			25,30

Total emisi solusi awal adalah:

$$f(S) = E(R_1) + E(R_2)$$

$$f(S) = 20,3 + 25,3 = 45,6$$

2. Neighborhood yang digunakan dalam contoh ini adalah:

$$N_1 = \text{swap}$$

$$N_2 = \text{relocate}$$

$$N_3 = \text{2-opt}$$

$$N_4 = \text{exchange}$$

3. Melakukan proses *shaking*

Misalkan digunakan neighborhood N_4 , yaitu exchange. Pada tahap ini dilakukan pertukaran pelanggan 1 pada rute 1 dengan pelanggan 5 pada rute 2.

Setelah pelanggan 1 dan pelanggan 5 ditukar, diperoleh solusi baru:

$$S' = \{R'_1, R'_2\}$$

dengan:

$$R'_1 = 0 - 5 - 4 - 0$$

$$R'_2 = 0 - 2 - 3 - 1 - 0$$

kapasitas:

$$R'_1 = q_5 + q_4 = 4 + 5 = 9 \leq 10$$

$$R'_2 = q_2 + q_3 + q_1 = 3 + 2 + 4 = 9 \leq 10$$

Karena kedua rute tetap tidak melebihi kapasitas kendaraan, maka solusi hasil shaking dinyatakan layak.

4. Melakukan *Local Search*

Melalui local search, urutan pelanggan diperbaiki agar emisi menjadi lebih rendah. Misalnya diperoleh solusi:

$$S'' = \{R_1'', R_2''\}$$

dengan:

$$R_1'' = 0 - 4 - 5 - 0$$

$$R_2'' = 0 - 1 - 2 - 3 - 0$$

Pemeriksaan kapasitas:

$$R_1'' = q_4 + q_5 = 5 + 4 = 9 \leq 10$$

$$R_2'' = q_1 + q_2 + q_3 = 4 + 3 + 2 = 9 \leq 10$$

Solusi hasil *local search* tetap layak karena tidak melanggar kapasitas kendaraan.

5. Setelah solusi hasil *local search* diperoleh, langkah berikutnya adalah menghitung nilai fungsi tujuan yaitu minimasi emisi.

Tabel 9 Perhitungan emisi rute 1 baru

Perhitungan emisi rute 1 baru

Ruas Perjalanan	Jarak (d_{ij})	Muatan (q_{ij})	Emisi
0 – 4	7	9	$(7(1+0,05(9))) = 10,15$
4 – 5	3	4	$(3(1+0,05(4))) = 3,60$
5 – 0	8	0	$(8(1+0,05(0))) = 8$
Total Emisi Rute 1 Baru			21,75

Tabel 10 Perhitungan emisi rute 2 baru

Ruas Perjalanan	Jarak (d_{ij})	Muatan (q_{ij})	Emisi
0 – 1	4	9	$(4(1+0,05(9))) = 5,8$
1 – 2	3	5	$(3(1+0,05(5))) = 3,75$
2 – 3	2	2	$(2(1+0,05(2))) = 2,2$
3 – 0	5	0	$(5(1+0,05(0))) = 5$
Total Emisi Rute 2 Baru			16,75

total emisi solusi baru adalah:

$$f(S'') = 21,75 + 16,75 = 38,5$$

Karena:

$$f(S'') < f(S)$$

$$38,5 < 45,6$$

maka solusi baru diterima sebagai solusi terbaik sementara.

Dengan demikian, solusi diperbarui menjadi:

$$S = S''$$

6. Mengulang dari Neighborhood pertama jika ada perbaikan
 Karena solusi baru lebih baik daripada solusi sebelumnya, maka algoritma kembali memulai pencarian dari neighborhood pertama.

$$k = 1$$

Hal ini dilakukan agar pencarian kembali dimulai dari perubahan yang paling sederhana. Dengan demikian, algoritma memiliki kesempatan untuk menemukan perbaikan lain dari solusi terbaru.

7. Berpindah ke Neighborhood berikutnya jika tidak ada perbaikan
 Apabila pada suatu neighborhood tidak ditemukan solusi yang lebih baik, maka algoritma berpindah ke neighborhood berikutnya.

$$k = k + 1$$

Misalnya, jika pada N_1 tidak ditemukan perbaikan, maka pencarian dilanjutkan ke N_2 . Jika pada N_2 juga tidak ada perbaikan, maka dilanjutkan ke N_3 , dan seterusnya.

Proses ini terus dilakukan sampai seluruh struktur neighborhood diperiksa atau sampai kriteria penghentian terpenuhi.

Dari proses VNS yang dilakukan diperoleh solusi rute

$$R_1'' = 0 - 4 - 5 - 0$$

$$R_2'' = 0 - 1 - 2 - 3 - 0$$

Dengan presentase penurunan emisi

$$\frac{45,60 - 38,50}{45,60} \times 100\% = 15,57\%$$

BAB II

METODOLOGI PENELITIAN

2.1 Metode penelitian

Metode yang digunakan dalam penelitian ini adalah metode kuantitatif dengan pendekatan eksperimen komputasi berbasis skenario pada permasalahan *Green Capacitated Vehicle Routing Problem* (GCVRP).

2.2 Jenis dan Sumber Data

Data yang digunakan dalam penelitian ini merupakan data sekunder, yaitu data yang tidak diperoleh melalui pengumpulan langsung di lapangan, tetapi berasal dari sumber yang telah tersedia. Data penelitian diperoleh dari situs resmi *Capacitated Vehicle Routing Problem Library* (CVRPLIB), yaitu basis data internasional yang menyediakan berbagai *benchmark dataset* untuk pengujian dan perbandingan metode penyelesaian *Capacitated Vehicle Routing Problem* (CVRP).

2.3 Transformasi Dataset CVRPLIB menjadi GCVRP

Dataset CVRPLIB yang memuat data dasar *Capacitated Vehicle Routing Problem*, seperti koordinat pelanggan, permintaan, kapasitas kendaraan, dan jumlah kendaraan dikembangkan dengan menambahkan atribut umur kendaraan, koefisien emisi yang disesuaikan dengan kondisi kendaraan, serta kapasitas kendaraan yang bersifat heterogen. Penambahan atribut tersebut menghasilkan dataset *Green Capacitated Vehicle Routing Problem* (GCVRP) yang dapat digunakan untuk menentukan rute distribusi dengan mempertimbangkan kapasitas kendaraan dan total emisi yang dihasilkan. Parameter tambahan tersebut meliputi tahun kendaraan, koefisien emisi dasar (α), dan koefisien emisi berdasarkan beban kendaraan (β). Emisi kendaraan pada setiap ruas perjalanan dihitung menggunakan rumus:

$$E_{ij} = (\alpha + \beta q_{ij}) d_{ij}$$

Dengan E_{ij} adalah emisi kendaraan pada ruas perjalanan dari titik (i) ke titik (j), d_{ij} adalah jarak dari titik (i) ke titik (j), α adalah koefisien emisi dasar kendaraan, β adalah koefisien emisi tambahan berdasarkan muatan, dan q_{ij} adalah muatan kendaraan saat melewati ruas perjalanan dari titik (i) ke titik (j). Dengan transformasi ini, dataset CVRPLIB dapat digunakan untuk menganalisis rute distribusi tidak hanya berdasarkan kapasitas dan jarak tempuh, tetapi juga berdasarkan total emisi kendaraan.

2.4 Prosedur Penelitian

Berikut prosedur penelitian yang dilakukan dalam penelitian ini yakni sebagai berikut.

1. Mengumpulkan Dataset CVRPLIB
Penelitian ini dimulai dengan mengumpulkan data CVRPLIB
2. Menambahkan parameter Green pada data CVRPLIB
Dataset CVRPLIB ditambahkan parameter Green sehingga dapat menjadi GCVRP

3. Membuat skenario
Skenario ditambahkan pada data-data yang telah dipilih dengan skenario berbeda yang digunakan sebagai dasar dalam penyelesaian masalah GCVRP
4. Menerapkan algoritma
Data yang telah dipilih akan diselesaikan dengan menggunakan algoritma Clarke-Wright Savings sebagai solusi awal dan dilanjutkan dengan algoritma Variable Neighborhood Search
5. Membandingkan dan menganalisa hasil
Setelah menerapkan kedua algoritma tersebut, hasil yang diperoleh akan dibandingkan berdasarkan cara kerja dan kompleksitasnya dalam menyelesaikan masalah GCVRP serta menganalisis hasil yang diperoleh dari 3 data dengan skenario yang berbeda untuk mengidentifikasi hal-hal yang menjadi kekurangan algoritma tersebut.
6. Menyimpulkan hasil penelitian
Peneliti akan menarik kesimpulan berdasarkan hasil perbandingan dan analisis yang telah diperoleh.

2.5 Pseudocode Algoritma

2.5.1 Pseudocode Clarke-Wright Savings

Input:

Data koordinat depot dan pelanggan, *Demand* pelanggan, kapasitas kendaraan, jumlah maksimum kendaraan.

Output:

Rute kendaraan hasil Clarke-Wright Savings, total *Demand*, total jarak, dan validasi kelayakan solusi.

Langkah-langkah:

1. Mulai.
2. Masukkan data depot, pelanggan, koordinat setiap titik, *Demand* pelanggan, kapasitas kendaraan, dan jumlah maksimum kendaraan.
3. Hitung jarak antar setiap pasangan titik menggunakan jarak Euclidean:
4. Bentuk matriks jarak (D_{ij}) untuk seluruh titik.
5. Bentuk solusi awal, yaitu setiap pelanggan dilayani oleh satu rute tersendiri
6. Hitung muatan awal setiap rute berdasarkan *Demand* pelanggan.
7. Hitung nilai saving untuk setiap pasangan pelanggan (i) dan (j). Urutkan seluruh nilai saving dari yang terbesar hingga terkecil.
8. Untuk setiap pasangan pelanggan (i) dan (j) berdasarkan urutan saving:
 - a. Periksa apakah pelanggan (i) dan (j) berada pada rute yang berbeda.

- b. Jika pelanggan (i) dan (j) berada pada rute yang sama, maka pasangan tersebut dilewati.
 - c. Periksa apakah total *Demand* dari dua rute yang akan digabungkan tidak melebihi kapasitas kendaraan.
 - d. Jika total *Demand* melebihi kapasitas kendaraan, maka penggabungan rute tidak dilakukan.
 - e. Jika total *Demand* tidak melebihi kapasitas kendaraan, periksa apakah pelanggan (i) dan (j) berada pada ujung rute.
 - f. Jika pelanggan berada pada ujung rute, gabungkan kedua rute sesuai posisi pelanggan.
 - g. Perbarui rute, total muatan, dan posisi pelanggan pada rute baru.
9. Setelah seluruh nilai saving diperiksa, diperoleh rute akhir hasil Clarke-Wright Savings.
10. Hitung total jarak setiap rute dengan menjumlahkan jarak dari depot ke pelanggan, antar pelanggan, dan kembali ke depot.
11. Hitung total jarak seluruh kendaraan.
12. Lakukan validasi solusi dengan memeriksa:
 - a. seluruh pelanggan dilayani tepat satu kali;
 - b. tidak ada pelanggan yang terlewat;
 - c. tidak ada pelanggan yang dilayani lebih dari satu kali;
 - d. muatan setiap kendaraan tidak melebihi kapasitas;
 - e. jumlah kendaraan yang digunakan tidak melebihi jumlah maksimum kendaraan.
14. Tampilkan hasil rute, kapasitas terpakai, total *Demand*, total jarak, dan hasil validasi.
15. Selesai.

2.5.2 Pseudocode Variable Neighborhood Search berbasis Emisi

Input:

Solusi awal, data pelanggan, *Demand*, kapasitas kendaraan, jarak antar titik, serta koefisien emisi kendaraan (α) dan (β).

Output:

Rute hasil optimasi VNS, total jarak, total emisi, dan validasi solusi.

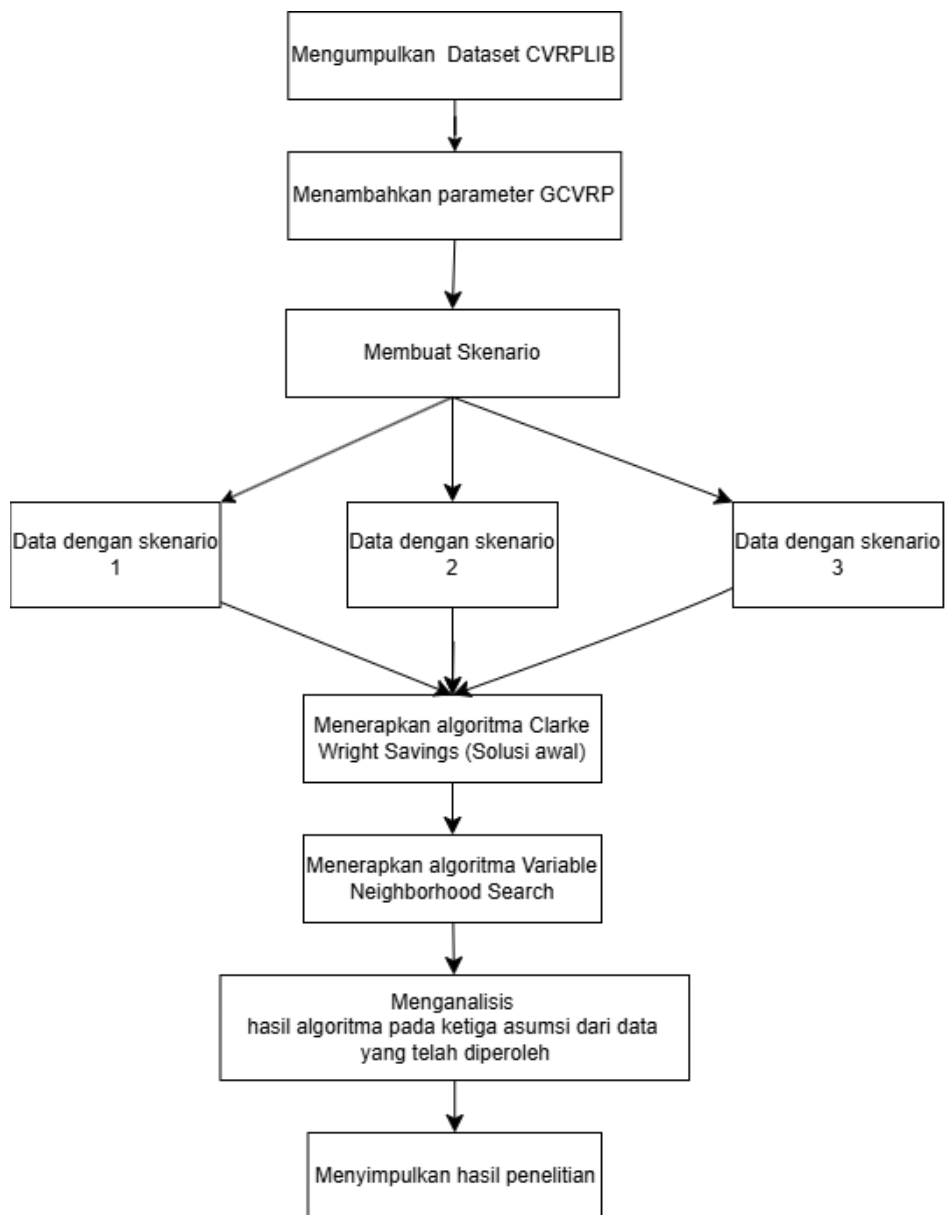
Langkah-langkah:

1. Mulai.
2. Masukkan solusi awal, data kendaraan, *Demand* pelanggan, kapasitas kendaraan, dan matriks jarak.

3. Hitung emisi setiap ruas perjalanan Tetapkan fungsi tujuan, yaitu meminimalkan total emisi kendaraan
5. Tetapkan solusi awal sebagai solusi terbaik sementara.
6. Tentukan struktur neighborhood yang digunakan, yaitu *two-opt*, *relocate*, *swap* dan *route exchange*
7. Lakukan local search pada solusi awal untuk memperoleh solusi terbaik sementara.
8. Untuk setiap iterasi sampai iterasi maksimum:
 - a. Tetapkan ($k=0$).
 - b. Selama ($k < k_{max}$), lakukan proses shaking menggunakan neighborhood ke- (k) .
 - c. Periksa kelayakan solusi hasil shaking berdasarkan kapasitas kendaraan dan keterlayanan pelanggan.
 - d. Jika solusi tidak layak, lanjutkan ke neighborhood berikutnya.
 - e. Jika solusi layak, lakukan local search untuk memperbaiki solusi tersebut.
 - f. Hitung total emisi solusi baru.
 - g. Jika total emisi solusi baru lebih kecil daripada solusi terbaik, maka solusi baru diterima sebagai solusi terbaik dan pencarian kembali dimulai dari ($k=0$).
 - h. Jika tidak terdapat perbaikan, lanjutkan ke neighborhood berikutnya.
9. Lanjutkan iterasi hingga kriteria penghentian tercapai, kemudian pilih solusi terbaik yang diperoleh. Pilih solusi dengan total emisi terkecil sebagai solusi akhir.
10. Hitung total jarak, total emisi, dan validasi solusi akhir.
11. Tampilkan rute hasil optimasi VNS.
12. Selesai.

2.6 Alur penelitian

Diagram alur pada penelitian ini digambarkan secara skematik sebagai berikut.



BAB III

HASIL DAN PEMBAHASAN

Dalam bab ini membahas dan menguraikan terkait hasil penelitian mengenai penerapan algoritma Clarke-Wright Savings dan algoritma Variable Neighborhood Search. Kedua algoritma tersebut diterapkan pada 3 data dengan 3 skenario yang berbeda. Setelah ketiga skenario dalam data tersebut diselesaikan, hasil yang diperoleh akan dibandingkan.

3.1 Hasil

3.1.1 Deskripsi data

Penelitian ini menggunakan tiga data Capacitated Vehicle Routing Problem (CVRP), yaitu data 32 titik, data 60 titik, dan data 80 titik. Ketiga data tersebut digunakan untuk menguji penyelesaian masalah rute distribusi menggunakan metode Clarke-Wright Savings (CWS) sebagai pembentuk solusi awal dan Variable Neighborhood Search (VNS) sebagai metode optimasi lanjutan. Setiap data terdiri atas satu depot sebagai titik awal dan akhir perjalanan, serta sejumlah pelanggan yang harus dilayani oleh kendaraan dengan memperhatikan batas kapasitas muatan.

Data pertama adalah data 32 titik, dengan depot berada pada titik 1 dan jumlah pelanggan sebanyak 31 titik. Total permintaan seluruh pelanggan pada data ini adalah 410 satuan. Pada skenario kapasitas seragam, data ini menggunakan 5 kendaraan dengan kapasitas masing-masing 100 satuan, sehingga total kapasitas kendaraan sebesar 500 satuan. Dengan demikian, total kapasitas kendaraan lebih besar daripada total permintaan pelanggan, sehingga data ini layak diselesaikan menggunakan model CVRP. Data ini digunakan sebagai data berukuran kecil untuk melihat kemampuan awal metode dalam membentuk rute yang memenuhi kendala kapasitas.

Data kedua adalah data 60 titik, dengan depot berada pada titik 1 dan jumlah pelanggan sebanyak 59 titik. Total permintaan seluruh pelanggan pada data ini adalah 829 satuan. Pada skenario kendaraan seragam, data ini menggunakan 9 kendaraan dengan kapasitas masing-masing 100 satuan, sehingga total kapasitas kendaraan sebesar 900 satuan. Pada skenario kapasitas berbeda, kendaraan memiliki kapasitas yang tidak sama, yaitu disesuaikan dengan karakteristik masing-masing kendaraan. Data ini digunakan sebagai data berukuran sedang untuk menguji kemampuan metode dalam menangani jumlah pelanggan yang lebih banyak dan distribusi permintaan yang lebih kompleks.

Data ketiga adalah data 80 titik, dengan depot berada pada titik 1 dan jumlah pelanggan sebanyak 79 titik. Total permintaan seluruh pelanggan pada data ini adalah 942 satuan. Pada skenario kendaraan seragam, data ini menggunakan 10 kendaraan dengan kapasitas masing-masing 100 satuan, sehingga total kapasitas kendaraan sebesar 1000 satuan. Pada skenario kapasitas berbeda, kendaraan memiliki kapasitas yang bervariasi, yaitu 120, 115, 110, 105, 100, 100, 95, 90, 85, dan 80 satuan. Total kapasitas kendaraan pada skenario tersebut adalah 1000 satuan, sehingga masih mencukupi untuk melayani

seluruh permintaan pelanggan. Data ini digunakan sebagai data berukuran besar untuk menguji kestabilan dan kemampuan metode dalam menyelesaikan permasalahan rute dengan jumlah pelanggan yang lebih banyak.

Tabel 11 Karakteristik data

Data	Jumlah Titik	Depot	Jumlah Pelanggan	Jumlah Kendaraan	Kapasitas Standar	Total Demand
A-n32-k5	32	1	31	5	100	410
A-n60-k9	60	1	59	9	100	829
A-n80-k10	80	1	79	10	100	942

Berdasarkan tabel tersebut, ketiga data memiliki tingkat kompleksitas yang berbeda. Data 32 memiliki jumlah pelanggan paling sedikit sehingga digunakan sebagai data awal untuk melihat pola kerja metode. Data 60 memiliki jumlah pelanggan yang lebih besar sehingga mampu menunjukkan kemampuan metode dalam menangani permasalahan dengan kompleksitas sedang. Data 80 merupakan data dengan jumlah pelanggan paling banyak dan total permintaan paling besar, sehingga digunakan untuk menguji performa metode pada permasalahan yang lebih kompleks. Dengan adanya tiga ukuran data tersebut, hasil penelitian dapat menunjukkan kinerja metode CWS dan VNS pada kondisi data yang berbeda.

Tabel 12 Parameter Eksperimen

Parameter	Nilai atau ketentuan
Dataset	A-n32-k5, A-n60-k9, dan A-n80-k10
Jumlah skenario	Tiga skenario
Skenario 1	Kapasitas dan koefisien emisi kendaraan homogen
Skenario 2	Kapasitas sama dengan koefisien emisi berbeda
Skenario 3	Kapasitas dan koefisien emisi berbeda
Perhitungan jarak	Jarak Euclidean yang dibulatkan ke bilangan bulat terdekat
Solusi awal	Hasil metode <i>Clarke-Wright Savings</i>
Fungsi tujuan VNS	Meminimalkan total nilai emisi kendaraan
Bobot jarak (x)	0
Bobot emisi (y)	1
Iterasi maksimum	2500 iterasi
<i>Random seed</i>	3
Jumlah percobaan	Satu kali untuk setiap dataset dan skenario

Lanjutan tabel...

Parameter	Nilai atau ketentuan
<i>Neighborhood</i> skenario 1	<i>Two-opt, relocate, dan swap</i>
Kmax skenario 1	3
<i>Neighborhood</i> skenario 2 dan 3	<i>Route exchange, two-opt, relocate, dan swap</i>
Kmax skenario 2 dan 3	4
Proses <i>shaking</i>	Perubahan solusi secara acak berdasarkan struktur <i>neighborhood</i> yang sedang digunakan
<i>Local search</i>	Pencarian perbaikan sampai tidak ditemukan solusi dengan emisi yang lebih rendah
Kriteria penerimaan	Solusi baru diterima apabila total emisinya lebih rendah dan tetap memenuhi seluruh kendala
Kriteria penghentian	Iterasi maksimum tercapai dan tidak ditemukan perbaikan pada struktur <i>neighborhood</i> yang diperiksa
Kendala kapasitas	Total permintaan pada setiap rute tidak melebihi kapasitas kendaraan
Kendala pelayanan	Setiap pelanggan dilayani tepat satu kali
Jumlah kendaraan	Tidak melebihi jumlah kendaraan yang tersedia pada setiap dataset
Keluaran eksperimen	Rute kendaraan, muatan, total jarak, total emisi, dan validasi kelayakan solusi
Perangkat lunak	Python 3 dengan modul math, random, dan matplotlib

Koefisien emisi pada setiap skenario ditetapkan sebagai parameter penelitian untuk membentuk beberapa kondisi pengujian. Perbedaan nilai koefisien emisi digunakan untuk mengevaluasi kinerja algoritma dalam menangani kendaraan dengan karakteristik emisi yang berbeda.. Kendaraan dengan tahun keluaran lebih baru diskenariokan memiliki koefisien lebih rendah, sedangkan kendaraan dengan tahun keluaran lebih lama diskenariokan memiliki koefisien lebih tinggi.

3.1.2 Implementasi Algoritma

3.1.2.1 Solusi awal dengan Clarke-Wright Savings

Hasil Clarke-Wright Savings skenario 1 dan 2 dari data A-n32-k5

Tabel 13 Hasil Clarke-Wright savings pada data A-n32-k5 dengan skenario 1 dan 2

Kendaraan	Rute	Kapasitas	Jarak
1	1 → 24 → 4 → 3 → 18 → 20 → 32 → 22 → 1	99/100	209
2	1 → 27 → 7 → 19 → 29 → 5 → 12 → 9 → 10 → 23 → 15 → 1	92/100	258
3	1 → 21 → 6 → 26 → 11 → 16 → 30 → 28 → 1	91/100	195
4	1 → 17 → 8 → 14 → 2 → 13 → 1	90/100	112
5	1 → 25 → 31 → 1	38/100	65
Total	-	410	839

Berdasarkan tabel tersebut, metode Clarke-Wright Savings menghasilkan 5 rute kendaraan untuk melayani seluruh pelanggan pada data 32. Seluruh kendaraan berangkat dari depot, yaitu titik 1, kemudian mengunjungi pelanggan sesuai urutan rute masing-masing dan kembali lagi ke depot. Total *Demand* yang dilayani adalah 410 satuan, sedangkan total jarak tempuh seluruh kendaraan adalah 839.

Hasil validasi menunjukkan bahwa seluruh pelanggan telah dilayani tepat satu kali dan tidak terdapat pelanggan yang terlewat maupun dilayani lebih dari satu kali. Selain itu, setiap kendaraan masih berada dalam batas kapasitas maksimum 100 satuan. Kendaraan 1 membawa muatan sebesar 99/100, kendaraan 2 sebesar 92/100, kendaraan 3 sebesar 91/100, kendaraan 4 sebesar 90/100, dan kendaraan 5 sebesar 38/100. Dengan demikian, solusi awal yang dihasilkan oleh metode Clarke-Wright Savings dinyatakan layak karena memenuhi kendala kapasitas dan keterlayanan pelanggan.

Jika dilihat dari pemanfaatan kapasitas, kendaraan 1 memiliki tingkat pemanfaatan kapasitas tertinggi, yaitu 99 satuan dari kapasitas maksimum 100 satuan. Sementara itu, kendaraan 5 memiliki pemanfaatan kapasitas paling rendah, yaitu 38 satuan dari kapasitas maksimum 100 satuan. Perbedaan ini menunjukkan bahwa pembentukan rute pada metode Clarke-Wright Savings lebih mengutamakan penghematan jarak berdasarkan nilai saving, selama tidak melanggar batas kapasitas kendaraan.

Hasil Clarke-Wright Savings dengan Kapasitas dan Koefisien Emisi Berbeda pada data A-n32-k5

Pada skenario ini, metode Clarke-Wright Savings diterapkan dengan kondisi kendaraan yang memiliki kapasitas dan koefisien emisi berbeda. Perbedaan kapasitas menyebabkan pembentukan rute harus disesuaikan dengan kemampuan angkut masing-masing kendaraan. Selain itu, perbedaan koefisien emisi menyebabkan nilai emisi setiap kendaraan tidak hanya dipengaruhi oleh jarak dan beban, tetapi juga oleh karakteristik kendaraan yang digunakan.

Tabel 14 Data Kendaraan A-n32-k5 dengan skenario 3

Kendaraan	Tahun	Kapasitas	Koefisien Emisi Dasar (α)	Koefisien Muatan (β)
1	2024	100	0,150	0,0050
2	2021	90	0,165	0,0055
3	2018	90	0,185	0,0062
4	2014	80	0,215	0,0072
5	2010	70	0,250	0,0085
Total	-	430	-	-

Tabel 15 Hasil Clarke-Wright Savings data A-n32-k5 dengan skenario 3

Kendaraan	Tahun	Rute	<i>Demand/</i> Kapasitas	Jarak	Emisi
1	2024	1 → 24 → 4 → 3 → 18 → 20 → 32 → 22 → 1	99/100	209	89,0550
2	2021	1 → 7 → 19 → 29 → 5 → 12 → 9 → 10 → 23 → 15 → 1	90/90	256	104,7145
3	2018	1 → 17 → 8 → 14 → 2 → 13 → 1	90/90	112	52,3896
4	2014	1 → 21 → 6 → 26 → 11 → 16 → 30 → 1	71/80	194	85,4212
5	2010	1 → 28 → 25 → 27 → 31 → 1	60/70	77	38,7830
Total	-	-	410/430	848	370,3633

Dari pemanfaatan kapasitas, kendaraan 2 dan kendaraan 3 terisi penuh sebesar 90/90, sedangkan kendaraan 1 hampir penuh sebesar 99/100. Kendaraan 4 dan kendaraan 5

masih memiliki sisa kapasitas, tetapi seluruhnya tetap berada dalam batas aman. Hal ini menunjukkan bahwa CWS mampu membentuk rute sesuai batas kapasitas kendaraan yang berbeda. Sedangkan dari sisi emisi, kendaraan 2 menghasilkan emisi terbesar, yaitu 104,7145, karena dipengaruhi oleh kombinasi jarak, beban, dan koefisien emisi. Sementara itu, kendaraan 5 memiliki koefisien emisi tertinggi, tetapi emisinya tidak menjadi yang terbesar karena jarak dan *Demand* yang dilayani relatif kecil.

Hasil Clarke-Wright Savings skenario 1 dan 2 dari data A-n60-k9

Tabel 16 Hasil Clarke-Wright Savings data A-n60-k9 dengan skenario 1 dan 2

Kendaraan	Rute	Kapasitas	Jarak
1	1 → 50 → 31 → 37 → 23 → 49 → 2 → 3 → 1	100/100	219
2	1 → 59 → 25 → 29 → 45 → 54 → 32 → 1	100/100	173
3	1 → 56 → 16 → 27 → 28 → 18 → 38 → 58 → 30 → 1	100/100	153
4	1 → 51 → 44 → 57 → 13 → 10 → 52 → 33 → 1	97/100	238
5	1 → 36 → 40 → 9 → 14 → 8 → 20 → 34 → 1	97/100	149
6	1 → 19 → 39 → 60 → 53 → 24 → 48 → 1	97/100	118
7	1 → 43 → 46 → 6 → 55 → 11 → 1	94/100	189
8	1 → 4 → 47 → 41 → 12 → 5 → 22 → 7 → 35 → 15 → 1	93/100	128
9	1 → 42 → 17 → 21 → 26 → 1	51/100	53
Total	-	829	1420

Perbedaan jarak antar kendaraan terjadi karena jumlah pelanggan, letak koordinat pelanggan, dan urutan kunjungan pada setiap rute berbeda. Kendaraan dengan jumlah pelanggan lebih banyak belum tentu selalu menghasilkan jarak terbesar, karena jarak juga dipengaruhi oleh sebaran lokasi pelanggan terhadap depot dan terhadap pelanggan lain dalam rute yang sama.

Hasil validasi menunjukkan bahwa seluruh pelanggan telah dilayani tepat satu kali. Tidak terdapat pelanggan yang terlewat maupun pelanggan yang dilayani lebih dari satu kali. Selain itu, seluruh kendaraan memiliki muatan yang tidak melebihi kapasitas maksimum 100 satuan. Dengan demikian, solusi yang dihasilkan oleh metode Clarke-Wright Savings pada data A-n60-k9 dinyatakan layak atau feasible..

Hasil Clarke-Wright Savings dengan Kapasitas dan Koefisien Emisi Berbeda pada data A-n60-k9

Pada skenario 3, metode Clarke-Wright Savings diterapkan dengan kondisi kendaraan yang memiliki kapasitas dan koefisien emisi berbeda. Perbedaan kapasitas menunjukkan bahwa setiap kendaraan memiliki kemampuan angkut yang tidak sama, sedangkan perbedaan koefisien emisi menunjukkan bahwa setiap kendaraan menghasilkan tingkat emisi yang berbeda. Dalam skenario ini, kendaraan dengan tahun keluaran lebih baru memiliki koefisien emisi lebih kecil, sedangkan kendaraan dengan tahun keluaran lebih lama memiliki koefisien emisi lebih besar.

Tabel 17 Hasil Clarke-Wright Savings Skenario 3 pada Data A-n60-k9

Kendaraan	Tahun	Rute	Demand/ Kapasitas	Jarak	Koefisien Emisi	Emisi
1	2024	1 → 43 → 46 →	114/120	189	$\alpha = 0,150$	83,025
		6 → 55 → 11 →			$\beta = 0,0050$	0
		23 → 1				
2	2023	1 → 20 → 27 →	109/110	153	$\alpha = 0,155$	69,709
		28 → 18 → 38 →			$\beta = 0,0052$	0
		58 → 30 → 14 →				
3	2021	1	102/105	246	$\alpha = 0,165$	98,906
		1 → 51 → 44 →			$\beta = 0,0055$	5
		57 → 13 → 10 →				
4	2019	52 → 33 → 16 →	98/100	217	$\alpha = 0,175$	105,96
		1			$\beta = 0,0059$	66
		1 → 45 → 50 →				
5	2018	31 → 37 → 49 →	94/95	167	$\alpha = 0,185$	78,232
		2 → 3 → 1			$\beta = 0,0062$	0
		1 → 59 → 25 →				
6	2016	29 → 32 → 54 →	86/90	111	$\alpha = 0,200$	62,085
		22 → 4 → 1			$\beta = 0,0067$	1
		1 → 35 → 7 → 5				
		→ 12 → 41 → 47				
		→ 26 → 1				

Lanjutan tabel...

Kendaraan	Tahun	Rute	Demand/ Kapasitas	Jarak	Koefisien Emisi	Emisi
7	2014	1 → 42 → 34 → 39 → 60 → 53 → 19 → 1	84/85	80	$\alpha = 0,215$ $\beta = 0,0072$	37,388 8
8	2012	1 → 8 → 9 → 40 → 56 → 36 → 1	77/80	144	$\alpha = 0,235$ $\beta = 0,0078$	77,169 0
9	2010	1 → 17 → 21 → 24 → 48 → 15 → 1	65/75	100	$\alpha = 0,280$ $\beta = 0,0085$	58,116 0
Total	-	-	829	1407	-	670,59 80

Dari sisi kapasitas, sebagian besar kendaraan memiliki tingkat pemanfaatan yang tinggi. Kendaraan dengan kapasitas besar digunakan untuk melayani *Demand* yang lebih besar, seperti kendaraan 1 dengan *Demand* 114/120 dan kendaraan 2 dengan *Demand* 109/110. Dari sisi jarak, rute terpanjang terdapat pada kendaraan 3 sebesar 246, sedangkan rute terpendek terdapat pada kendaraan 7 sebesar 80. Dari sisi emisi, kendaraan 4 menghasilkan emisi terbesar sebesar 105,9666, meskipun bukan rute dengan jarak paling panjang. Hal ini menunjukkan bahwa emisi tidak hanya dipengaruhi oleh jarak, tetapi juga oleh beban dan koefisien emisi kendaraan.

Validasi solusi menunjukkan bahwa seluruh pelanggan dilayani tepat satu kali dan tidak ada kendaraan yang melebihi kapasitas. Dengan demikian, solusi CWS pada skenario 3 untuk data A-n60-k9 dinyatakan layak dan dapat digunakan sebagai solusi awal untuk optimasi lanjutan menggunakan Variable Neighborhood Search.

Hasil Clarke-Wright Savings skenario 1 dan 2 pada data A-n80-k10

Tabel 18 Hasil Clarke-Wright Savings Skenario 1 dan 2 pada Data A-n80-k10

Kendaraan	Rute	Kapasitas	Jarak
1	1 → 72 → 35 → 3 → 38 → 9 → 79 → 31 → 7 → 25 → 1	100/100	222
2	1 → 73 → 55 → 10 → 56 → 16 → 42 → 26 → 76 → 21 → 47 → 1	100/100	248

Lanjutan tabel...

Kendaraan	Rute	Kapasitas	Jarak
3	1 → 34 → 48 → 57 → 70 → 66 → 36 → 27 → 20 → 58 → 62 → 17 → 44 → 69 → 1	100/100	307
4	1 → 11 → 64 → 24 → 45 → 13 → 30 → 75 → 1	99/100	159
5	1 → 12 → 53 → 29 → 80 → 19 → 49 → 15 → 1	99/100	192
6	1 → 14 → 43 → 37 → 54 → 67 → 68 → 39 → 1	99/100	128
7	1 → 6 → 28 → 60 → 32 → 18 → 65 → 61 → 40 → 1	99/100	184
8	1 → 59 → 33 → 5 → 23 → 46 → 51 → 77 → 71 → 1	99/100	195
9	1 → 2 → 8 → 63 → 4 → 78 → 52 → 1	96/100	136
10	1 → 50 → 74 → 22 → 41 → 1	51/100	93
Total	-	942	1864

Berdasarkan hasil Clarke-Wright Savings pada data A-n80-k10, diperoleh 10 rute kendaraan dengan total jarak 1864 dan total *Demand* 942 dari total kapasitas 1000. Seluruh kendaraan digunakan dan tidak ada rute yang melebihi kapasitas, sehingga seluruh permintaan pelanggan dapat dilayani.

Jarak terbesar terdapat pada kendaraan 3, yaitu 307, sedangkan jarak terkecil terdapat pada kendaraan 10, yaitu 93. Hasil validasi menunjukkan bahwa seluruh pelanggan dilayani tepat satu kali dan seluruh kendaraan berada dalam batas kapasitas. Dengan demikian, solusi CWS pada data A-n80-k10 dinyatakan layak atau feasible.

Hasil Clarke-Wright Savings dengan Kapasitas dan Koefisien Emisi Berbeda pada data A-n80-k10

Tabel 19 Hasil Clarke-Wright Savings Data 80 Skenario 3

Kendaraan	Tahun	Rute	<i>Demand</i> / Kapasitas	Jarak	Koefisien Emisi	Emisi
1	2024	1 → 26 → 48 → 57 → 70 → 66 → 36 → 27 → 20 → 58 → 62 → 17 → 44 → 69 → 9 → 1	120/120	314	$\alpha = 0,150$ $\beta = 0,0050$	145,4 150

Lanjutan tabel...

Kendaraan	Tahun	Rute	Demand/ Kapasitas	Jarak	Koefisien Emisi	Emisi
2	2023	1 → 12 → 35 → 3 → 38 → 79 → 76 → 21 → 60 → 28 → 30 → 1	111/115	222	$\alpha = 0,155$ $\beta = 0,0052$	100,6 528
3	2022	1 → 78 → 61 → 65 → 34 → 55 → 10 → 56 → 16 → 42 → 47 → 32 → 18 → 1	109/110	234	$\alpha = 0,160$ $\beta = 0,0054$	110,7 180
4	2021	1 → 59 → 33 → 5 → 23 → 46 → 51 → 73 → 77 → 71 → 1	101/105	201	$\alpha = 0,165$ $\beta = 0,0056$	95,48 18
5	2020	1 → 72 → 53 → 29 → 80 → 19 → 49 → 15 → 1	97/100	191	$\alpha = 0,175$ $\beta = 0,0059$	82,51 30
6	2018	1 → 63 → 64 → 11 → 8 → 2 → 1	99/100	132	$\alpha = 0,185$ $\beta = 0,0062$	72,53 20
7	2016	1 → 13 → 45 → 6 → 31 → 7 → 25 → 24 → 1	89/95	159	$\alpha = 0,200$ $\beta = 0,0067$	76,81 73
8	2014	1 → 43 → 52 → 4 → 40 → 75 → 1	89/90	122	$\alpha = 0,215$ $\beta = 0,0072$	58,00 36
9	2012	1 → 37 → 54 → 67 → 68 → 39 → 74 → 1	76/85	117	$\alpha = 0,235$ $\beta = 0,0078$	61,77 60
10	2010	1 → 50 → 14 → 22 → 41 → 1	51/80	93	$\alpha = 0,250$ $\beta = 0,0085$	43,05 50
Total	-	-	942	1785	-	846,9 645

Berdasarkan hasil Clarke-Wright Savings pada data 80 skenario 3, diperoleh 10 rute kendaraan dengan total *Demand* 942, total jarak 1785, dan total emisi 846,9645. Seluruh kendaraan digunakan dan setiap rute masih berada dalam batas kapasitas masing-masing, sehingga 30olusi yang dihasilkan dinyatakan layak.

Dari sisi kapasitas, sebagian besar kendaraan memiliki pemanfaatan yang tinggi. Kendaraan 1 terisi penuh sebesar 120/120, sedangkan kendaraan lain juga mendekati kapasitas maksimumnya. Hal ini menunjukkan bahwa CWS mampu membentuk rute

dengan menyesuaikan perbedaan kapasitas kendaraan. Dari sisi jarak dan emisi, kendaraan 1 memiliki jarak terbesar, yaitu 314, sekaligus menghasilkan emisi tertinggi sebesar 145,4150. Sementara itu, kendaraan 10 memiliki jarak dan emisi terendah, yaitu 93 dan 43,0550. Hasil ini menunjukkan bahwa emisi tidak hanya dipengaruhi oleh koefisien kendaraan, tetapi juga oleh jarak tempuh dan beban rute.

Validasi solusi menunjukkan bahwa seluruh pelanggan dilayani tepat satu kali dan tidak terdapat kendaraan yang melebihi kapasitas. Dengan demikian, solusi CWS pada data 80 skenario 3 dapat digunakan sebagai solusi awal untuk optimasi lanjutan menggunakan Variable Neighborhood Search.

3.1.2.2 Optimasi emisi dengan Variable Neighborhood Search

Hasil Implementasi Variable Neighborhood Search Data 32 skenario 1

Berdasarkan hasil implementasi VNS, diperoleh 5 rute kendaraan dengan total *Demand* sebesar 410. Seluruh kendaraan memiliki kapasitas maksimum 100 satuan. Total jarak yang dihasilkan oleh VNS adalah 839, sedangkan total emisi yang diperoleh adalah 268,760. Nilai objektif pada hasil VNS sama dengan total emisi, yaitu 268,760, karena fungsi tujuan yang digunakan adalah meminimalkan total emisi kendaraan.

Tabel 20 Hasil Implementasi Variable Neighborhood Search

Kendaraan	Rute	Kapasitas	Jarak	Koefisien Emisi	Emisi
1	1 → 27 → 8 → 14 → 18 → 20 → 32 → 22 → 1	98/100	155	$\alpha = 0,150$ $\beta = 0,0050$	56.015
2	1 → 7 → 4 → 3 → 24 → 29 → 5 → 12 → 30 → 1	97/100	254	$\alpha = 0,150$ $\beta = 0,0050$	85.930
3	1 → 21 → 6 → 26 → 11 → 16 → 23 → 10 → 9 → 19 → 1	96/100	244	$\alpha = 0,150$ $\beta = 0,0050$	85.030
4	1 → 31 → 17 → 2 → 13 → 1	72/100	73	$\alpha = 0,150$ $\beta = 0,0050$	22.360
5	1 → 28 → 25 → 15 → 1	47/100	64	$\alpha = 0,150$ $\beta = 0,0050$	16.835
Total	-	410	790		266.170

Jika dibandingkan dengan solusi awal Clarke-Wright Savings, total jarak hasil VNS tidak mengalami perubahan. Total jarak CWS adalah 839 dan total jarak VNS adalah 790. Dengan demikian, terdapat penurunan jarak tempuh atau persentase penurunan jarak sebesar 5,84%. Kemudian dari sisi emisi, VNS mampu memberikan perbaikan yang cukup besar. Total emisi pada solusi awal CWS adalah 320,685, sedangkan total emisi hasil VNS adalah 266.170. Dengan demikian, terjadi penurunan emisi sebesar 17 %

Hasil Implementasi Variable Neighborhood Search dengan Koefisien Emisi Berbeda data 32 skenario 2

Hasil optimasi menggunakan Variable Neighborhood Search menghasilkan 5 rute kendaraan dengan total *Demand* sebesar 410. Seluruh pelanggan berhasil dilayani tepat satu kali dan tidak terdapat kendaraan yang melebihi kapasitas maksimum. Total jarak yang diperoleh dari hasil VNS adalah 817, sedangkan total emisi yang dihasilkan adalah 316,2780. Nilai objektif pada hasil ini sama dengan total emisi, yaitu 316,2780, karena fungsi tujuan yang digunakan adalah meminimalkan total emisi kendaraan.

Tabel 21 Hasil Optimasi Variable Neighborhood Search

Kendaraan	Tahun	Rute	Kapasitas	Jarak	Koefisien Emisi	Emisi
1	2024	1 → 28 → 29	95/100	246	$\alpha = 0,150$ $\beta = 0,0050$	83.4700
		→ 5 → 12 →				
		9 → 10 → 23				
		→ 19 → 1				
2	2021	1 → 21 → 6	71/100	194	$\alpha = 0,165$ $\beta = 0,0055$	65.4005
		→ 26 → 11 →				
		16 → 30 → 1				
3	2018	1 → 27 → 8	98/100	155	$\alpha = 0,185$ $\beta = 0,0062$	69.3036
		→ 14 → 18 →				
		20 → 32 →				
		22 → 1				
4	2014	1 → 25 → 15	74/100	171	$\alpha = 0,215$ $\beta = 0,0072$	68.1714
		→ 7 → 4 → 3				
		→ 24 → 1				
5	2010	1 → 31 → 17	72/100	73	$\alpha = 0,250$ $\beta = 0,0085$	37.6470
		→ 2 → 13 →				
		1				
Total	-	-	410	839		323.9925

Berdasarkan tabel tersebut, metode VNS menghasilkan 5 rute yang seluruhnya memenuhi kapasitas kendaraan dengan total permintaan 410 unit. Solusi yang diperoleh memiliki total jarak 839 dan total emisi 323,9925, serta seluruh pelanggan berhasil dilayani tepat satu kali sehingga solusi dinyatakan feasible. Selain itu, VNS mampu menurunkan total emisi dibandingkan solusi awal.

Hasil Implementasi Variable Neighborhood Search Data 32 Skenario 3

Pada skenario 3, metode Variable Neighborhood Search diterapkan untuk mengoptimalkan solusi awal Clarke-Wright Savings dengan mempertimbangkan kapasitas dan koefisien emisi yang berbeda pada setiap kendaraan. Tujuan utama dari optimasi ini adalah menurunkan total emisi kendaraan, dengan tetap menjaga agar seluruh pelanggan dilayani tepat satu kali dan setiap rute tidak melebihi kapasitas kendaraan yang digunakan.

Tabel 22 Hasil Optimasi VNS Data 32 Skenario 3

Kendaraan	Tahun	Rute	Demand/ Kapasitas	Jarak	Koefisien Emisi	Emisi
1	2024	1 → 22 → 32 → 20 → 18 → 3 → 4 → 24 → 1	99/100	209	$\alpha = 0,150$ $\beta = 0,0050$	77,1000
2	2021	1 → 27 → 7 → 29 → 5 → 12 → 9 → 10 → 23 → 19 → 1	89/90	248	$\alpha = 0,165$ $\beta = 0,0055$	93,9070
3	2018	1 → 17 → 13 → 2 → 8 → 14 → 1	90/90	122	$\alpha = 0,185$ $\beta = 0,0062$	48,0396
4	2014	1 → 21 → 6 → 26 → 11 → 16 → 30 → 1	71/80	194	$\alpha = 0,215$ $\beta = 0,0072$	85,4212
5	2010	1 → 28 → 25 → 15 → 31 → 1	61/70	77	$\alpha = 0,250$ $\beta = 0,0085$	38,8085
Total	-	-	410	850		343,2763

Berdasarkan tabel tersebut, hasil optimasi VNS menghasilkan 5 rute kendaraan dengan total *demand* sebesar 410. Setiap kendaraan memiliki kapasitas yang berbeda, yaitu kendaraan 1 sebesar 100, kendaraan 2 sebesar 90, kendaraan 3 sebesar 90, kendaraan 4 sebesar 80, dan kendaraan 5 sebesar 70. Seluruh rute yang terbentuk masih berada dalam batas kapasitas kendaraan, sehingga solusi yang dihasilkan dinyatakan layak.

Kendaraan 1 melayani rute dengan *demand* 99 dari kapasitas 100 dan jarak tempuh 209. Kendaraan 2 melayani *demand* 89 dari kapasitas 90 dengan jarak 248. Kendaraan 3

membawa *demand* penuh sebesar 90 dari kapasitas 90 dengan jarak 122. Kendaraan 4 membawa *demand* 71 dari kapasitas 80 dengan jarak 194. Kendaraan 5 membawa *demand* 61 dari kapasitas 70 dengan jarak 77. Hasil ini menunjukkan bahwa VNS mampu menyusun kembali rute dengan tetap menyesuaikan batas kapasitas kendaraan yang berbeda.

Dari sisi emisi, kendaraan 2 menghasilkan emisi terbesar, yaitu 93,9070. Hal ini dipengaruhi oleh jarak tempuh yang paling besar, yaitu 248, serta *demand* yang hampir penuh, yaitu 89 dari kapasitas 90. Kendaraan 4 menghasilkan emisi sebesar 85,4212 meskipun *demand* yang dibawa lebih kecil daripada kendaraan 2 dan kendaraan 3. Kondisi ini terjadi karena kendaraan 4 memiliki koefisien emisi lebih besar dan menempuh jarak yang cukup panjang. Sementara itu, emisi terkecil terdapat pada kendaraan 5, yaitu sebesar 38,8085, karena kendaraan tersebut memiliki jarak tempuh yang paling pendek, yaitu 77, meskipun koefisien emisinya paling besar.

Hasil Implementasi Variable Neighborhood Search pada Data A-n60-k9 untuk Minimasi Emisi Skenario 1

Berdasarkan hasil implementasi *Variable Neighborhood Search* (VNS), diperoleh sembilan rute kendaraan dengan total *demand* sebesar 829 satuan. Setiap kendaraan memiliki kapasitas maksimum sebesar 100 satuan. Total jarak yang dihasilkan oleh VNS adalah 1.402, sedangkan total emisi yang diperoleh sebesar 530,9122.

Tabel 23 Hasil Implementasi Variable Neighborhood Search A-n60-k9

Kendaraan	Rute	Kapasitas	Jarak	Koefisien Emisi	Emisi
1	1 → 35 → 3 → 2 → 49 → 23 → 37 → 1	98/100	183	$\alpha = 0,150$ $\beta = 0,0050$	65.8450
2	1 → 7 → 29 → 32 → 54 → 31 → 50 → 45 → 1	95/100	180	$\alpha = 0,150$ $\beta = 0,0050$	61.3000
3	1 → 19 → 14 → 9 → 28 → 18 → 27 → 16 → 1	95/100	138	$\alpha = 0,150$ $\beta = 0,0050$	51.0850
4	1 → 36 → 44 → 57 → 13 → 52 → 10 → 33 → 1	98/100	230	$\alpha = 0,150$ $\beta = 0,0050$	82.1350
5	1 → 39 → 60 → 8 → 30 → 58 → 38 → 1	99/100	138	$\alpha = 0,150$ $\beta = 0,0050$	46.6900
6	1 → 42 → 34 → 53 → 20 → 56 → 40 → 51 → 1	80/100	120	$\alpha = 0,150$ $\beta = 0,0050$	32.6600

Lanjutan tabel...

Kendaraan	Rute	Kapasitas	Jarak	Koefisien Emisi	Emisi
7	1 → 46 → 43 → 6 → 55 → 11 → 1	94/100	190	$\alpha = 0,150$ $\beta = 0,0050$	71.2250
8	1 → 26 → 47 → 41 → 12 → 5 → 22 → 4 → 21 → 17 → 1	84/100	103	$\alpha = 0,150$ $\beta = 0,0050$	33.4100
9	1 → 15 → 48 → 24 → 59 → 25 → 1	86/100	113	$\alpha = 0,150$ $\beta = 0,0050$	34.4450
Total	–	829	1.395		478.7950

Berdasarkan Tabel 23, seluruh pelanggan dapat dilayani tepat satu kali dengan menggunakan sembilan kendaraan. Muatan masing-masing kendaraan juga tidak melebihi kapasitas maksimum sebesar 100 satuan. Kapasitas kendaraan berada pada rentang 84 sampai 99 satuan. Kendaraan 5 memiliki muatan tertinggi sebesar 99 satuan, sedangkan kendaraan 8 memiliki muatan terendah sebesar 84 satuan. Dengan demikian, solusi yang dihasilkan VNS telah memenuhi kendala kapasitas kendaraan dan pelayanan pelanggan.

Jika dibandingkan dengan solusi awal *Clarke–Wright Savings*, total jarak hasil VNS mengalami penurunan. Total jarak pada solusi CWS adalah 1.420, sedangkan total jarak hasil VNS menjadi 1.395. Dengan demikian, terjadi penurunan jarak tempuh sebesar 25 satuan atau 13,50%. Dari sisi emisi, VNS juga mampu memberikan perbaikan yang lebih besar. Total emisi pada solusi awal CWS adalah 553.510, sedangkan total emisi hasil VNS menjadi 478,795. Dengan demikian, terjadi penurunan emisi sebesar 74,7150.

Hasil Optimasi Variable Neighborhood Search Berbasis Emisi pada Data A-n60-k9 Skenario 2

Tabel 24 Hasil Optimasi VNS Berbasis Emisi pada Data A-n60-k9

Kendaraan	Tahun	Rute	Demand/ Kapasitas	Jarak	Koefisien Emisi	Emisi
1	2024	1 → 36 → 44 → 57 → 13 → 52 → 10 → 33 → 1	98/100	230	$\alpha = 0,150$ $\beta = 0,0050$	82.1350

Lanjutan tabel...

Kendaraan	Tahun	Rute	Demand/ Kapasitas	Jarak	Koefisien Emisi	Emisi
2	2023	1 → 3 → 2 → 49 → 23 → 37 → 31 → 50 → 1	100/100	219	$\alpha = 0,155$ $\beta = 0,0052$	80.1626
3	2021	1 → 46 → 43 → 6 → 55 → 11 → 1	94/100	190	$\alpha = 0,165$ $\beta = 0,0055$	78.3475
4	2019	1 → 7 → 32 → 29 → 45 → 54 → 22 → 5 → 1	100/100	159	$\alpha = 0,175$ $\beta = 0,0059$	70.2932
5	2018	1 → 56 → 16 → 27 → 18 → 58 → 38 → 30 → 1	98/100	149	$\alpha = 0,185$ $\beta = 0,0062$	71.1696
6	2016	1 → 19 → 20 → 8 → 14 → 9 → 28 → 40 → 51 → 1	94/100	149	$\alpha = 0,200$ $\beta = 0,0067$	71.3266
7	2014	1 → 15 → 48 → 24 → 59 → 25 → 35 → 1	95/100	112	$\alpha = 0,215$ $\beta = 0,0072$	54.4568
8	2012	1 → 26 → 47 → 41 → 12 → 4 → 21 → 17 → 1	68/100	84	$\alpha = 0,235$ $\beta = 0,0078$	39.2088
9	2010	1 → 42 → 34 → 53 → 60 → 39 → 1	82/100	81	$\alpha = 0,250$ $\beta = 0,0085$	40.2505
Total	-	-	829		-	587.350 6

Dari sisi emisi, kendaraan 1 menghasilkan emisi terbesar, yaitu 82,1350. Hal ini dipengaruhi oleh jarak tempuh yang paling besar dan *Demand* yang tinggi. Kendaraan 2 menghasilkan emisi sebesar 80,1626, sedangkan kendaraan 3 menghasilkan emisi sebesar 78,3475. Emisi terkecil terdapat pada kendaraan 8, yaitu sebesar 39,2088.

Hasil Implementasi Variable Neighborhood Search Data A-n60-k9 Skenario 3

Pada skenario 3, metode Variable Neighborhood Search digunakan untuk mengoptimalkan solusi awal yang diperoleh dari Clarke-Wright Savings dengan mempertimbangkan kapasitas dan koefisien emisi kendaraan yang berbeda. Setiap kendaraan memiliki kapasitas yang tidak sama, mulai dari 120 sampai 75 satuan muatan. Selain itu, setiap kendaraan juga memiliki nilai koefisien emisi yang berbeda berdasarkan tahun keluaran kendaraan. Kendaraan dengan tahun keluaran lebih baru memiliki koefisien emisi lebih rendah, sedangkan kendaraan dengan tahun keluaran lebih lama memiliki koefisien emisi lebih tinggi.

Tabel 25 Hasil Optimasi Variable Neighborhood Search Data A-n60-k9 Skenario 3

Kendaraan	Tahun	Rute	<i>Demand</i> / Kapasitas	Jarak	Koefisien Emisi	Emisi
1	2024	1 → 25 → 46 → 43 → 6 → 55 → 11 → 3 → 1	120/12 0	194	$\alpha = 0,150$ $\beta = 0,0050$	80.285 0
2	2023	1 → 36 → 56 → 44 → 57 → 13 → 52 → 10 → 33 → 1	107/110	231	$\alpha = 0,155$ $\beta = 0,0052$	87.794 6
3	2021	1 → 35 → 59 → 49 → 23 → 37 → 2 → 1	105/105	183	$\alpha = 0,165$ $\beta = 0,0055$	78.419 0
4	2019	1 → 7 → 32 → 29 → 45 → 50 → 31 → 54 → 22 → 1	100/100	182	$\alpha = 0,175$ $\beta = 0,0059$	76.743 1
5	2018	1 → 20 → 14 → 9 → 28 → 27 → 16 → 40 → 51 → 1	95/95	153	$\alpha = 0,185$ $\beta = 0,0062$	70.527 0
6	2016	1 → 19 → 8 → 30 → 58 → 18 → 38 → 1	88/90	146	$\alpha = 0,200$ $\beta = 0,0067$	64.723 4

Lanjutan tabel...

Kendaraan	Tahun	Rute	Demand/ Kapabilitas	Jarak	Koefisien Emisi	Emisi
7	2014	1 → 42 → 34 → 53 → 60 → 39 → 1	82/85	81	$\alpha = 0,215$ $\beta = 0,0072$	34.356 6
8	2012	1 → 26 → 47 → 41 → 12 → 5 → 4 → 21 → 17 → 1	79/80	99	$\alpha = 0,235$ $\beta = 0,0078$	48.817 8
9	2010	1 → 15 → 48 → 24 → 1	53/75	73	$\alpha = 0,280$ $\beta = 0,0085$	31.442 0
Total	-	-	829	1342	-	573.10 85

Dari sisi jarak, kendaraan 2 memiliki jarak tempuh terbesar, yaitu 231. Rute kendaraan 2 melewati pelanggan 36, 56, 44, 57, 13, 52, 10, dan 33 sebelum kembali ke depot. Jarak terbesar berikutnya terdapat pada kendaraan 1 sebesar 194 dan kendaraan 3 sebesar 183. Sementara itu, jarak terkecil terdapat pada kendaraan 9, yaitu sebesar 73. Perbedaan jarak antar kendaraan dipengaruhi oleh jumlah pelanggan yang dilayani, lokasi pelanggan terhadap depot, serta urutan kunjungan pada setiap rute.

Dari sisi emisi, kendaraan 2 menghasilkan emisi terbesar, yaitu 87,7946. Hal ini dipengaruhi oleh jarak tempuh yang paling panjang serta besarnya beban yang diangkut. Kendaraan 1 menghasilkan emisi sebesar 80,2850, sedangkan kendaraan 3 menghasilkan emisi sebesar 78,4190. Emisi terkecil terdapat pada kendaraan 9, yaitu sebesar 31,4420, karena kendaraan tersebut memiliki rute yang paling pendek dan mengangkut beban yang relatif lebih kecil. Hasil ini menunjukkan bahwa besarnya emisi dipengaruhi oleh koefisien emisi kendaraan, jarak tempuh, dan beban kendaraan pada setiap ruas perjalanan.

Hasil Optimasi Variable Neighborhood Search Berbasis Emisi pada Data A-n80-k10 Skenario 1

Tabel 26 Hasil Optimasi VNS Berbasis Emisi pada Data A-n80-k10

Kendaraan	Rute	Kapasitas	Jarak	Koefisien Emisi	Emisi
1	1 → 41 → 12 → 35 → 3 → 38 → 9 → 69 → 44 → 17 → 1	92/100	231	$\alpha = 0,150$ $\beta = 0,0050$	73.9100
2	1 → 52 → 78 → 65 → 47 → 26 → 42 → 16 → 56 → 10 → 55 → 73 → 1	90/100	222	$\alpha = 0,150$ $\beta = 0,0050$	78.1600

Lanjutan tabel...

Kendaraan	Rute	Kapasitas	Jarak	Koefisien Emisi	Emisi
3	1 → 63 → 31 → 79 → 62 → 58 → 27 → 36 → 66 → 70 → 57 → 34 → 1	97/100	274	$\alpha = 0,150$ $\beta = 0,0050$	92.6200
4	1 → 43 → 13 → 45 → 24 → 7 → 25 → 1	92/100	162	$\alpha = 0,150$ $\beta = 0,0050$	53.0600
5	1 → 22 → 53 → 29 → 80 → 49 → 19 → 15 → 1	98/100	193	$\alpha = 0,150$ $\beta = 0,0050$	69.1600
6	1 → 50 → 74 → 37 → 67 → 68 → 71 → 59 → 39 → 1	88/100	125	$\alpha = 0,150$ $\beta = 0,0050$	38.9600
7	1 → 4 → 40 → 61 → 32 → 21 → 76 → 20 → 48 → 1	94/100	221	$\alpha = 0,150$ $\beta = 0,0050$	67.0100
8	1 → 54 → 77 → 51 → 46 → 23 → 5 → 33 → 1	100/100	197	$\alpha = 0,150$ $\beta = 0,0050$	69.7200
9	1 → 2 → 8 → 11 → 72 → 64 → 1	93/100	135	$\alpha = 0,150$ $\beta = 0,0050$	44.3800
10	1 → 14 → 75 → 30 → 18 → 28 → 60 → 6 → 1	98/100	154	$\alpha = 0,150$ $\beta = 0,0050$	54.6900
Total	-	942	1914		641.6700

Dari hasil tersebut kendaraan 3 memiliki jarak tempuh terbesar, yaitu 307. Kendaraan 2 memiliki jarak sebesar 222, sedangkan kendaraan 1 memiliki jarak sebesar 207. Jarak terkecil terdapat pada kendaraan 10, yaitu sebesar 125. Perbedaan jarak tersebut dipengaruhi oleh jumlah pelanggan dalam rute, posisi pelanggan terhadap depot, serta urutan kunjungan pelanggan pada masing-masing kendaraan. Dari sisi emisi, kendaraan 3 juga menghasilkan emisi terbesar, yaitu 113,2600. Hal ini disebabkan oleh kombinasi antara jarak tempuh yang paling besar dan kapasitas yang terpakai penuh, yaitu 100/100. Kendaraan 2 menghasilkan emisi sebesar 78,1600, sedangkan kendaraan 1 menghasilkan emisi sebesar 70,9900. Emisi terkecil terdapat pada kendaraan 10, yaitu sebesar 38,9600, karena rute kendaraan tersebut memiliki jarak tempuh paling rendah dan *Demand* yang relatif lebih kecil dibandingkan kendaraan lainnya.

Hasil Implementasi Variable Neighborhood Search pada Data A-n80-k10 untuk Minimasi Emisi

Tabel 27 Hasil Optimasi Rute Distribusi Data A-n80-k10

Kendaraan	Tahun	Rute	Demand/ Kapasitas	Jarak	Koefisien Emisi	Emisi
1	2024	1 → 63 → 25 → 7 → 3 → 38 → 9 → 35 → 1	100/100	206	$\alpha = 0,150$ $\beta = 0,0050$	70.3000
2	2023	1 → 52 → 78 → 40 → 61 → 47 → 26 → 42 → 57 → 70 → 66 → 36 → 27 → 1	96/100	266	$\alpha = 0,155$ $\beta = 0,0051$	83.3458
3	2022	1 → 30 → 28 → 60 → 21 → 76 → 58 → 20 → 48 → 16 → 65 → 1	95/100	232	$\alpha = 0,160$ $\beta = 0,0052$	86.4368
4	2021	1 → 50 → 39 → 59 → 77 → 73 → 55 → 10 → 56 → 34 → 1	99/100	214	$\alpha = 0,165$ $\beta = 0,0053$	74.8692
5	2020	1 → 2 → 24 → 31 → 79 → 62 → 17 → 44 → 69 → 1	100/100	218	$\alpha = 0,170$ $\beta = 0,0054$	75.5026
6	2019	1 → 4 → 75 → 18 → 32 → 6 → 45 → 13 → 1	99/100	145	$\alpha = 0,175$ $\beta = 0,0055$	61.1855
7	2018	1 → 12 → 53 → 29 → 80 → 49 → 19 → 15 → 1	99/100	193	$\alpha = 0,180$ $\beta = 0,0056$	82.5416
8	2017	1 → 74 → 68 → 71 → 51 → 46 → 23 → 5 → 33 → 1	95/100	194	$\alpha = 0,185$ $\beta = 0,0057$	75.7729

Lanjutan tabel...

Kendaraan	Tahun	Rute	Demand/ Kapasitas	Jarak	Koefisien Emisi	Emisi
9	2016	1 → 14 → 43 → 54 → 67 → 37 → 1	71/100	94	$\alpha = 0,190$ $\beta = 0,0058$	34.8888
10	2015	1 → 41 → 22 → 8 → 11 → 72 → 64 → 1	95/100	139	$\alpha = 0,195$ $\beta = 0,0059$	56.3808
Total			942	1901		701.2240

Secara keseluruhan, hasil VNS menghasilkan total jarak tempuh sebesar 1901 dan total emisi sebesar 701,2240. Semua kendaraan berada dalam batas kapasitas yang diperbolehkan, dengan kapasitas terpakai berkisar antara 71 hingga 100 satuan. Hal ini menunjukkan bahwa solusi hasil VNS bersifat feasible karena memenuhi dua kendala utama, yaitu seluruh pelanggan dilayani tepat satu kali dan kapasitas setiap kendaraan tidak terlampaui. Selain itu, penggunaan kendaraan dengan koefisien emisi yang berbeda mampu menurunkan total emisi distribusi tanpa melanggar kendala operasional yang telah ditetapkan.

Hasil Implementasi Variable Neighborhood Search Data 80 Skenario 3

Tabel 28 Hasil Optimasi Variable Neighborhood Search Data 80 Skenario 3

Kendaraan	Tahun	Rute	Demand/ Kapasitas	Jarak	Koefisien Emisi	Emisi
1	2024	1 → 31 → 69 → 44 → 17 → 62 → 58 → 20 → 27 → 36 → 66 → 70 → 57 → 48 → 26 → 1	120/120	302	$\alpha = 0,150$ $\beta = 0,0050$	127.735 0
2	2023	1 → 30 → 28 → 60 → 21 → 76 → 79 → 9 → 38 → 3 → 35 → 1	106/115	229	$\alpha = 0,155$ $\beta = 0,0052$	93.0902

Lanjutan tabel...

Kendaraan	Tahun	Rute	Demand/ Kapasitas	Jarak	Koefisien Emisi	Emisi
3	2022	1 → 52 → 61 → 18 → 32 → 47 → 42 → 16 → 56 → 10 → 55 → 34 → 65 → 1	110/110	235	$\alpha = 0,160$ $\beta = 0,0054$	94,4674
4	2021	1 → 68 → 67 → 77 → 51 → 46 → 23 → 5 → 33 → 73 → 1	105/105	206	$\alpha = 0,165$ $\beta = 0,0056$	79,8260
5	2020	1 → 72 → 53 → 29 → 80 → 49 → 19 → 15 → 1	97/100	192	$\alpha = 0,175$ $\beta = 0,0059$	82,4815
6	2018	1 → 2 → 8 → 11 → 64 → 63 → 1	99/100	132	$\alpha = 0,185$ $\beta = 0,0062$	57,3296
7	2016	1 → 13 → 45 → 6 → 24 → 7 → 25 → 12 → 1	89/95	162	$\alpha = 0,200$ $\beta = 0,0067$	81.1911
8	2014	1 → 43 → 78 → 4 → 75 → 40 → 1	88/90	124	$\alpha = 0,215$ $\beta = 0,0072$	57,7928
9	2012	1 → 50 → 74 → 37 → 54 → 71 → 59 → 39 → 1	85/85	130	$\alpha = 0,235$ $\beta = 0,0078$	62,3428

Lanjutan tabel...

Kendaraan	Tahun	Rute	Demand/ Kapasitas	Jarak	Koefisien Emisi	Emisi
10	2010	1 → 41 → 22 → 14 → 1	38/80	79	$\alpha = 0,250$ $\beta = 0,0085$	31,3100
Total	-	-	942	1796	-	767.566 4

Dari sisi jarak, kendaraan 1 memiliki jarak tempuh terbesar, yaitu 302. Rute kendaraan 1 melewati pelanggan 31, 69, 44, 17, 62, 58, 20, 27, 36, 66, 70, 57, 48, dan 26 sebelum kembali ke depot. Jarak terbesar berikutnya terdapat pada kendaraan 3 sebesar 235, kendaraan 2 sebesar 229, dan kendaraan 4 sebesar 206. Sementara itu, jarak terkecil terdapat pada kendaraan 10, yaitu sebesar 79. Perbedaan jarak antar kendaraan dipengaruhi oleh jumlah pelanggan yang dilayani, posisi pelanggan terhadap depot, serta urutan kunjungan pada setiap rute.

Dari sisi emisi, kendaraan 1 menghasilkan emisi terbesar, yaitu 127,7350, karena memiliki jarak tempuh paling panjang dan mengangkut muatan maksimum (120/120). Emisi terbesar berikutnya dihasilkan oleh kendaraan 3 sebesar 94,4674 dan kendaraan 2 sebesar 93,0902. Emisi terkecil terdapat pada kendaraan 10, yaitu sebesar 31,3100, karena kendaraan tersebut memiliki jarak tempuh paling pendek dan hanya mengangkut 38 satuan muatan dari kapasitas 80. Hasil ini menunjukkan bahwa besarnya emisi dipengaruhi oleh koefisien emisi kendaraan, jarak tempuh, dan beban kendaraan pada setiap rute distribusi.

Tabel 29 Perbandingan Jarak hasil CWS dan VNS

Data	Skenario	Metode		Selisih	Persentase
		CWS	VNS		
A-n32-k5	1	839	790	49	5,84%
	2	839	839	0	0,00%
	3	848	850	-2	-0,24%
A-n60-k9	1	1420	1395	25	1,76%
	2	1420	1373	47	3,31%
	3	1407	1342	65	4,62%
A-n80-k10	1	1864	1914	-50	-2,68%
	2	1864	1901	-37	-1,98%
	3	1785	1796	-11	-0,62%

Tabel 30 Perbandingan Emisi hasil CWS dan VNS

Data	Skenario	Metode		Selisih	Persentase
		CWS	VNS		
A-n32-k5	1	320.6850	266.1700	54.5150	17.00%
	2	378.0303	323.9925	54.0378	14.29%
	3	370.3636	343.2763	27.0873	7.31%
A-n60-k9	1	603.1032	478.7950	124.3082	20.61%
	2	675.9292	587.3506	88.5786	13.11%
	3	670.5980	573.1085	97.4895	14.54%
A-n80-k10	1	733.4200	641.6700	91.7500	12.51%
	2	802.8504	701.2240	101.6264	12.66%
	3	846.9645	767.5644	79.4001	9.38%

Berdasarkan hasil perbandingan, metode Variable Neighborhood Search (VNS) tidak selalu menghasilkan jarak tempuh yang lebih pendek dibandingkan metode Clarke-Wright Savings (CWS). Dari sembilan skenario yang diuji, VNS menghasilkan jarak yang lebih pendek pada empat skenario, jarak yang sama pada satu skenario, dan jarak yang lebih panjang pada empat skenario. Hal ini menunjukkan bahwa proses optimasi VNS tidak hanya berfokus pada pengurangan jarak tempuh, tetapi lebih mengutamakan pencapaian tujuan penelitian, yaitu meminimalkan total emisi kendaraan. Oleh karena itu, pada beberapa kasus VNS memilih rute yang sedikit lebih panjang apabila mampu menghasilkan emisi yang lebih rendah.

Pada perbandingan emisi, VNS menghasilkan nilai emisi yang lebih rendah pada seluruh skenario dibandingkan CWS. Persentase penurunan emisi berkisar antara 7,31% hingga 20,61%, dengan rata-rata sekitar 13,49%. Penurunan emisi tertinggi terjadi pada dataset A-n60-k9 skenario 1, yaitu sebesar 20,61%, yang juga memberikan penurunan emisi absolut terbesar, yaitu 124,3082 satuan.

Hasil tersebut menunjukkan bahwa jarak tempuh yang lebih panjang tidak selalu menghasilkan emisi yang lebih besar. Besarnya emisi dipengaruhi oleh kombinasi jarak tempuh, distribusi beban kendaraan, urutan kunjungan pelanggan, serta koefisien emisi masing-masing kendaraan. Dengan demikian, metode VNS lebih efektif dalam memperbaiki solusi awal CWS untuk mencapai tujuan utama penelitian, yaitu meminimalkan total emisi kendaraan, meskipun pada beberapa skenario harus mengorbankan sedikit peningkatan pada total jarak tempuh.

3.1.3 Evaluasi Gap terhadap *Best Known Solution*

Untuk mengevaluasi kualitas solusi dari sisi jarak, hasil metode dibandingkan dengan nilai *Best Known Solution* (BKS) pada CVRPLIB. Rumus yang digunakan adalah

$$Gap = \frac{Z_{hasil} - Z_{BKS}}{Z_{BKS}} \times 100 \%$$

dengan:

Z_{hasil} = total jarak hasil CWS atau VNS;

Z_{BKS} = total jarak terbaik pada CVRPLIB.

Tabel 31 Gap hasil dengan *Best Known Solution* dataset

Data	Metode	Total jarak	BKS	Gap (%)
A-n32-k5	CWS	839	784	7,02
	VNS skenario 1	790	784	0,77
	VNS skenario 2	839	784	7,02
A-n60-k9	CWS	1420	1354	4,87
	VNS skenario 1	1395	1354	3,03
	VNS skenario 2	1373	1354	1,40
A-n80-k10	CWS	1864	1763	5,73
	VNS skenario 1	1914	1763	8,56
	VNS skenario 2	1901	1763	7,83

Semakin kecil nilai *gap*, semakin dekat total jarak yang diperoleh terhadap solusi terbaik pada dataset tersebut. Perbandingan ini hanya digunakan sebagai evaluasi tambahan terhadap kualitas jarak, karena fungsi tujuan utama penelitian adalah meminimalkan total emisi kendaraan. Selain itu, perbandingan dengan BKS hanya diterapkan pada skenario berkapasitas homogen karena BKS CVRPLIB berasal dari model CVRP dengan kapasitas kendaraan yang sama. Walaupun gap terhadap BKS mencapai 8,56%, hal tersebut tidak menunjukkan kualitas solusi yang buruk karena BKS diperoleh dari model CVRP yang meminimalkan jarak, sedangkan penelitian ini meminimalkan emisi.

3.2 Pembahasan

Metode Clarke-Wright Savings menghasilkan solusi awal yang cukup baik dengan membentuk rute berdasarkan nilai *saving* terbesar sehingga mampu mengurangi jarak tempuh dan memanfaatkan kapasitas kendaraan secara efektif tanpa melanggar kendala kapasitas. Namun, solusi tersebut belum optimal untuk GCVRP karena pembentukan rute hanya mempertimbangkan penghematan jarak. Pada GCVRP, emisi dipengaruhi oleh jarak, sisa muatan, dan koefisien emisi kendaraan. Akibatnya, rute dengan jarak lebih pendek belum tentu menghasilkan emisi yang lebih rendah apabila kendaraan membawa muatan besar atau memiliki koefisien emisi yang tinggi.

Perbedaan tersebut lebih terlihat pada skenario kendaraan heterogen karena setiap kendaraan memiliki kapasitas dan koefisien emisi yang berbeda. Akibatnya, kendaraan dengan koefisien emisi rendah tidak selalu menghasilkan emisi paling kecil, dan sebaliknya. Hal ini menunjukkan bahwa total emisi dipengaruhi oleh kombinasi karakteristik kendaraan, panjang rute, dan distribusi muatan.

Untuk memperoleh solusi yang lebih baik, dilakukan optimasi menggunakan Variable Neighborhood Search (VNS). Melalui perubahan susunan pelanggan dalam dan antar rute, VNS memungkinkan kendaraan dengan koefisien emisi lebih rendah melayani pelanggan dengan permintaan lebih besar, sedangkan kendaraan dengan koefisien emisi lebih tinggi melayani rute yang lebih pendek atau permintaan lebih kecil. Pembagian ini mampu menurunkan total emisi tanpa menambah jumlah kendaraan maupun melanggar kapasitas kendaraan.

Hasil penelitian juga menunjukkan bahwa ukuran dataset memengaruhi tingkat kesulitan proses optimasi. Semakin banyak jumlah pelanggan, semakin banyak pula kemungkinan rute yang dapat dibentuk. Meskipun demikian, Variable Neighborhood Search tetap mampu menghasilkan solusi yang layak pada ketiga dataset yang digunakan. Hal ini menunjukkan bahwa algoritma tersebut cukup stabil dalam menangani permasalahan dengan ukuran data yang berbeda. Kemampuan berpindah dari satu struktur *neighborhood* ke struktur lainnya membuat proses pencarian tidak mudah berhenti pada solusi lokal sehingga peluang memperoleh solusi yang lebih baik menjadi lebih besar.

Evaluasi terhadap *Best Known Solution* menunjukkan bahwa jarak hasil penelitian masih berada di atas jarak terbaik CVRPLIB. Namun, perbandingan tersebut hanya digunakan untuk mengevaluasi kualitas solusi dari sisi jarak. *BKS* CVRPLIB dibentuk berdasarkan fungsi tujuan minimasi jarak pada CVRP, sedangkan penelitian ini menggunakan minimasi emisi. Karena tujuan optimasinya berbeda, nilai *gap* yang lebih besar tidak secara langsung menunjukkan bahwa solusi VNS lebih buruk. Solusi dengan jarak yang lebih jauh masih dapat dianggap lebih baik dalam konteks penelitian ini.

Secara keseluruhan, penurunan emisi sebesar antara 7,31% hingga 20,61%, menunjukkan bahwa VNS mampu memperbaiki solusi awal CWS pada seluruh dataset dan skenario yang diuji. Namun, hasil tersebut tidak dapat diartikan sebagai jaminan bahwa solusi yang diperoleh merupakan optimum global karena VNS merupakan metode metaheuristik. Kesimpulan yang lebih tepat adalah bahwa VNS mampu memperoleh solusi yang lebih baik daripada solusi awal CWS berdasarkan fungsi tujuan emisi.

Model emisi dalam penelitian ini menggunakan hubungan linear antara jarak, koefisien kendaraan, dan muatan. Oleh karena itu, nilai yang diperoleh merupakan nilai emisi berdasarkan model atau skenario penelitian, bukan hasil pengukuran emisi aktual kendaraan. Faktor lain seperti jenis bahan bakar, kecepatan, kondisi jalan, kemacetan, dan konsumsi bahan bakar belum diperhitungkan. Meskipun demikian, model tersebut dapat menunjukkan pengaruh perubahan urutan rute, beban, kapasitas, dan karakteristik kendaraan terhadap nilai emisi relatif.

Berdasarkan pembahasan tersebut, kombinasi CWS dan VNS dapat digunakan sebagai pendekatan penyelesaian GCVRP. CWS menyediakan solusi awal yang layak, sedangkan VNS memberikan mekanisme pencarian yang lebih luas untuk menurunkan total emisi. Efektivitas pendekatan ini terutama terlihat dari kemampuannya menghasilkan rute dengan emisi lebih rendah tanpa melanggar kendala kapasitas dan keterlayanan pelanggan.

BAB IV

PENUTUP

4.1 Kesimpulan

Berdasarkan hasil penelitian dan pembahasan yang telah dilakukan, maka dapat disimpulkan bahwa:

1. Penerapan metode Clarke-Wright Savings pada Green Capacitated Vehicle Routing Problem (GCVRP) berperan sebagai tahap awal dalam pembentukan rute distribusi yang feasible. Metode ini berhasil menyusun rute dengan menggabungkan pelanggan berdasarkan nilai penghematan jarak tertinggi serta tetap memenuhi batas kapasitas kendaraan. Namun, solusi yang dihasilkan masih bersifat parsial karena hanya berorientasi pada minimisasi jarak tempuh tanpa mempertimbangkan faktor emisi kendaraan secara langsung.
2. Penerapan metode Variable Neighborhood Search (VNS) pada GCVRP terbukti mampu meningkatkan kualitas solusi yang diperoleh dari Clarke-Wright Savings melalui proses eksplorasi beberapa struktur neighborhood, yaitu relocate, swap, reverse, dan two-opt. Hasil optimasi menunjukkan bahwa VNS mampu menghasilkan rute yang lebih baik dengan menurunkan total emisi pada seluruh data, tanpa melanggar batas kapasitas kendaraan dan tetap melayani seluruh pelanggan. Dengan demikian, berdasarkan tiga dataset dan skenario yang diuji, VNS mampu memperbaiki solusi awal Clarke-Wright Savings dengan menghasilkan total emisi yang lebih rendah.

4.2 Saran

Saran untuk penelitian selanjutnya menggunakan jarak berbasis jaringan jalan agar lebih realistis, serta mengembangkan model emisi dengan pendekatan yang lebih akurat seperti konsumsi bahan bakar atau standar emisi kendaraan. Selain itu, dapat dilakukan perbandingan dengan metode metaheuristik lain seperti *Genetic Algorithm*, *Tabu Search*, atau *Simulated Annealing* untuk memperoleh hasil optimasi yang lebih optimal. Dalam implementasi, perusahaan sebaiknya tidak hanya berfokus pada jarak terpendek, tetapi juga mempertimbangkan karakteristik kendaraan seperti usia, kapasitas, dan tingkat emisi agar distribusi lebih ramah lingkungan.

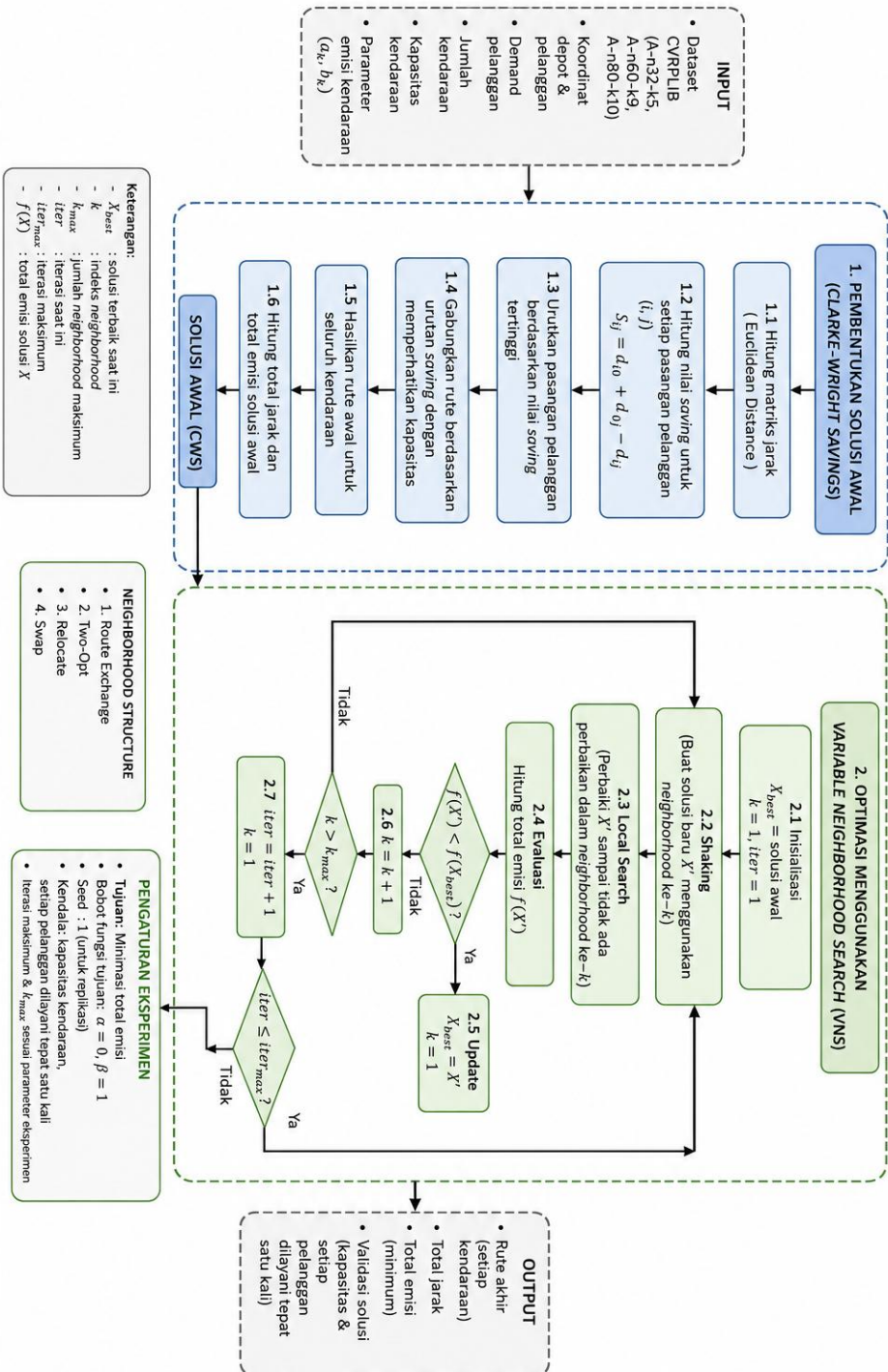
DAFTAR PUSTAKA

- Asian Transport Observatory. (2024). *Indonesia Transport and Climate Profile 2024*. Asian Transport Observatory. Diakses dari laman Asian Transport Observatory.
- Augerat, P, Naddef, D, Belenguer, J M, Benavent, E, Corberan, A, and Rinaldi, G. *Computational results with a branch and cut code for the capacitated vehicle routing problem*. France: N. p., 1995. Web.
- Bektaş, T., & Laporte, G. (2011). *The pollution-routing problem*. *Transportation Research Part B: Methodological*, 45(8), 1232–1250.
- Bowersox, D. J. (2006). *Manajemen Logistik: Integrasi Sistem-Sistem Manajemen Distribusi Fisik dan Manajemen Material* Jilid 1. Jakarta: Bumi Aksara.
- Clarke, G., & Wright, J. W. (1964). *Scheduling of Vehicles from a Central Depot to a Number of Delivery Points*. *Operations Research*, 12, 568–581.
- Erdoğan, S., & Miller-Hooks, E. (2012). A green vehicle routing problem. *Transportation Research Part E: Logistics and Transportation Review*, 48(1), 100–114. doi:10.1016/j.tre.2011.08.001.
- Hansen, P, Mladenović, N., & Moreno Pérez, J. A. (2010). *Variable neighbourhood search: Methods and applications*. *Annals of Operations Research*, 175(1), 367–407.
- Mindaputra, E. (2009). Penggunaan Algoritma Ant Colony System Dalam Travelling Salesman Problem (TSP) Pada PT. Eka Jaya Motor. Semarang: Universitas Diponegoro Semarang.
- Pichpibul, T., & Kawtummachai, R. (2013). *A Heuristic Approach Based on Clarke-Wright Algorithm for Open Vehicle Routing Problem*. *The Scientific World Journal*, 2013, 1–11
- Subagyo, P. (2000). *Manajemen Operasi*. Yogyakarta: BPFE.
- Supardi, E., & Sianturi, R. C. (2020). *Metode Saving Matrix dalam Penentuan Rute Distribusi Premium di Depot SPBU Bandung*. *Jurnal Logistik Bisnis*, 10(1), 89–98.
- Toth, P., & Vigo, D. (2002). *The Vehicle Routing Problem*. Philadelphia: Society for Industrial and Applied Mathematics.
- Úbeda, S., Faulin, J., Serrano, A., & Arcelus, F. J. (2014). Solving the Green Capacitated Vehicle Routing Problem Using a Tabu Search Algorithm. *Lecture Notes in Management Science*, 6, 141–149.
- Xiao, Y., Zhao, Q., Kaku, I., & Xu, Y. (2012). *Development of a fuel consumption optimization model for the capacitated vehicle routing problem*. *Computers & Operations Research*, 39(7), 1419–1431.

LAMPIRAN

Lampiran 1 Diagram Implementasi Algoritma

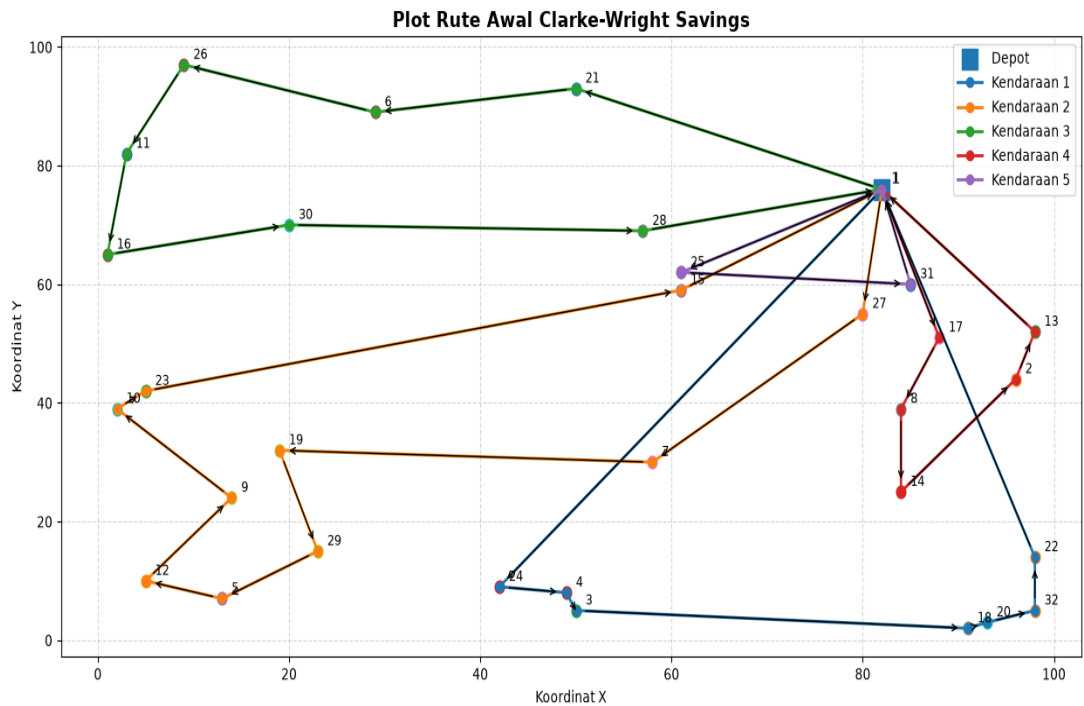
DIAGRAM IMPLEMENTASI ALGORITMA



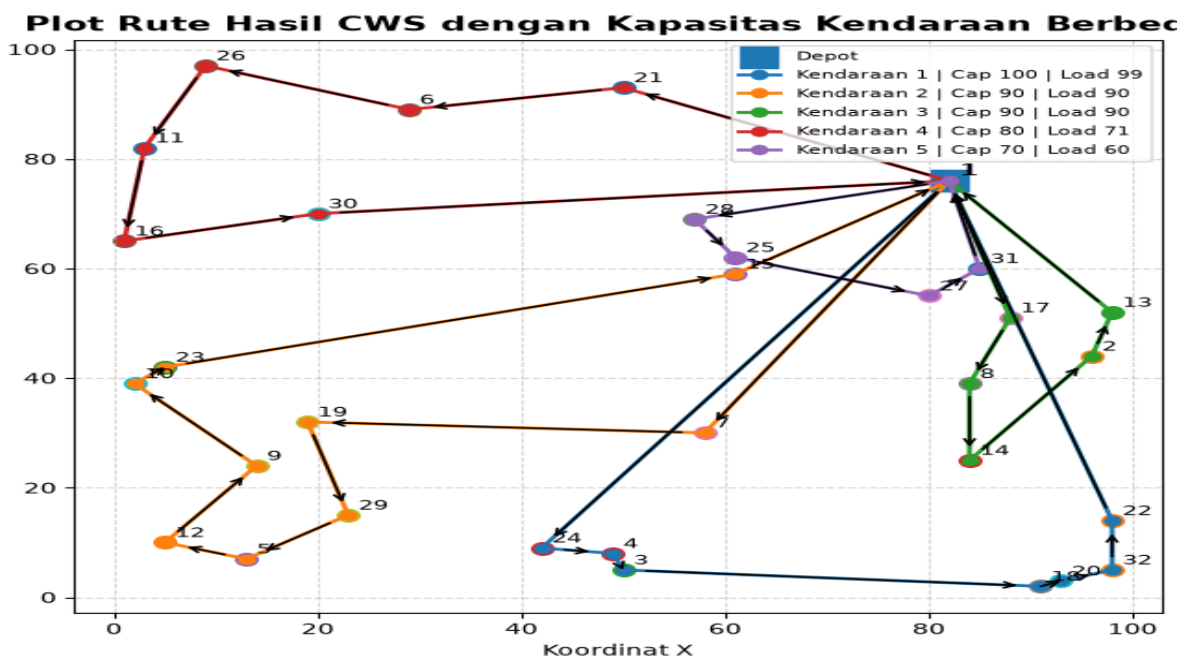
Lampiran 2 Gambar Rute hasil tiap Dataset

• Data A-n32-k5

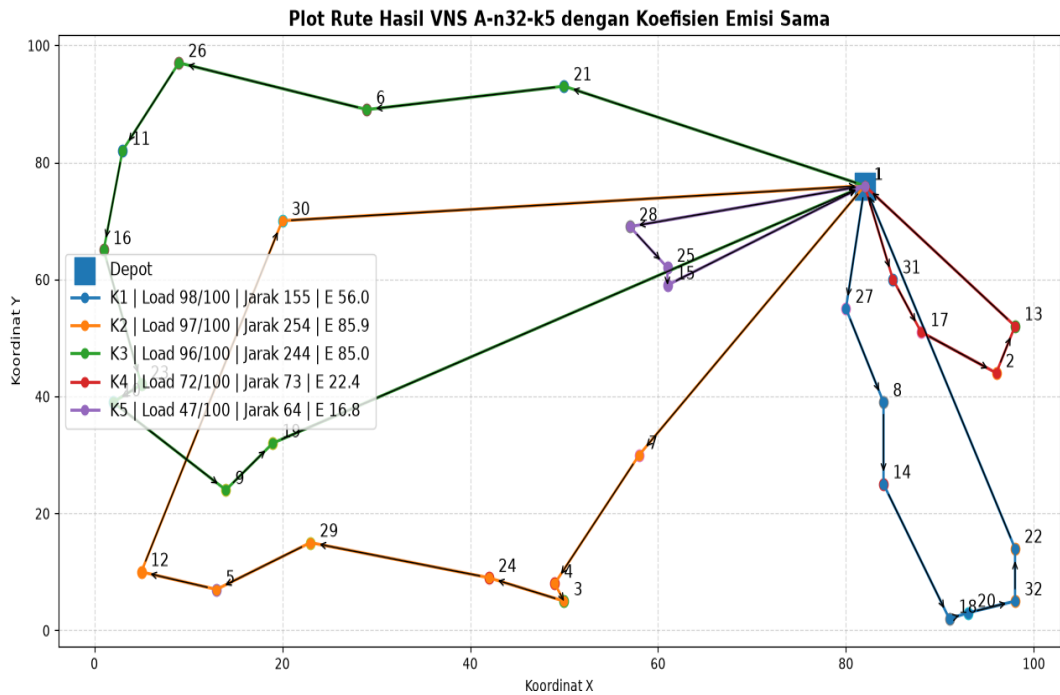
CWS skenario 1 dan 2



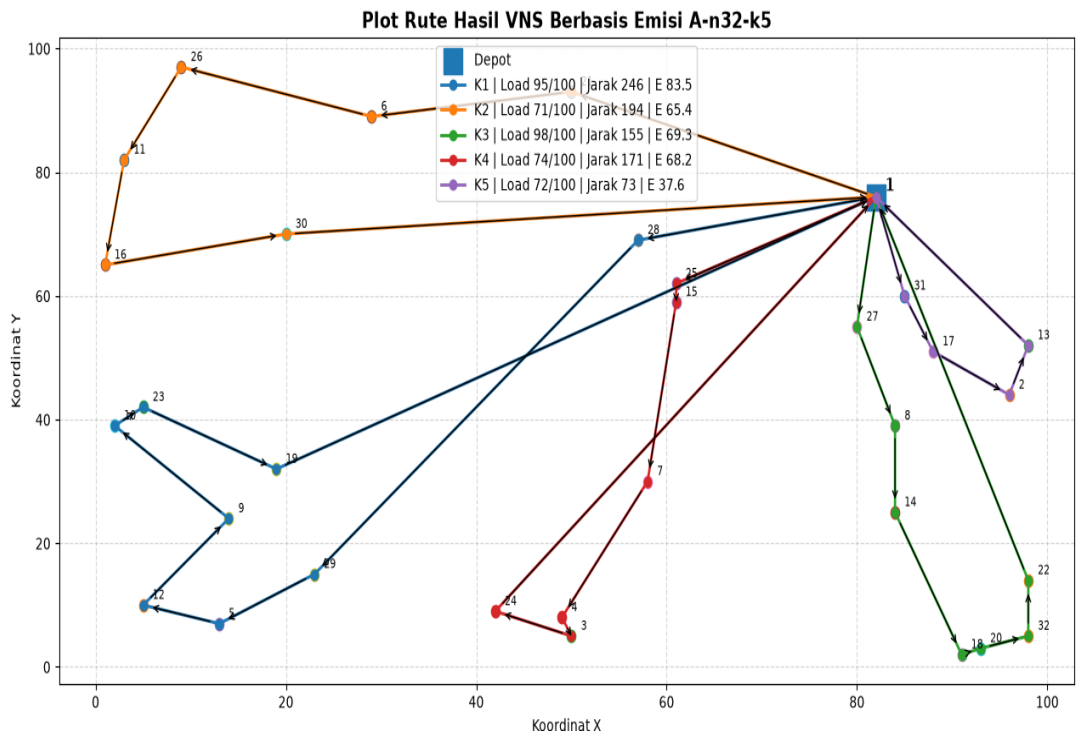
CWS Skenario 3



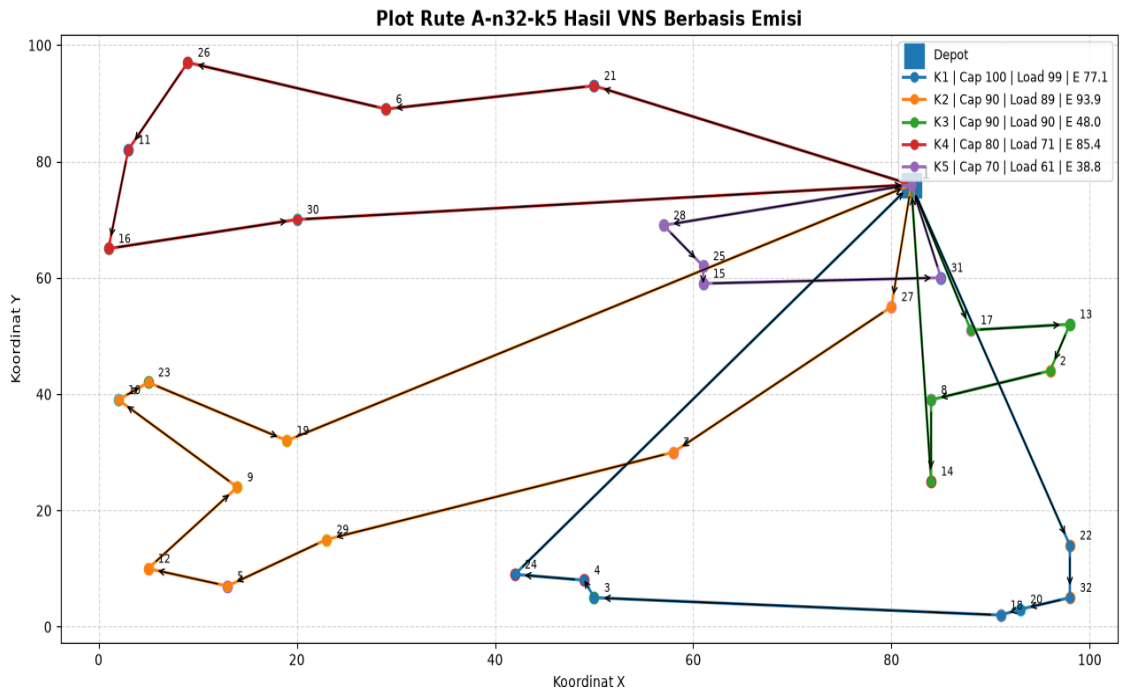
VNS skenario 1



VNS skenario 2

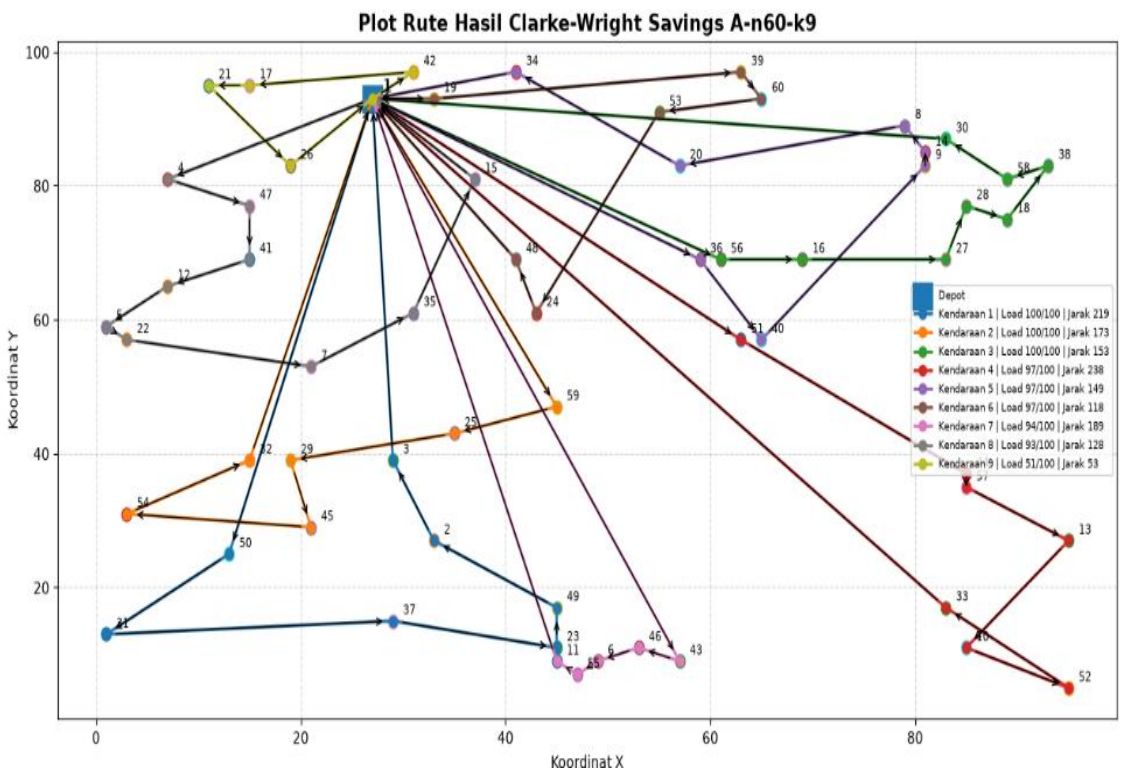


VNS Skenario 3



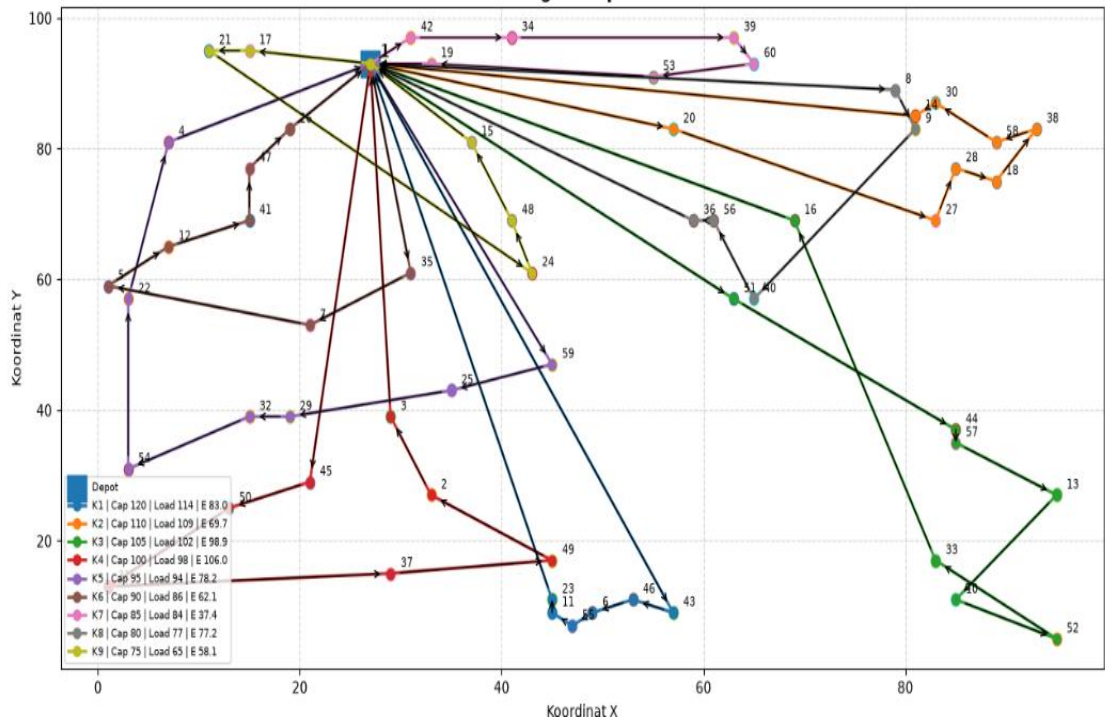
- **Data A-n60-k9**

CWS Skenario 1 dan 2



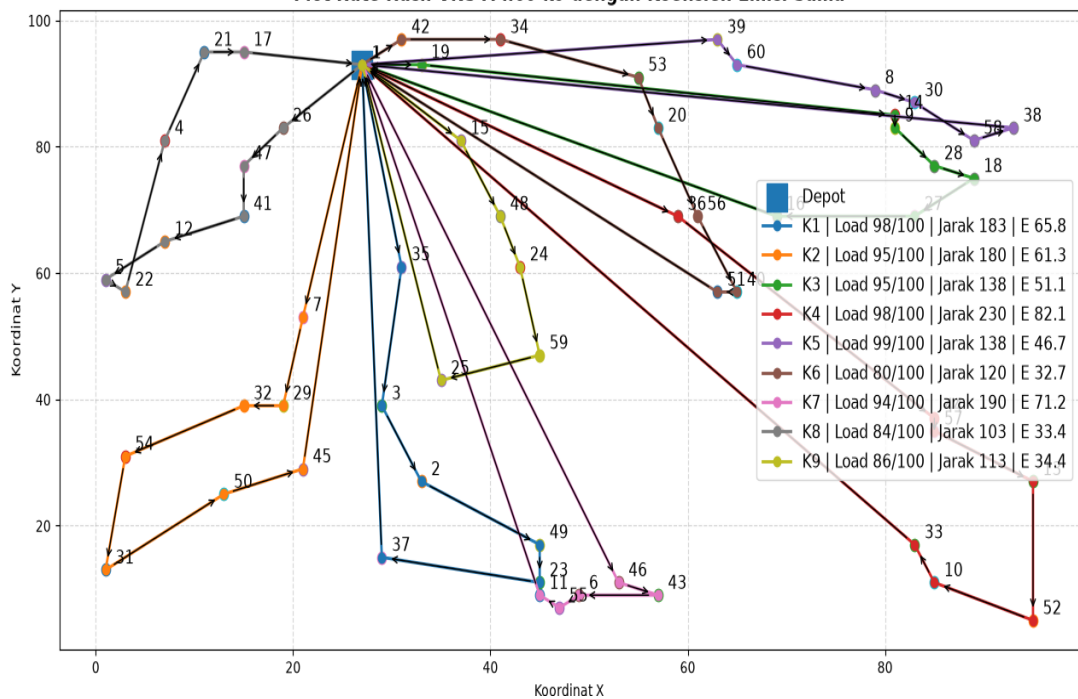
CWS Skenario 3

Plot Rute CWS A-n60-k9 dengan Kapasitas dan Koefisien Berbeda

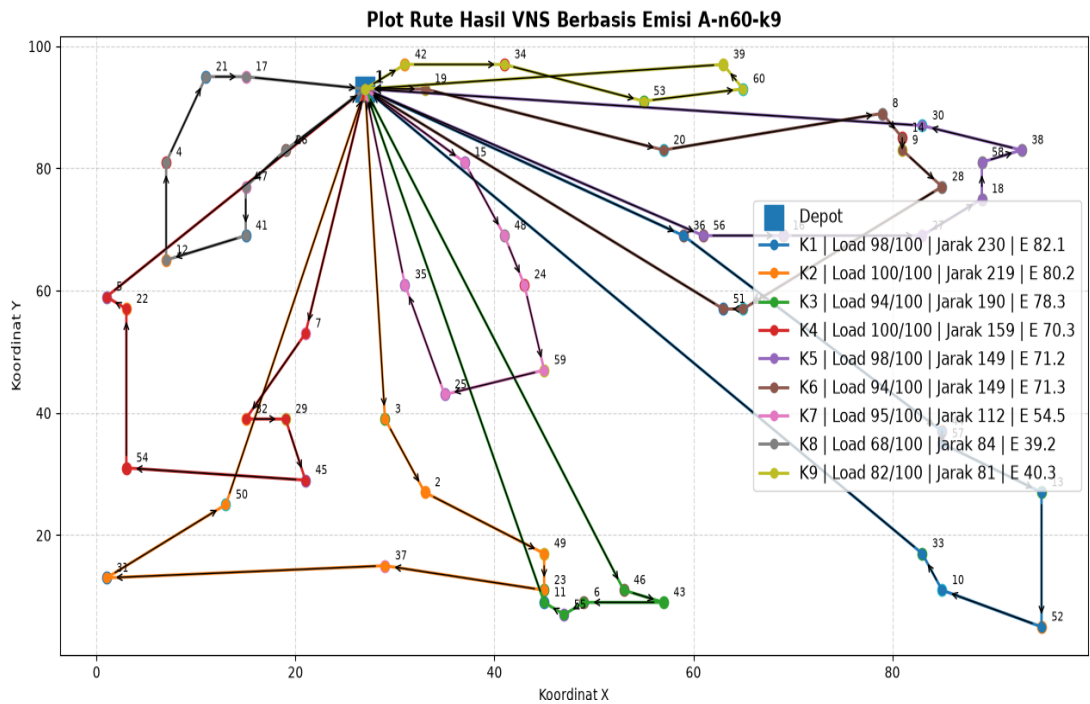


VNS Skenario 1

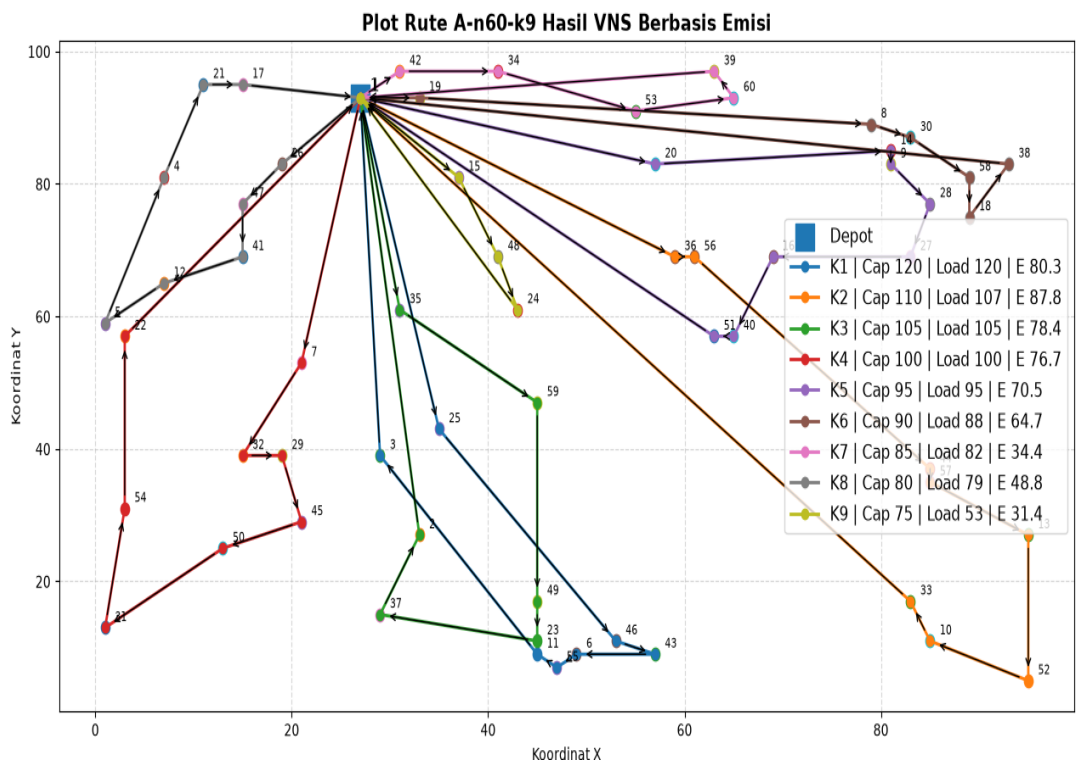
Plot Rute Hasil VNS A-n60-k9 dengan Koefisien Emisi Sama



VNS Skenario 2

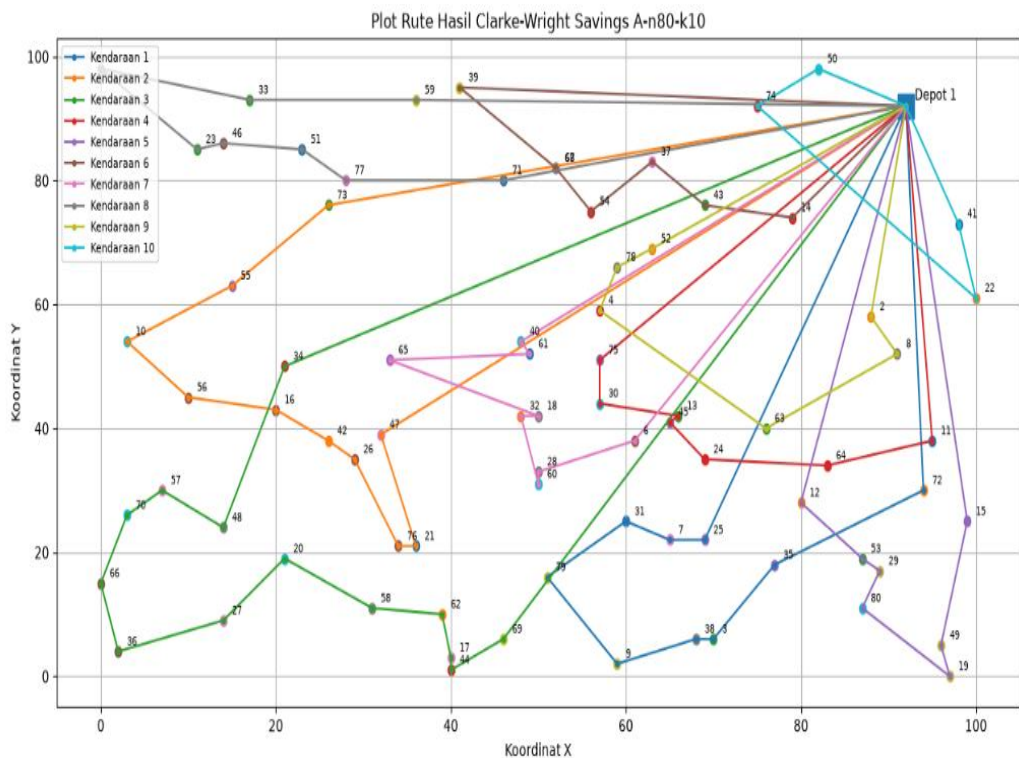


VNS Skenario 3

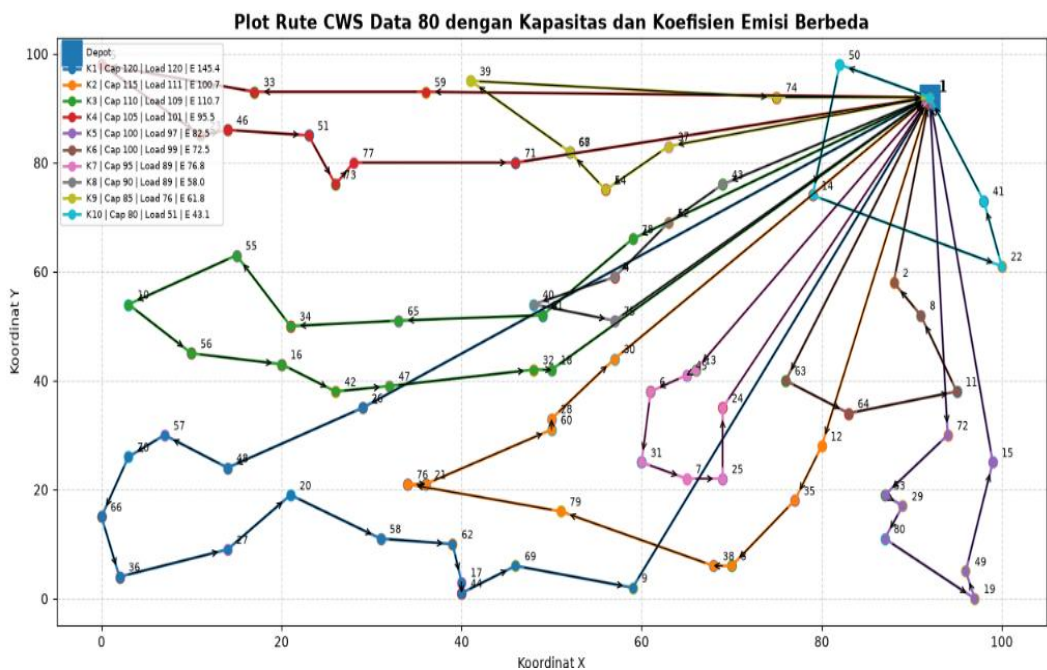


- Data A-n80-k10

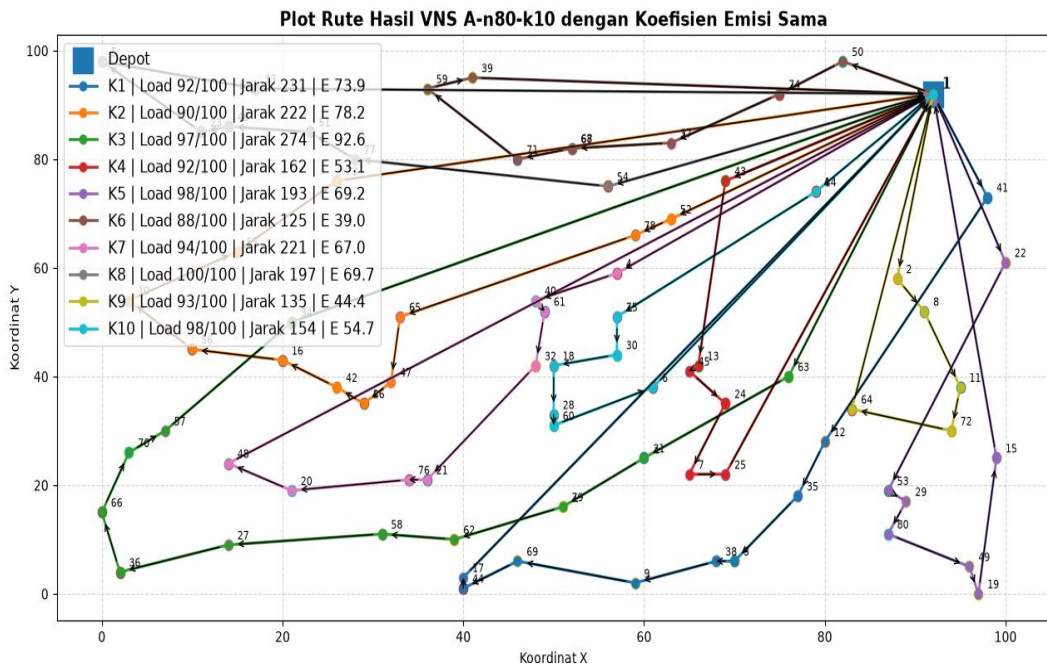
CWS Skenario 1 dan 2



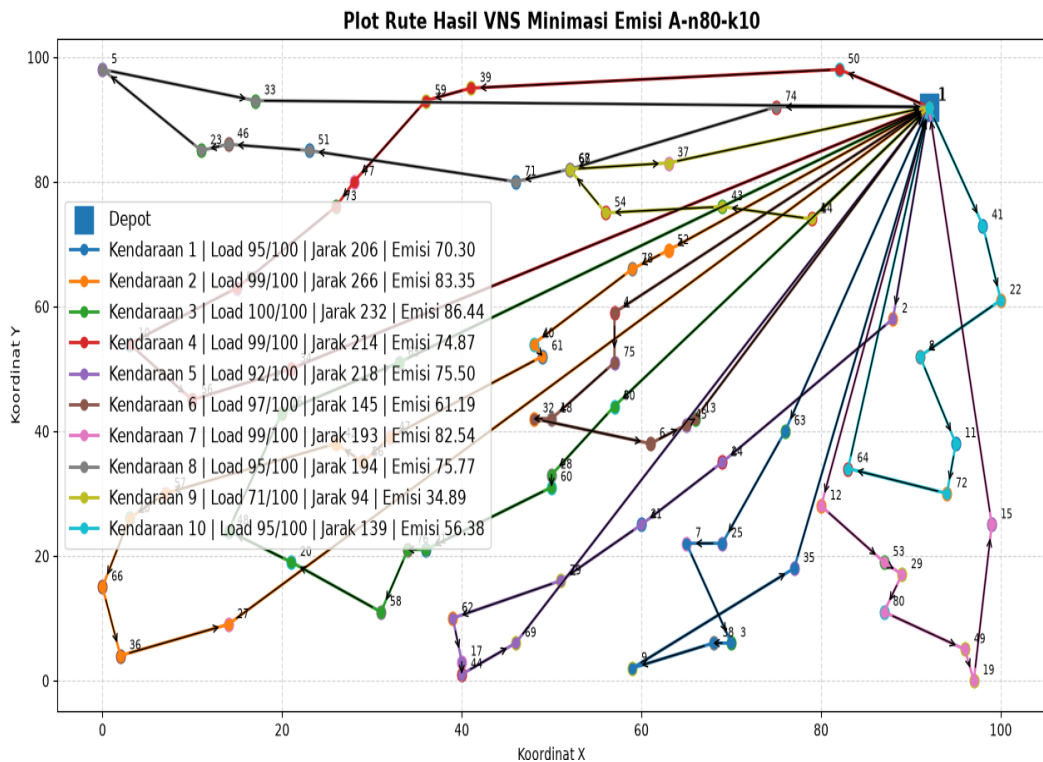
CWS Skenario 3



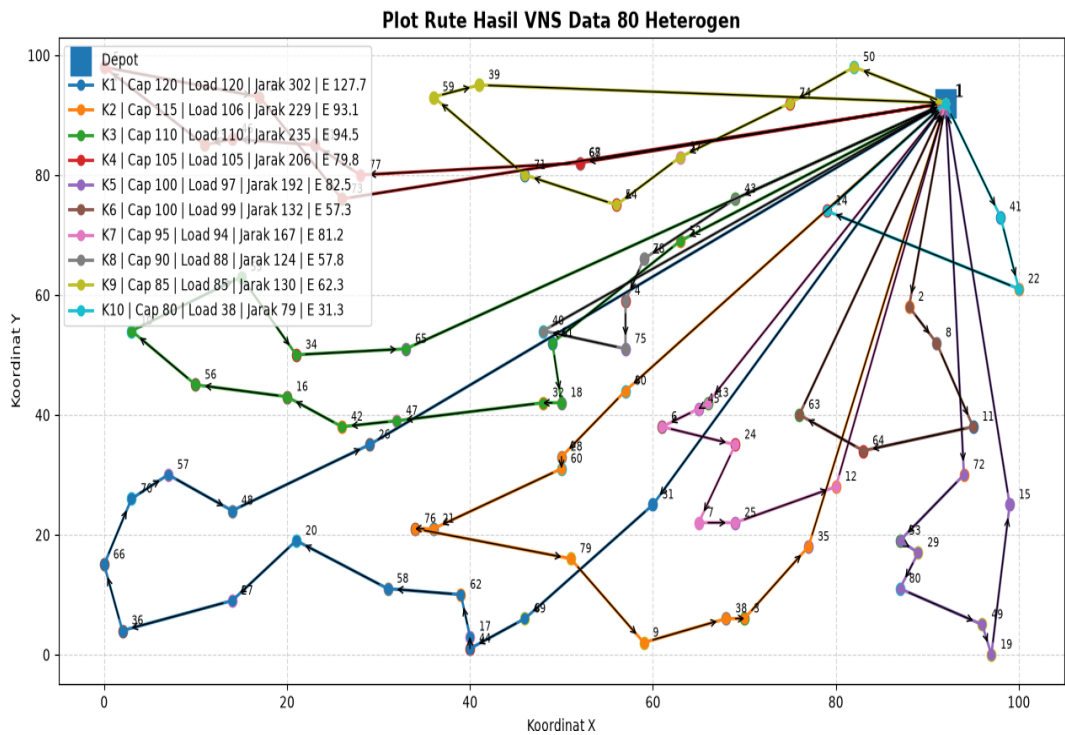
VNS Skenario 1



VNS Skenario 2



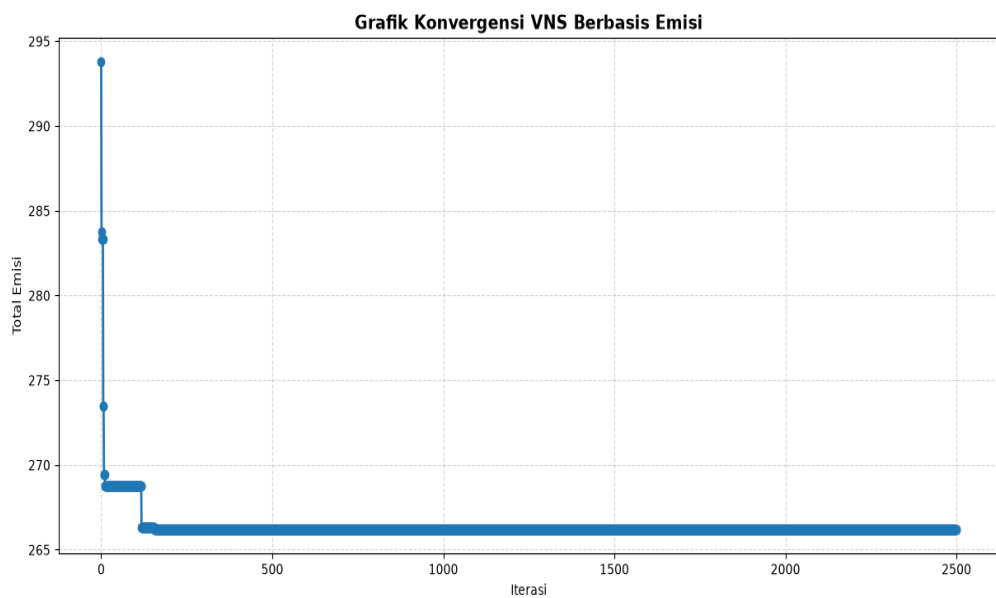
VNS Skenario 3

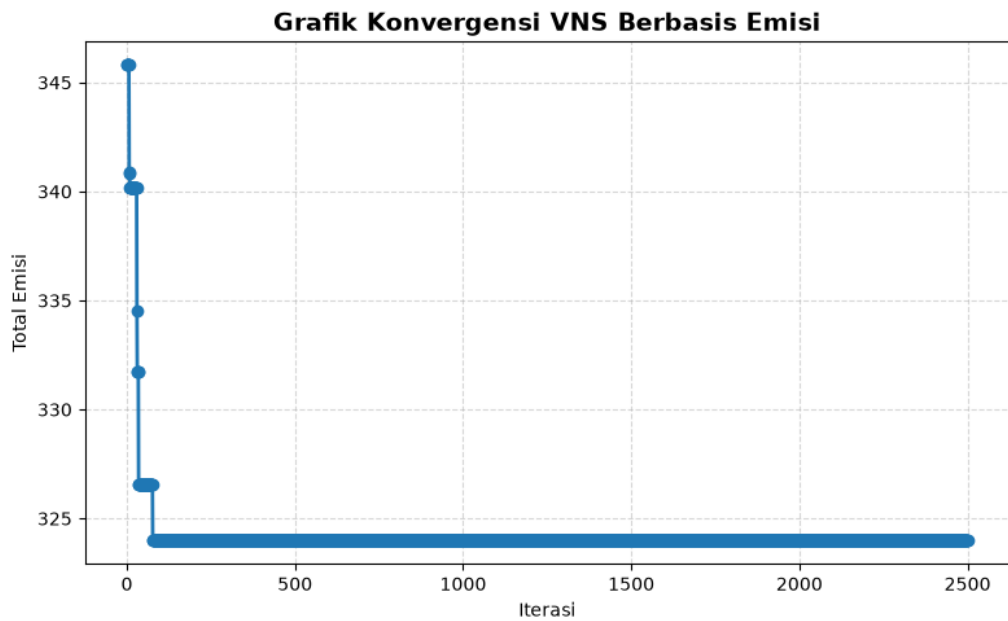
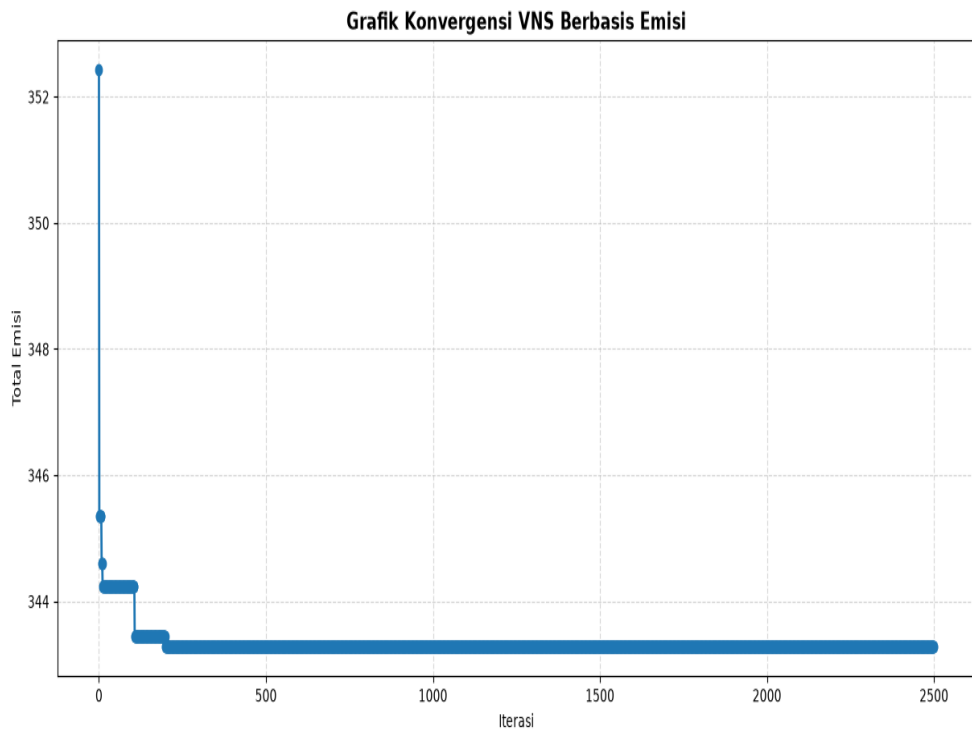


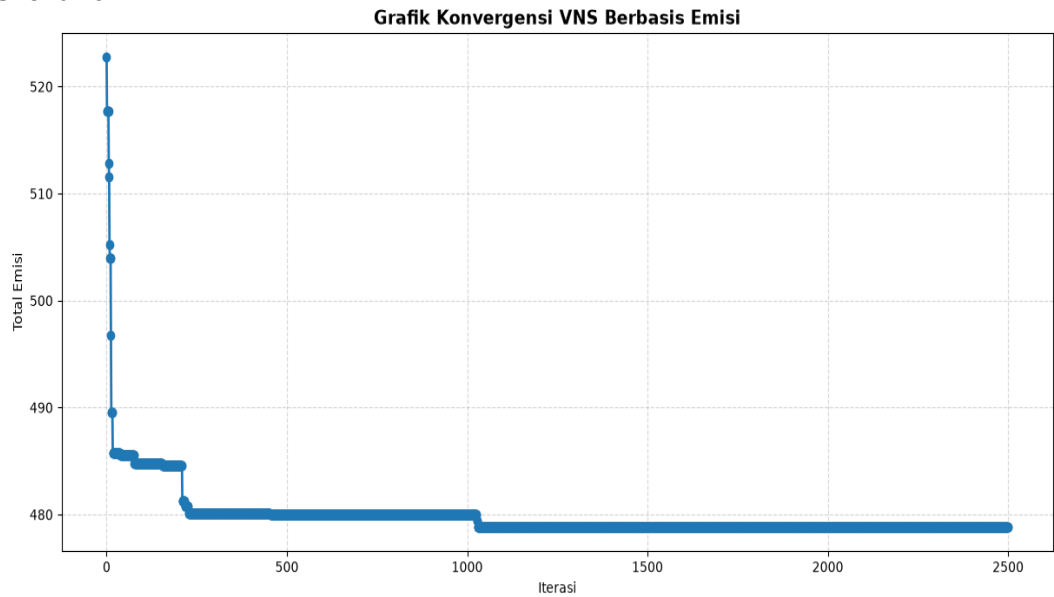
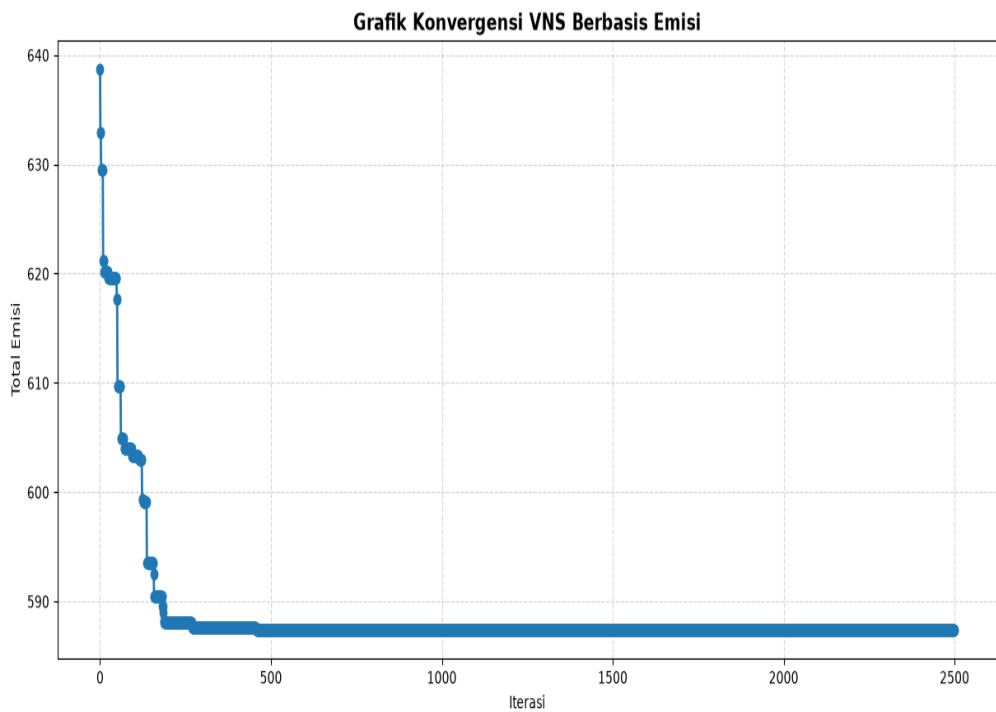
Lampiran 3 Grafik Konvergensi VNS

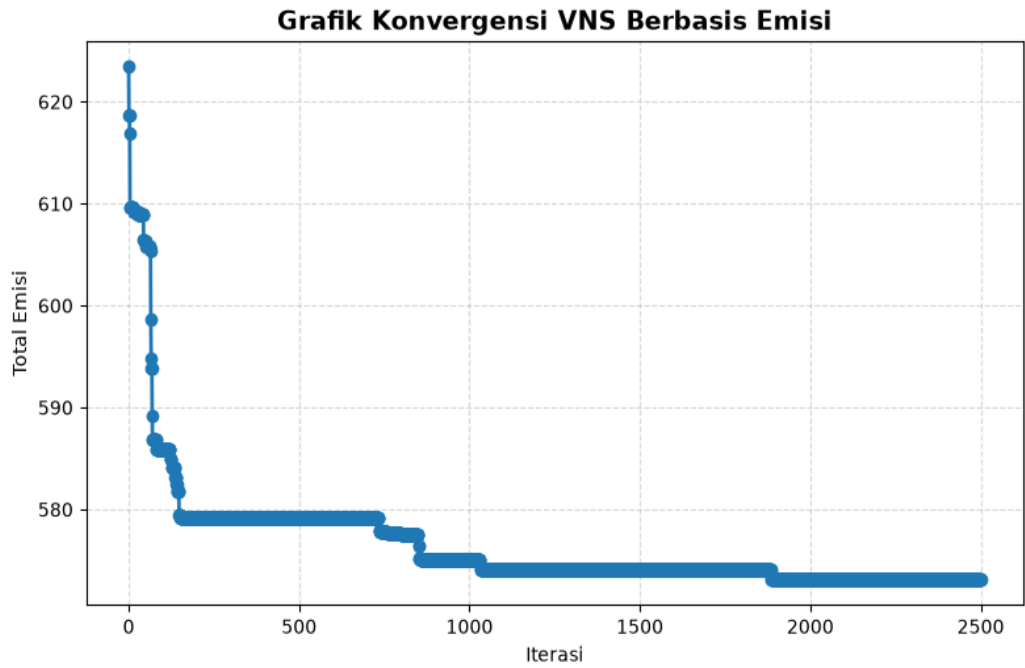
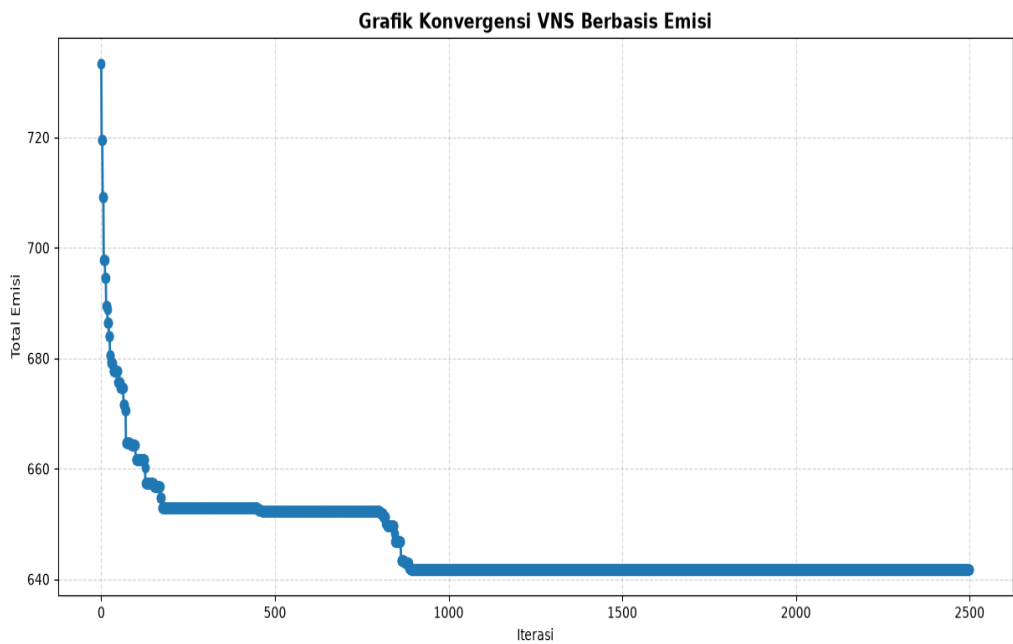
Data A-n32-k5

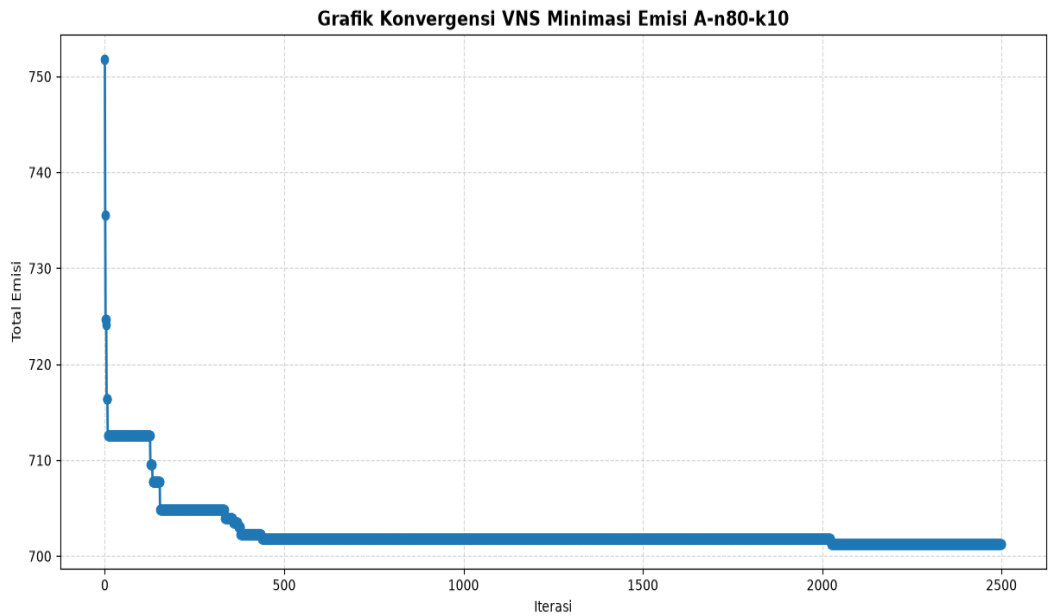
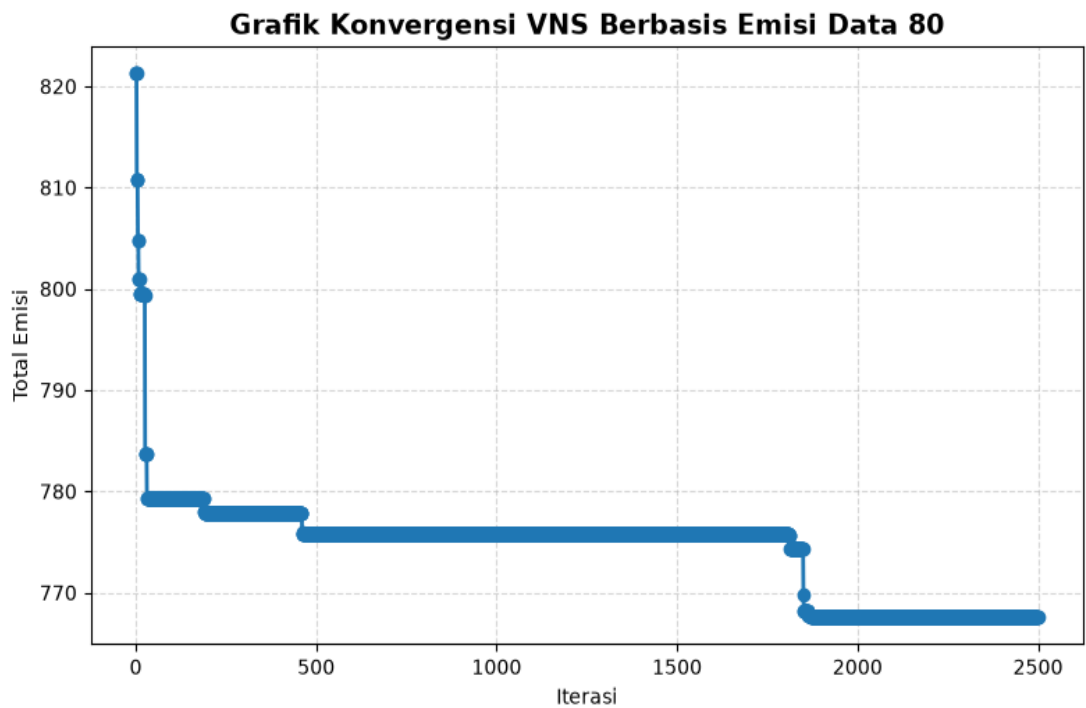
Skenario 1



Skenario 2**Skenario 3**

Data A-n60-k9**Skenario 1****Skenario 2**

Skenario 3**Data A-n60-k9****Skenario 1**

Skenario 2**Skenario 3**

Lampiran 4 Kode Program

CWS data 1(32) skenario 1

```
import math

# =====
# DATA CVRP A-n32-k5
# Augerat et al
# No of trucks: 5
# Optimal value: 784
# Capacity: 100
# Depot: node 1
# =====

CAPACITY = 100
MAX_TRUCKS = 5
DEPOT = 1

coords = {
    1: (82, 76), 2: (96, 44), 3: (50, 5), 4: (49, 8), 5: (13, 7),
    6: (29, 89), 7: (58, 30), 8: (84, 39), 9: (14, 24), 10: (2, 39),
    11: (3, 82), 12: (5, 10), 13: (98, 52), 14: (84, 25), 15: (61, 59),
    16: (1, 65), 17: (88, 51), 18: (91, 2), 19: (19, 32), 20: (93, 3),
    21: (50, 93), 22: (98, 14), 23: (5, 42), 24: (42, 9), 25: (61, 62),
    26: (9, 97), 27: (80, 55), 28: (57, 69), 29: (23, 15), 30: (20, 70),
    31: (85, 60), 32: (98, 5)
}

Demand = {
    1: 0, 2: 19, 3: 21, 4: 6, 5: 19, 6: 7, 7: 12, 8: 16,
    9: 6, 10: 16, 11: 8, 12: 14, 13: 21, 14: 16, 15: 3,
    16: 22, 17: 18, 18: 19, 19: 1, 20: 24, 21: 8, 22: 12,
    23: 4, 24: 8, 25: 24, 26: 24, 27: 2, 28: 20, 29: 15,
    30: 2, 31: 14, 32: 9
}

# =====
# FUNGSI DASAR
# =====

def euclidean_distance(i, j):
    xi, yi = coords[i]
    xj, yj = coords[j]

    # EUC_2D dibulatkan ke integer terdekat
    return int(math.sqrt((xi - xj) ** 2 + (yi - yj) ** 2) + 0.5)

def build_distance_matrix():
    D = {}

    for i in coords:
        for j in coords:
            D[i, j] = euclidean_distance(i, j)

    return D

def route_load(route):
    return sum(Demand[node] for node in route)

def route_distance(route, D):
    full_route = [DEPOT] + route + [DEPOT]
    total = 0
```

```

    for i in range(len(full_route) - 1):
        total += D[full_route[i], full_route[i + 1]]

    return total

def total_distance(routes, D):
    return sum(route_distance(route, D) for route in routes)

# =====
# VALIDASI SOLUSI
# =====

def validate_solution(routes):
    all_customers = []

    for route in routes:
        all_customers.extend(route)

    expected_customers = set(coords.keys()) - {DEPOT}
    actual_customers = set(all_customers)

    print("\nVALIDASI SOLUSI CWS")

    if actual_customers == expected_customers and len(all_customers) ==
len(set(all_customers)):
        print("Semua pelanggan dilayani tepat satu kali.")
    else:
        print("PERINGATAN: Ada pelanggan yang hilang atau terlayani ganda.")

        missing = expected_customers - actual_customers
        duplicate = [x for x in all_customers if all_customers.count(x) > 1]

        if missing:
            print("Pelanggan hilang:", missing)

        if duplicate:
            print("Pelanggan ganda:", set(duplicate))

    for idx, route in enumerate(routes, start=1):
        load = route_load(route)

        if load <= CAPACITY:
            print(f"Kendaraan {idx}: kapasitas aman ({load}/{CAPACITY})")
        else:
            print(f"Kendaraan {idx}: melebihi kapasitas ({load}/{CAPACITY})")

    if len(routes) <= MAX_TRUCKS:
        print(f"Jumlah kendaraan digunakan: {len(routes)} dari maksimum {MAX_TRUCKS}")
    else:
        print(f"PERINGATAN: jumlah kendaraan {len(routes)} melebihi maksimum
{MAX_TRUCKS}")

# =====
# CLARKE-WRIGHT SAVINGS
# =====

def clarke_wright_savings():
    D = build_distance_matrix()

    customers = [node for node in coords if node != DEPOT]

    # Solusi awal: setiap pelanggan menjadi satu rute
    routes = {}
    loads = {}

```

```

customer_to_route = {}

for customer in customers:
    routes[customer] = [customer]
    loads[customer] = Demand[customer]
    customer_to_route[customer] = customer

# Menghitung nilai saving
savings = []

for idx, i in enumerate(customers):
    for j in customers[idx + 1:]:
        saving_value = D[DEPOT, i] + D[DEPOT, j] - D[i, j]
        savings.append((saving_value, i, j))

# Urutkan saving dari terbesar ke terkecil
savings.sort(reverse=True)

# Proses penggabungan rute
for saving_value, i, j in savings:
    route_i_id = customer_to_route[i]
    route_j_id = customer_to_route[j]

    if route_i_id == route_j_id:
        continue

    route_i = routes[route_i_id]
    route_j = routes[route_j_id]

    # Cek kapasitas kendaraan
    if loads[route_i_id] + loads[route_j_id] > CAPACITY:
        continue

    merged_route = None

    # Penggabungan hanya jika i dan j berada pada ujung rute
    if route_i[-1] == i and route_j[0] == j:
        merged_route = route_i + route_j

    elif route_i[0] == i and route_j[-1] == j:
        merged_route = route_j + route_i

    elif route_i[0] == i and route_j[0] == j:
        merged_route = list(reversed(route_i)) + route_j

    elif route_i[-1] == i and route_j[-1] == j:
        merged_route = route_i + list(reversed(route_j))

    if merged_route is None:
        continue

    routes[route_i_id] = merged_route
    loads[route_i_id] += loads[route_j_id]

    for customer in merged_route:
        customer_to_route[customer] = route_i_id

    del routes[route_j_id]
    del loads[route_j_id]

final_routes = list(routes.values())

# Urutkan berdasarkan muatan terbesar agar tampilan rapi
final_routes.sort(key=lambda r: route_load(r), reverse=True)

```

```

    return final_routes, D

# =====
# CETAK HASIL CWS
# =====

def print_cws_solution(routes, D):
    print("\n" + "=" * 70)
    print("HASIL CLARKE-WRIGHT SAVINGS")
    print("=" * 70)

    total_jarak = 0

    for idx, route in enumerate(routes, start=1):
        full_route = [DEPOT] + route + [DEPOT]
        load = route_load(route)
        jarak = route_distance(route, D)

        total_jarak += jarak

        print(f"\nKendaraan {idx}")
        print("Rute      :", " -> ".join(map(str, full_route)))
        print("Kapasitas :", load, "/", CAPACITY)
        print("Jarak      :", jarak)

    print("\n" + "-" * 70)
    print("Total kendaraan :", len(routes))
    print("Total Demand    :", sum(Demand.values()))
    print("Total jarak     :", total_jarak)
    print("-" * 70)

# =====
# PLOT RUTE CWS
# =====

def plot_cws_routes(routes):
    try:
        import matplotlib.pyplot as plt
    except ImportError:
        print("\nMatplotlib belum terpasang. Plot rute tidak ditampilkan.")
        print("Install matplotlib dengan perintah: pip install matplotlib")
        return

    plt.figure(figsize=(11, 8))

    # Plot depot dan pelanggan
    for node, (x, y) in coords.items():
        if node == DEPOT:
            plt.scatter(x, y, s=200, marker="s")
            plt.text(x + 1, y + 1, f"Depot {node}", fontsize=9)
        else:
            plt.scatter(x, y, s=35)
            plt.text(x + 1, y + 1, str(node), fontsize=7)

    # Plot setiap rute kendaraan
    for idx, route in enumerate(routes, start=1):
        full_route = [DEPOT] + route + [DEPOT]

        xs = [coords[node][0] for node in full_route]
        ys = [coords[node][1] for node in full_route]

        plt.plot(
            xs,

```

```

        ys,
        linewidth=1.5,
        marker="o",
        markersize=3,
        label=f"Kendaraan {idx}"
    )

plt.title("Plot Rute Hasil Clarke-Wright Savings A-n32-k5")
plt.xlabel("Koordinat X")
plt.ylabel("Koordinat Y")
plt.legend(fontsize=8)
plt.grid(True)
plt.tight_layout()
plt.show()

# =====
# PROGRAM UTAMA
# =====

if __name__ == "__main__":
    routes, D = clarke_wright_savings()

    print_cws_solution(routes, D)
    validate_solution(routes)
    plot_cws_routes(routes)

```

VNS 1 data 32

```

import math
import random
import matplotlib.pyplot as plt

# =====
# DATA CVRP A-n32-k5
# Depot = titik 1
# Kapasitas kendaraan = 100
# =====
# Parameter VNS
MAX_ITER = 2500
K_MAX = 3
TOTAL_SEED = 1

coords = {
    1: (82, 76), 2: (96, 44), 3: (50, 5), 4: (49, 8), 5: (13, 7),
    6: (29, 89), 7: (58, 30), 8: (84, 39), 9: (14, 24), 10: (2, 39),
    11: (3, 82), 12: (5, 10), 13: (98, 52), 14: (84, 25), 15: (61, 59),
    16: (1, 65), 17: (88, 51), 18: (91, 2), 19: (19, 32), 20: (93, 3),
    21: (50, 93), 22: (98, 14), 23: (5, 42), 24: (42, 9), 25: (61, 62),
    26: (9, 97), 27: (80, 55), 28: (57, 69), 29: (23, 15), 30: (20, 70),
    31: (85, 60), 32: (98, 5)
}

Demand = {
    1: 0, 2: 19, 3: 21, 4: 6, 5: 19, 6: 7, 7: 12, 8: 16,
    9: 6, 10: 16, 11: 8, 12: 14, 13: 21, 14: 16, 15: 3,
    16: 22, 17: 18, 18: 19, 19: 1, 20: 24, 21: 8, 22: 12,
    23: 4, 24: 8, 25: 24, 26: 24, 27: 2, 28: 20, 29: 15,
    30: 2, 31: 14, 32: 9
}

DEPOT = 1
CAPACITY = 100

```

```

# =====
# BOBOT FUNGSI TUJUAN
# =====
# Jika fokus hanya emisi:
ALPHA = 0
BETA = 1

# =====
# SOLUSI AWAL HASIL CLARKE-WRIGHT SAVINGS
# =====

initial_solution = [
    [24, 4, 3, 18, 20, 32, 22],
    [27, 7, 19, 29, 5, 12, 9, 10, 23, 15],
    [21, 6, 26, 11, 16, 30, 28],
    [17, 8, 14, 2, 13],
    [25, 31]
]

# =====
# FUNGSI PERHITUNGAN DASAR
# =====

def distance(i, j):
    xi, yi = coords[i]
    xj, yj = coords[j]
    return int(round(math.sqrt((xi - xj) ** 2 + (yi - yj) ** 2)))

def route_load(route):
    return sum(Demand[node] for node in route)

def route_distance(route):
    total = 0
    previous = DEPOT

    for node in route:
        total += distance(previous, node)
        previous = node

    total += distance(previous, DEPOT)
    return total

def route_emission(route):
    """
    Emisi dihitung berdasarkan beban kendaraan saat melewati ruas.

    Rumus:
     $E = (0.15 + 0.005 \times \text{load}) \times d$ 

    Load awal adalah total Demand dalam rute.
    Setelah kendaraan melayani pelanggan, load berkurang.
    """
    load = route_load(route)
    total_emission = 0
    previous = DEPOT

    for node in route:
        d = distance(previous, node)
        emission = (0.15 + 0.005 * load) * d
        total_emission += emission

        load -= Demand[node]
        previous = node

```

```

        # Perjalanan kembali ke depot setelah seluruh barang terkirim
        d = distance(previous, DEPOT)
        emission = (0.15 + 0.005 * load) * d
        total_emission += emission

    return total_emission

def total_distance(solution):
    return sum(route_distance(route) for route in solution)

def total_emission(solution):
    return sum(route_emission(route) for route in solution)

def objective(solution):
    return ALPHA * total_distance(solution) + BETA * total_emission(solution)

def is_feasible(solution):
    all_customers = []

    for route in solution:
        if route_load(route) > CAPACITY:
            return False

        all_customers.extend(route)

    return sorted(all_customers) == list(range(2, 33))

def copy_solution(solution):
    return [route[:] for route in solution]

# =====
# NEIGHBORHOOD 1: 2-OPT
# Membalik urutan sebagian rute
# =====

def best_two_opt(solution):
    best_solution = None
    best_delta = 0

    for r in range(len(solution)):
        route = solution[r]
        old_obj = objective([route])

        for i in range(len(route) - 1):
            for j in range(i + 1, len(route)):
                new_route = route[:i] + list(reversed(route[i:j + 1])) + route[j + 1:]

                new_obj = objective([new_route])
                delta = new_obj - old_obj

                if delta < best_delta:
                    candidate = copy_solution(solution)
                    candidate[r] = new_route
                    best_solution = candidate
                    best_delta = delta

    return best_solution

# =====
# NEIGHBORHOOD 2: RELOCATE
# Memindahkan satu pelanggan ke posisi lain
# =====

def best_relocate(solution):

```

```

best_solution = None
best_delta = 0
current_obj = objective(solution)

for r1 in range(len(solution)):
    for i in range(len(solution[r1])):
        for r2 in range(len(solution)):
            for j in range(len(solution[r2]) + 1):

                candidate = copy_solution(solution)
                moved_node = candidate[r1].pop(i)

                if r1 == r2 and j > i:
                    j -= 1

                candidate[r2].insert(j, moved_node)

                if candidate == solution:
                    continue

                if not is_feasible(candidate):
                    continue

                new_obj = objective(candidate)
                delta = new_obj - current_obj

                if delta < best_delta:
                    best_solution = candidate
                    best_delta = delta

    return best_solution

# =====
# NEIGHBORHOOD 3: SWAP
# Menukar dua pelangan
# =====

def best_swap(solution):
    best_solution = None
    best_delta = 0
    current_obj = objective(solution)

    for r1 in range(len(solution)):
        for i in range(len(solution[r1])):
            for r2 in range(r1, len(solution)):
                start_j = i + 1 if r1 == r2 else 0

                for j in range(start_j, len(solution[r2])):
                    candidate = copy_solution(solution)

                    candidate[r1][i], candidate[r2][j] = candidate[r2][j],
candidate[r1][i]

                    if not is_feasible(candidate):
                        continue

                    new_obj = objective(candidate)
                    delta = new_obj - current_obj

                    if delta < best_delta:
                        best_solution = candidate
                        best_delta = delta

    return best_solution

```

```

# =====
# LOCAL SEARCH / VND
# =====

def local_search(solution):
    current_solution = copy_solution(solution)

    neighborhoods = [
        best_two_opt,
        best_relocate,
        best_swap
    ]

    k = 0

    while k < len(neighborhoods):
        new_solution = neighborhoods[k](current_solution)

        if new_solution is not None and objective(new_solution) <
objective(current_solution):
            current_solution = new_solution
            k = 0
        else:
            k += 1

    return current_solution

# =====
# SHAKING VNS
# =====

def random_two_opt(solution):
    candidate = copy_solution(solution)
    available_routes = [i for i, route in enumerate(candidate) if len(route) >= 2]

    if not available_routes:
        return candidate

    r = random.choice(available_routes)
    i, j = sorted(random.sample(range(len(candidate[r])), 2))

    candidate[r][i:j + 1] = list(reversed(candidate[r][i:j + 1]))

    return candidate

def random_relocate(solution):
    candidate = copy_solution(solution)
    non_empty_routes = [i for i, route in enumerate(candidate) if len(route) > 0]

    for _ in range(100):
        r1 = random.choice(non_empty_routes)
        i = random.randrange(len(candidate[r1]))
        node = candidate[r1][i]

        r2 = random.randrange(len(candidate))

        if r1 != r2:
            if route_load(candidate[r2]) + Demand[node] > CAPACITY:
                continue

        new_candidate = copy_solution(candidate)
        moved_node = new_candidate[r1].pop(i)

```

```

        j = random.randrange(len(new_candidate[r2]) + 1)
        new_candidate[r2].insert(j, moved_node)

        if is_feasible(new_candidate):
            return new_candidate

    return candidate

def random_swap(solution):
    candidate = copy_solution(solution)

    for _ in range(100):
        r1, r2 = random.sample(range(len(candidate)), 2)

        if len(candidate[r1]) == 0 or len(candidate[r2]) == 0:
            continue

        i = random.randrange(len(candidate[r1]))
        j = random.randrange(len(candidate[r2]))

        node1 = candidate[r1][i]
        node2 = candidate[r2][j]

        load_r1 = route_load(candidate[r1]) - Demand[node1] + Demand[node2]
        load_r2 = route_load(candidate[r2]) - Demand[node2] + Demand[node1]

        if load_r1 <= CAPACITY and load_r2 <= CAPACITY:
            candidate[r1][i], candidate[r2][j] = candidate[r2][j], candidate[r1][i]
            return candidate

    return candidate

def shake(solution, k):
    if k == 0:
        return random_two_opt(solution)
    elif k == 1:
        return random_relocate(solution)
    else:
        return random_swap(solution)

# =====
# ALGORITHM VARIABLE NEIGHBORHOOD SEARCH
# =====

def vns(initial_solution, max_iter=100, k_max=3, seed=1):
    random.seed(seed)

    best_solution = local_search(initial_solution)
    best_obj = objective(best_solution)

    iteration = 0

    while iteration < max_iter:
        k = 0

        while k < k_max:
            shaken_solution = shake(best_solution, k)

            if not is_feasible(shaken_solution):
                k += 1
                continue

            local_solution = local_search(shaken_solution)
            local_obj = objective(local_solution)

```

```

        if local_obj < best_obj:
            best_solution = local_solution
            best_obj = local_obj
            k = 0
        else:
            k += 1

    iteration += 1

    return best_solution

# =====
# CETAK HASIL
# =====

def print_solution(title, solution):
    print("=" * 75)
    print(title)
    print("=" * 75)

    for idx, route in enumerate(solution, start=1):
        full_route = [DEPOT] + route + [DEPOT]

        print(f"\nKendaraan {idx}")
        print("Rute      :", " -> ".join(map(str, full_route)))
        print(f"Kapasitas : {route_load(route)} / {CAPACITY}")
        print(f"Jarak      : {route_distance(route)}")
        print(f"Emisi      : {route_emission(route):.3f}")

    print("\n" + "-" * 75)
    print(f"Total kendaraan : {len(solution)}")
    print(f"Total Demand    : {sum(route_load(route) for route in solution)}")
    print(f"Total jarak     : {total_distance(solution)}")
    print(f"Total emisi     : {total_emission(solution):.3f}")
    print(f"Nilai objektif   : {objective(solution):.3f}")
    print("-" * 75)

# =====
# VALIDASI HASIL
# =====

def validate_solution(solution, title):
    print("\nVALIDASI", title)

    if is_feasible(solution):
        print("Semua pelanggan dilayani tepat satu kali.")

        for i, route in enumerate(solution, start=1):
            print(f"Kendaraan {i}: kapasitas aman ({route_load(route)}/{CAPACITY}")

        print(f"Jumlah kendaraan digunakan: {len(solution)}")
    else:
        print("Solusi tidak feasible.")

# =====
# PLOT RUTE
# =====

def plot_solution(solution, title):
    plt.figure(figsize=(10, 8))

    # Plot semua titik pelanggan
    for node, (x, y) in coords.items():

```

```

    if node == DEPOT:
        plt.scatter(x, y, s=180, marker="s", label="Depot")
        plt.text(x + 1, y + 1, str(node), fontsize=10, fontweight="bold")
    else:
        plt.scatter(x, y, s=60)
        plt.text(x + 1, y + 1, str(node), fontsize=9)

# Plot setiap rute kendaraan
for idx, route in enumerate(solution, start=1):
    full_route = [DEPOT] + route + [DEPOT]

    x_values = [coords[node][0] for node in full_route]
    y_values = [coords[node][1] for node in full_route]

    plt.plot(
        x_values,
        y_values,
        marker="o",
        linewidth=2,
        label=f"Kendaraan {idx}"
    )

# Tambahkan arah panah pada setiap ruas
for a, b in zip(full_route[:-1], full_route[1:]):
    x1, y1 = coords[a]
    x2, y2 = coords[b]

    plt.annotate(
        "",
        xy=(x2, y2),
        xytext=(x1, y1),
        arrowprops=dict(
            arrowstyle="->",
            lw=1,
            shrinkA=8,
            shrinkB=8
        )
    )

plt.title(title, fontsize=14, fontweight="bold")
plt.xlabel("Koordinat X")
plt.ylabel("Koordinat Y")
plt.grid(True, linestyle="--", alpha=0.5)
plt.legend()
plt.tight_layout()
plt.show()

# =====
# PLOT PERBANDINGAN CWS DAN VNS
# =====

def plot_comparison(cws_solution, vns_solution):
    labels = ["Total Jarak", "Total Emisi"]

    cws_values = [
        total_distance(cws_solution),
        total_emission(cws_solution)
    ]

    vns_values = [
        total_distance(vns_solution),
        total_emission(vns_solution)
    ]

```

```

x = range(len(labels))
width = 0.35

plt.figure(figsize=(8, 6))

plt.bar(
    [i - width / 2 for i in x],
    cws_values,
    width=width,
    label="CWS"
)

plt.bar(
    [i + width / 2 for i in x],
    vns_values,
    width=width,
    label="VNS"
)

plt.xticks(list(x), labels)
plt.ylabel("Nilai")
plt.title("Perbandingan Hasil CWS dan VNS", fontsize=14, fontweight="bold")
plt.grid(axis="y", linestyle="--", alpha=0.5)
plt.legend()

for i, value in enumerate(cws_values):
    plt.text(i - width / 2, value, f"{value:.2f}", ha="center", va="bottom",
    fontsize=9)

for i, value in enumerate(vns_values):
    plt.text(i + width / 2, value, f"{value:.2f}", ha="center", va="bottom",
    fontsize=9)

plt.tight_layout()
plt.show()

# =====
# EKSEKUSI PROGRAM
# =====

print_solution("HASIL AWAL CLARKE-WRIGHT SAVINGS", initial_solution)
validate_solution(initial_solution, "SOLUSI CWS")

best_solution = vns(
    initial_solution=initial_solution,
    max_iter=100,
    k_max=3,
    seed=1
)

print_solution("HASIL OPTIMASI VARIABLE NEIGHBORHOOD SEARCH", best_solution)
validate_solution(best_solution, "SOLUSI VNS")

print("\nPERBANDINGAN CWS DAN VNS")
print("-" * 75)

cws_distance = total_distance(initial_solution)
vns_distance = total_distance(best_solution)

cws_emission = total_emission(initial_solution)
vns_emission = total_emission(best_solution)

distance_reduction = cws_distance - vns_distance
emission_reduction = cws_emission - vns_emission

```

```

distance_percentage = (distance_reduction / cws_distance) * 100
emission_percentage = (emission_reduction / cws_emission) * 100

print(f"Total jarak CWS      : {cws_distance}")
print(f"Total jarak VNS      : {vns_distance}")
print(f"Penurunan jarak       : {distance_reduction}")
print(f"Persentase penurunan : {distance_percentage:.2f}%")

print()

print(f"Total emisi CWS      : {cws_emission:.3f}")
print(f"Total emisi VNS      : {vns_emission:.3f}")
print(f"Penurunan emisi       : {emission_reduction:.3f}")
print(f"Persentase penurunan : {emission_percentage:.2f}%")
print("-" * 75)

# =====
# TAMPILKAN PLOT
# =====

plot_solution(initial_solution, "Plot Rute Awal Clarke-Wright Savings")
plot_solution(best_solution, "Plot Rute Hasil Optimasi VNS")
plot_comparison(initial_solution, best_solution)

```

Vns 2 data 32

```

import math
import random
import matplotlib.pyplot as plt

# =====
# DATA CVRP A-n32-k5
# Depot = titik 1
# =====
# Parameter VNS
MAX_ITER = 2500
K_MAX = 3
TOTAL_SEED = 1

coords = {
    1: (82, 76), 2: (96, 44), 3: (50, 5), 4: (49, 8), 5: (13, 7),
    6: (29, 89), 7: (58, 30), 8: (84, 39), 9: (14, 24), 10: (2, 39),
    11: (3, 82), 12: (5, 10), 13: (98, 52), 14: (84, 25), 15: (61, 59),
    16: (1, 65), 17: (88, 51), 18: (91, 2), 19: (19, 32), 20: (93, 3),
    21: (50, 93), 22: (98, 14), 23: (5, 42), 24: (42, 9), 25: (61, 62),
    26: (9, 97), 27: (80, 55), 28: (57, 69), 29: (23, 15), 30: (20, 70),
    31: (85, 60), 32: (98, 5)
}

Demand = {
    1: 0, 2: 19, 3: 21, 4: 6, 5: 19, 6: 7, 7: 12, 8: 16,
    9: 6, 10: 16, 11: 8, 12: 14, 13: 21, 14: 16, 15: 3,
    16: 22, 17: 18, 18: 19, 19: 1, 20: 24, 21: 8, 22: 12,
    23: 4, 24: 8, 25: 24, 26: 24, 27: 2, 28: 20, 29: 15,
    30: 2, 31: 14, 32: 9
}

DEPOT = 1

# =====
# DATA KENDARAAN
# Koefisien emisi berbeda berdasarkan tahun keluaran kendaraan
# E = (a + b x load) x d

```

```

# =====

vehicles = [
    {"nama": "Kendaraan 1", "tahun": 2024, "capacity": 100, "a": 0.150, "b": 0.0050},
    {"nama": "Kendaraan 2", "tahun": 2021, "capacity": 100, "a": 0.165, "b": 0.0055},
    {"nama": "Kendaraan 3", "tahun": 2018, "capacity": 100, "a": 0.185, "b": 0.0062},
    {"nama": "Kendaraan 4", "tahun": 2014, "capacity": 100, "a": 0.215, "b": 0.0072},
    {"nama": "Kendaraan 5", "tahun": 2010, "capacity": 100, "a": 0.250, "b": 0.0085}
]

# =====
# BOBOT FUNGSI TUJUAN
# ALPHA = bobot jarak
# BETA = bobot emisi
# Jika fokus emisi, gunakan ALPHA = 0 dan BETA = 1
# =====

ALPHA = 0
BETA = 1

# =====
# SOLUSI AWAL HASIL CLARKE-WRIGHT SAVINGS
# =====

initial_solution = [
    [24, 4, 3, 18, 20, 32, 22],
    [27, 7, 19, 29, 5, 12, 9, 10, 23, 15],
    [21, 6, 26, 11, 16, 30, 28],
    [17, 8, 14, 2, 13],
    [25, 31]
]

# =====
# FUNGSI DASAR
# =====

def distance(i, j):
    xi, yi = coords[i]
    xj, yj = coords[j]
    return int(round(math.sqrt((xi - xj) ** 2 + (yi - yj) ** 2)))

def route_load(route):
    return sum(Demand[node] for node in route)

def route_distance(route):
    total = 0
    previous = DEPOT

    for node in route:
        total += distance(previous, node)
        previous = node

    total += distance(previous, DEPOT)
    return total

def route_emission(route, vehicle_index):
    vehicle = vehicles[vehicle_index]

    a = vehicle["a"]
    b = vehicle["b"]

    load = route_load(route)
    total = 0
    previous = DEPOT

```

```

    for node in route:
        d = distance(previous, node)
        e = (a + b * load) * d
        total += e

        load -= Demand[node]
        previous = node

    d = distance(previous, DEPOT)
    e = (a + b * load) * d
    total += e

    return total

def total_distance(solution):
    return sum(route_distance(route) for route in solution)

def total_emission(solution):
    total = 0

    for vehicle_index, route in enumerate(solution):
        total += route_emission(route, vehicle_index)

    return total

def objective(solution):
    return ALPHA * total_distance(solution) + BETA * total_emission(solution)

def route_objective(route, vehicle_index):
    return ALPHA * route_distance(route) + BETA * route_emission(route, vehicle_index)

def copy_solution(solution):
    return [route[:] for route in solution]

def is_feasible(solution):
    if len(solution) != len(vehicles):
        return False

    all_customers = []

    for vehicle_index, route in enumerate(solution):
        if route_load(route) > vehicles[vehicle_index]["capacity"]:
            return False

        all_customers.extend(route)

    return sorted(all_customers) == list(range(2, 33))

# =====
# NEIGHBORHOOD 1: ROUTE EXCHANGE
# Menukar seluruh rute antar kendaraan
# Penting karena tiap kendaraan memiliki koefisien emisi berbeda
# =====

def best_route_exchange(solution):
    best_solution = None
    best_delta = 0
    current_obj = objective(solution)

    for r1 in range(len(solution) - 1):
        for r2 in range(r1 + 1, len(solution)):
            load_r1 = route_load(solution[r1])
            load_r2 = route_load(solution[r2])

```

```

        if load_r2 > vehicles[r1]["capacity"]:
            continue

        if load_r1 > vehicles[r2]["capacity"]:
            continue

        candidate = copy_solution(solution)
        candidate[r1], candidate[r2] = candidate[r2], candidate[r1]

        if not is_feasible(candidate):
            continue

        new_obj = objective(candidate)
        delta = new_obj - current_obj

        if delta < best_delta:
            best_solution = candidate
            best_delta = delta

    return best_solution

# =====
# NEIGHBORHOOD 2: 2-OPT
# Membalik sebagian urutan pelanggan dalam satu rute
# =====

def best_two_opt(solution):
    best_solution = None
    best_delta = 0

    for vehicle_index in range(len(solution)):
        route = solution[vehicle_index]
        old_obj = route_objective(route, vehicle_index)

        for i in range(len(route) - 1):
            for j in range(i + 1, len(route)):
                new_route = route[:i] + list(reversed(route[i:j + 1])) + route[j + 1:]
                new_obj = route_objective(new_route, vehicle_index)
                delta = new_obj - old_obj

                if delta < best_delta:
                    candidate = copy_solution(solution)
                    candidate[vehicle_index] = new_route
                    best_solution = candidate
                    best_delta = delta

    return best_solution

# =====
# NEIGHBORHOOD 3: RELOCATE
# Memindahkan satu pelanggan ke posisi lain
# =====

def best_relocate(solution):
    best_solution = None
    best_delta = 0
    current_obj = objective(solution)

    for r1 in range(len(solution)):
        for i in range(len(solution[r1])):
            node = solution[r1][i]

            for r2 in range(len(solution)):

```

```

        if r1 != r2:
            if route_load(solution[r2]) + Demand[node] >
vehicles[r2]["capacity"]:
                continue

        for j in range(len(solution[r2]) + 1):
            if r1 == r2 and (j == i or j == i + 1):
                continue

            candidate = copy_solution(solution)
            moved_node = candidate[r1].pop(i)

            if r1 == r2 and j > i:
                j -= 1

            candidate[r2].insert(j, moved_node)

            if not is_feasible(candidate):
                continue

            new_obj = objective(candidate)
            delta = new_obj - current_obj

            if delta < best_delta:
                best_solution = candidate
                best_delta = delta

    return best_solution

# =====
# NEIGHBORHOOD 4: SWAP
# Menukar dua pelanggan
# =====

def best_swap(solution):
    best_solution = None
    best_delta = 0
    current_obj = objective(solution)

    route_loads = [route_load(route) for route in solution]

    for r1 in range(len(solution)):
        for i in range(len(solution[r1])):
            node1 = solution[r1][i]

            for r2 in range(r1, len(solution)):
                start_j = i + 1 if r1 == r2 else 0

                for j in range(start_j, len(solution[r2])):
                    node2 = solution[r2][j]

                    if r1 != r2:
                        new_load_r1 = route_loads[r1] - Demand[node1] + Demand[node2]
                        new_load_r2 = route_loads[r2] - Demand[node2] + Demand[node1]

                        if new_load_r1 > vehicles[r1]["capacity"]:
                            continue

                        if new_load_r2 > vehicles[r2]["capacity"]:
                            continue

                    candidate = copy_solution(solution)
                    candidate[r1][i], candidate[r2][j] = candidate[r2][j],
candidate[r1][i]

```

```

        if not is_feasible(candidate):
            continue

        new_obj = objective(candidate)
        delta = new_obj - current_obj

        if delta < best_delta:
            best_solution = candidate
            best_delta = delta

    return best_solution

# =====
# LOCAL SEARCH / VND
# =====

def local_search(solution):
    current_solution = copy_solution(solution)

    neighborhoods = [
        best_route_exchange,
        best_two_opt,
        best_relocate,
        best_swap
    ]

    k = 0

    while k < len(neighborhoods):
        new_solution = neighborhoods[k](current_solution)

        if new_solution is not None and objective(new_solution) <
objective(current_solution):
            current_solution = new_solution
            k = 0
        else:
            k += 1

    return current_solution

# =====
# SHAKING VNS
# =====

def random_route_exchange(solution):
    candidate = copy_solution(solution)
    r1, r2 = random.sample(range(len(candidate)), 2)
    candidate[r1], candidate[r2] = candidate[r2], candidate[r1]

    if is_feasible(candidate):
        return candidate

    return copy_solution(solution)

def random_two_opt(solution):
    candidate = copy_solution(solution)

    available_routes = [
        i for i, route in enumerate(candidate)
        if len(route) >= 2
    ]

    if not available_routes:

```

```

        return candidate

    r = random.choice(available_routes)
    i, j = sorted(random.sample(range(len(candidate[r])), 2))

    candidate[r][i:j + 1] = list(reversed(candidate[r][i:j + 1]))

    return candidate

def random_relocate(solution):
    for _ in range(100):
        candidate = copy_solution(solution)

        non_empty_routes = [
            i for i, route in enumerate(candidate)
            if len(route) > 0
        ]

        r1 = random.choice(non_empty_routes)
        i = random.randrange(len(candidate[r1]))

        node = candidate[r1][i]
        r2 = random.randrange(len(candidate))

        if r1 != r2:
            if route_load(candidate[r2]) + Demand[node] > vehicles[r2]["capacity"]:
                continue

            moved_node = candidate[r1].pop(i)
            j = random.randrange(len(candidate[r2]) + 1)
            candidate[r2].insert(j, moved_node)

            if is_feasible(candidate):
                return candidate

    return copy_solution(solution)

def random_swap(solution):
    for _ in range(100):
        candidate = copy_solution(solution)

        r1, r2 = random.sample(range(len(candidate)), 2)

        if len(candidate[r1]) == 0 or len(candidate[r2]) == 0:
            continue

        i = random.randrange(len(candidate[r1]))
        j = random.randrange(len(candidate[r2]))

        candidate[r1][i], candidate[r2][j] = candidate[r2][j], candidate[r1][i]

        if is_feasible(candidate):
            return candidate

    return copy_solution(solution)

def shake(solution, k):
    if k == 0:
        return random_route_exchange(solution)
    elif k == 1:
        return random_two_opt(solution)
    elif k == 2:
        return random_relocate(solution)
    else:

```

```

        return random_swap(solution)

# =====
# ALGORITMA VARIABLE NEIGHBORHOOD SEARCH
# =====

def vns(initial_solution, max_iter=100, k_max=4, seed=1):
    random.seed(seed)

    best_solution = local_search(initial_solution)
    best_obj = objective(best_solution)

    history = [best_obj]

    for iteration in range(max_iter):
        k = 0

        while k < k_max:
            shaken_solution = shake(best_solution, k)

            if not is_feasible(shaken_solution):
                k += 1
                continue

            local_solution = local_search(shaken_solution)
            local_obj = objective(local_solution)

            if local_obj < best_obj:
                best_solution = local_solution
                best_obj = local_obj
                k = 0
            else:
                k += 1

        history.append(best_obj)

    return best_solution, history

# =====
# CETAK DATA KENDARAAN
# =====

def print_vehicle_data():
    print("=" * 90)
    print("DATA KENDARAAN DAN KOEFISIEN EMISI")
    print("=" * 90)

    for vehicle in vehicles:
        print(
            f"{vehicle['nama']} | "
            f"Tahun: {vehicle['tahun']} | "
            f"Kapasitas: {vehicle['capacity']} | "
            f"a: {vehicle['a']} | "
            f"b: {vehicle['b']}"
        )

    print("=" * 90)

# =====
# CETAK HASIL SOLUSI
# =====

def print_solution(title, solution):
    print("\n" + "=" * 90)

```

```

print(title)
print("=" * 90)

for vehicle_index, route in enumerate(solution):
    vehicle = vehicles[vehicle_index]
    full_route = [DEPOT] + route + [DEPOT]

    print(f"\n{vehicle['nama']}")
    print(f"Tahun      : {vehicle['tahun']}")
    print(f"Koefisien : a = {vehicle['a']}, b = {vehicle['b']}")
    print(f"Rute       :", " -> ".join(map(str, full_route)))
    print(f"Kapasitas : {route_load(route)} / {vehicle['capacity']}")
    print(f"Jarak      : {route_distance(route)}")
    print(f"Emisi      : {route_emission(route, vehicle_index):.4f}")

print("\n" + "-" * 90)
print(f"Total kendaraan : {len(solution)}")
print(f"Total Demand    : {sum(route_load(route) for route in solution)}")
print(f"Total jarak     : {total_distance(solution)}")
print(f"Total emisi     : {total_emission(solution):.4f}")
print(f"Nilai objektif  : {objective(solution):.4f}")
print("-" * 90)

# =====
# VALIDASI SOLUSI
# =====

def validate_solution(solution, title):
    print("\nVALIDASI", title)

    if is_feasible(solution):
        print("Semua pelanggan dilayani tepat satu kali.")

        for vehicle_index, route in enumerate(solution):
            vehicle = vehicles[vehicle_index]
            print(
                f"{vehicle['nama']}: kapasitas aman "
                f"{(route_load(route)/vehicle['capacity'])}"
            )

        print(f"Jumlah kendaraan digunakan: {len(solution)} dari {len(vehicles)}")
    else:
        print("Solusi tidak feasible.")

# =====
# CETAK PERBANDINGAN
# =====

def print_comparison(cws_solution, vns_solution):
    cws_distance = total_distance(cws_solution)
    vns_distance = total_distance(vns_solution)

    cws_emission = total_emission(cws_solution)
    vns_emission = total_emission(vns_solution)

    distance_reduction = cws_distance - vns_distance
    emission_reduction = cws_emission - vns_emission

    distance_percentage = (distance_reduction / cws_distance) * 100
    emission_percentage = (emission_reduction / cws_emission) * 100

    print("\n" + "=" * 90)
    print("PERBANDINGAN HASIL CWS DAN VNS")
    print("=" * 90)

```

```

print(f"Total jarak CWS      : {cws_distance}")
print(f"Total jarak VNS      : {vns_distance}")
print(f"Selisih jarak         : {distance_reduction}")
print(f"Persentase jarak      : {distance_percentage:.2f}%")

print()

print(f"Total emisi CWS        : {cws_emission:.4f}")
print(f"Total emisi VNS          : {vns_emission:.4f}")
print(f"Penurunan emisi          : {emission_reduction:.4f}")
print(f"Persentase emisi         : {emission_percentage:.2f}%")

print("=" * 90)

# =====
# PLOT RUTE
# =====

def plot_solution(solution, title, filename):
    plt.figure(figsize=(10, 8))

    for node, (x, y) in coords.items():
        if node == DEPOT:
            plt.scatter(x, y, s=180, marker="s", label="Depot")
            plt.text(x + 1, y + 1, str(node), fontsize=10, fontweight="bold")
        else:
            plt.scatter(x, y, s=60)
            plt.text(x + 1, y + 1, str(node), fontsize=9)

    for vehicle_index, route in enumerate(solution):
        vehicle = vehicles[vehicle_index]
        full_route = [DEPOT] + route + [DEPOT]

        x_values = [coords[node][0] for node in full_route]
        y_values = [coords[node][1] for node in full_route]

        label_text = f"{vehicle['nama']} ({vehicle['tahun']})"

        plt.plot(
            x_values,
            y_values,
            marker="o",
            linewidth=2,
            label=label_text
        )

    for a, b in zip(full_route[:-1], full_route[1:]):
        x1, y1 = coords[a]
        x2, y2 = coords[b]

        plt.annotate(
            "",
            xy=(x2, y2),
            xytext=(x1, y1),
            arrowprops=dict(
                arrowstyle="->",
                lw=1,
                shrinkA=8,
                shrinkB=8
            )
        )

    plt.title(title, fontsize=14, fontweight="bold")

```

```

plt.xlabel("Koordinat X")
plt.ylabel("Koordinat Y")
plt.grid(True, linestyle="--", alpha=0.5)
plt.legend()
plt.tight_layout()
plt.savefig(filename, dpi=300)
plt.show()

# =====
# PLOT PERBANDINGAN CWS DAN VNS
# =====

def plot_comparison(cws_solution, vns_solution):
    labels = ["Total Jarak", "Total Emisi"]

    cws_values = [
        total_distance(cws_solution),
        total_emission(cws_solution)
    ]

    vns_values = [
        total_distance(vns_solution),
        total_emission(vns_solution)
    ]

    x = range(len(labels))
    width = 0.35

    plt.figure(figsize=(8, 6))

    plt.bar(
        [i - width / 2 for i in x],
        cws_values,
        width=width,
        label="CWS"
    )

    plt.bar(
        [i + width / 2 for i in x],
        vns_values,
        width=width,
        label="VNS"
    )

    plt.xticks(list(x), labels)
    plt.ylabel("Nilai")
    plt.title("Perbandingan Hasil CWS dan VNS", fontsize=14, fontweight="bold")
    plt.grid(axis="y", linestyle="--", alpha=0.5)
    plt.legend()

    for i, value in enumerate(cws_values):
        plt.text(
            i - width / 2,
            value,
            f"{value:.2f}",
            ha="center",
            va="bottom",
            fontsize=9
        )

    for i, value in enumerate(vns_values):
        plt.text(
            i + width / 2,
            value,

```

```

        f"{value:.2f}",
        ha="center",
        va="bottom",
        fontsize=9
    )

    plt.tight_layout()
    plt.savefig("perbandingan_cws_vns.png", dpi=300)
    plt.show()
# =====
# PLOT KONVERGENSI VNS
# =====

def plot_convergence(history):
    plt.figure(figsize=(8, 5))

    plt.plot(
        range(len(history)),
        history,
        marker="o",
        linewidth=2
    )

    plt.title("Grafik Konvergensi VNS", fontsize=14, fontweight="bold")
    plt.xlabel("Iterasi")
    plt.ylabel("Nilai Objektif / Total Emisi")
    plt.grid(True, linestyle="--", alpha=0.5)
    plt.tight_layout()
    plt.savefig("konvergensi_vns.png", dpi=300)
    plt.show()

# =====
# EKSEKUSI PROGRAM
# =====

print_vehicle_data()

print_solution("HASIL AWAL CLARKE-WRIGHT SAVINGS", initial_solution)
validate_solution(initial_solution, "SOLUSI CWS")

best_solution, history = vns(
    initial_solution=initial_solution,
    max_iter=100,
    k_max=4,
    seed=1
)

print_solution("HASIL OPTIMASI VARIABLE NEIGHBORHOOD SEARCH", best_solution)
validate_solution(best_solution, "SOLUSI VNS")

print_comparison(initial_solution, best_solution)

plot_solution(
    initial_solution,
    "Plot Rute Awal Clarke-Wright Savings",
    "plot_rute_cws.png"
)

plot_solution(
    best_solution,
    "Plot Rute Hasil Optimasi VNS dengan Koefisien Emisi Berbeda",
    "plot_rute_vns.png"
)

```

```
plot_comparison(initial_solution, best_solution)
plot_convergence(history)
```

CWS 2 data 32 dengan kapasitas beda

```
import math
import matplotlib.pyplot as plt

# =====
# DATA CVRP A-n32-k5
# Depot = titik 1
# =====

coords = {
    1: (82, 76), 2: (96, 44), 3: (50, 5), 4: (49, 8), 5: (13, 7),
    6: (29, 89), 7: (58, 30), 8: (84, 39), 9: (14, 24), 10: (2, 39),
    11: (3, 82), 12: (5, 10), 13: (98, 52), 14: (84, 25), 15: (61, 59),
    16: (1, 65), 17: (88, 51), 18: (91, 2), 19: (19, 32), 20: (93, 3),
    21: (50, 93), 22: (98, 14), 23: (5, 42), 24: (42, 9), 25: (61, 62),
    26: (9, 97), 27: (80, 55), 28: (57, 69), 29: (23, 15), 30: (20, 70),
    31: (85, 60), 32: (98, 5)
}

Demand = {
    1: 0, 2: 19, 3: 21, 4: 6, 5: 19, 6: 7, 7: 12, 8: 16,
    9: 6, 10: 16, 11: 8, 12: 14, 13: 21, 14: 16, 15: 3,
    16: 22, 17: 18, 18: 19, 19: 1, 20: 24, 21: 8, 22: 12,
    23: 4, 24: 8, 25: 24, 26: 24, 27: 2, 28: 20, 29: 15,
    30: 2, 31: 14, 32: 9
}

DEPOT = 1

# =====
# DATA KENDARAAN
# Kapasitas berbeda dan koefisien emisi berbeda
#  $E = (a + b \times \text{load}) \times d$ 
# =====

vehicles = [
    {"nama": "Kendaraan 1", "tahun": 2024, "capacity": 100, "a": 0.150, "b": 0.0050},
    {"nama": "Kendaraan 2", "tahun": 2021, "capacity": 90, "a": 0.165, "b": 0.0055},
    {"nama": "Kendaraan 3", "tahun": 2018, "capacity": 90, "a": 0.185, "b": 0.0062},
    {"nama": "Kendaraan 4", "tahun": 2014, "capacity": 80, "a": 0.215, "b": 0.0072},
    {"nama": "Kendaraan 5", "tahun": 2010, "capacity": 70, "a": 0.250, "b": 0.0085}
]

# =====
# FUNGSI DASAR
# =====

def distance(i, j):
    xi, yi = coords[i]
    xj, yj = coords[j]
    return int(round(math.sqrt((xi - xj) ** 2 + (yi - yj) ** 2)))

def route_load(route):
    return sum(Demand[node] for node in route)

def route_distance(route):
    if len(route) == 0:
        return 0
```

```

total = 0
previous = DEPOT

for node in route:
    total += distance(previous, node)
    previous = node

total += distance(previous, DEPOT)
return total

def route_emission(route, vehicle):
    if len(route) == 0:
        return 0

    a = vehicle["a"]
    b = vehicle["b"]

    load = route_load(route)
    total_emission = 0
    previous = DEPOT

    for node in route:
        d = distance(previous, node)
        emission = (a + b * load) * d
        total_emission += emission

        load -= Demand[node]
        previous = node

    d = distance(previous, DEPOT)
    emission = (a + b * load) * d
    total_emission += emission

    return total_emission

def total_distance(solution):
    return sum(route_distance(route) for route in solution)

def total_emission(solution):
    total = 0

    for vehicle_index, route in enumerate(solution):
        vehicle = vehicles[vehicle_index]
        total += route_emission(route, vehicle)

    return total

# =====
# SAVING MATRIX
# Rumus:
#  $S_{ij} = d(1,i) + d(1,j) - d(i,j)$ 
# Karena depot = titik 1
# =====

def compute_savings():
    customers = [node for node in coords if node != DEPOT]
    savings = []

    for index, i in enumerate(customers):
        for j in customers[index + 1:]:
            saving_value = distance(DEPOT, i) + distance(DEPOT, j) - distance(i, j)
            savings.append((saving_value, i, j))

```

```

    savings.sort(reverse=True)
    return savings

# =====
# FUNGSI BANTU CWS
# =====

def find_route_index(routes, node):
    for index, route in enumerate(routes):
        if node in route:
            return index
    return None

def merge_routes(route_i, route_j, i, j):
    """
    Penggabungan hanya boleh dilakukan apabila i dan j berada
    di ujung rute masing-masing.
    """

    if i not in (route_i[0], route_i[-1]):
        return None

    if j not in (route_j[0], route_j[-1]):
        return None

    if i == route_i[-1] and j == route_j[0]:
        return route_i + route_j

    if i == route_i[0] and j == route_j[-1]:
        return route_j + route_i

    if i == route_i[0] and j == route_j[0]:
        return list(reversed(route_i)) + route_j

    if i == route_i[-1] and j == route_j[-1]:
        return route_i + list(reversed(route_j))

    return None

# =====
# CEK KAPASITAS HETEROGEN
# Agar rute tidak kembali sama seperti kapasitas 100 semua
# =====

def threshold_capacity_ok(routes):
    """
    Mengecek apakah kumpulan rute masih mungkin ditugaskan ke kendaraan
    dengan kapasitas berbeda.

    Contoh:
    Dengan kapasitas 100, 90, 90, 80, 70,
    hanya boleh ada satu rute dengan Demand > 90,
    hanya boleh ada tiga rute dengan Demand > 80,
    dan seterusnya.
    """

    capacities = [vehicle["capacity"] for vehicle in vehicles]

    for threshold in sorted(set(capacities)):
        route_count = sum(1 for route in routes if route_load(route) > threshold)
        vehicle_count = sum(1 for cap in capacities if cap > threshold)

        if route_count > vehicle_count:
            return False

```

```

    return True

def can_assign_routes(routes):
    """
    Mengecek apakah rute dapat ditugaskan ke kendaraan berdasarkan kapasitas.
    """

    route_loads = [route_load(route) for route in routes]
    capacities = [vehicle["capacity"] for vehicle in vehicles]

    order = sorted(range(len(routes)), key=lambda i: route_loads[i], reverse=True)
    used_vehicle = [False for _ in vehicles]

    def backtrack(position):
        if position == len(order):
            return True

        route_index = order[position]
        load = route_loads[route_index]

        for vehicle_index, capacity in enumerate(capacities):
            if used_vehicle[vehicle_index]:
                continue

            if load <= capacity:
                used_vehicle[vehicle_index] = True

                if backtrack(position + 1):
                    return True

                used_vehicle[vehicle_index] = False

        return False

    return backtrack(0)

# =====
# METODE CLARKE-WRIGHT SAVINGS
# Dengan kapasitas kendaraan berbeda
# =====

def clarke_wright_savings():
    customers = [node for node in coords if node != DEPOT]

    # Setiap pelanggan awalnya menjadi satu rute sendiri
    routes = [[customer] for customer in customers]

    savings = compute_savings()
    max_capacity = max(vehicle["capacity"] for vehicle in vehicles)

    for saving_value, i, j in savings:
        route_i_index = find_route_index(routes, i)
        route_j_index = find_route_index(routes, j)

        if route_i_index is None or route_j_index is None:
            continue

        if route_i_index == route_j_index:
            continue

        route_i = routes[route_i_index]
        route_j = routes[route_j_index]

```

```

new_route = merge_routes(route_i, route_j, i, j)

if new_route is None:
    continue

if route_load(new_route) > max_capacity:
    continue

candidate_routes = [route[:] for route in routes]

for index in sorted([route_i_index, route_j_index], reverse=True):
    candidate_routes.pop(index)

candidate_routes.append(new_route)

# Syarat tambahan agar rute akhir mengikuti kapasitas berbeda
if not threshold_capacity_ok(candidate_routes):
    continue

# Jika jumlah rute sudah <= jumlah kendaraan,
# maka harus benar-benar bisa ditugaskan ke kendaraan.
if len(candidate_routes) <= len(vehicles):
    if not can_assign_routes(candidate_routes):
        continue

routes = candidate_routes

return routes

# =====
# PENUGASAN RUTE KE KENDARAAN
# Dipilih penugasan dengan total emisi paling kecil
# =====

def assign_routes_to_vehicles(routes):
    routes_sorted = sorted(routes, key=lambda route: route_load(route), reverse=True)

    if len(routes_sorted) > len(vehicles):
        raise ValueError(
            "Jumlah rute lebih banyak daripada jumlah kendaraan. "
            "Solusi tidak feasible dengan jumlah kendaraan tersedia."
        )

    best_assignment = None
    best_emission = float("inf")

    assigned = [[] for _ in vehicles]
    used_vehicle = [False for _ in vehicles]

    def backtrack(route_index):
        nonlocal best_assignment, best_emission

        if route_index == len(routes_sorted):
            current_emission = total_emission(assigned)

            if current_emission < best_emission:
                best_emission = current_emission
                best_assignment = [route[:] for route in assigned]

        return

    route = routes_sorted[route_index]
    load = route_load(route)

```

```

    for vehicle_index, vehicle in enumerate(vehicles):
        if used_vehicle[vehicle_index]:
            continue

        if load <= vehicle["capacity"]:
            assigned[vehicle_index] = route
            used_vehicle[vehicle_index] = True

            backtrack(route_index + 1)

            assigned[vehicle_index] = []
            used_vehicle[vehicle_index] = False

    backtrack(0)

    if best_assignment is None:
        raise ValueError(
            "Rute hasil CWS tidak dapat ditugaskan ke kendaraan "
            "karena kapasitas tidak mencukupi."
        )

    return best_assignment

# =====
# VALIDASI SOLUSI
# =====

def validate_solution(solution):
    all_customers = []

    for vehicle_index, route in enumerate(solution):
        vehicle = vehicles[vehicle_index]

        if route_load(route) > vehicle["capacity"]:
            return False

        all_customers.extend(route)

    return sorted(all_customers) == list(range(2, 33))

# =====
# CETAK HASIL
# =====

def print_vehicle_data():
    print("=" * 90)
    print("DATA KENDARAAN")
    print("=" * 90)

    for vehicle in vehicles:
        print(
            f"{vehicle['nama']} | "
            f"Tahun: {vehicle['tahun']} | "
            f"Kapasitas: {vehicle['capacity']} | "
            f"a: {vehicle['a']} | "
            f"b: {vehicle['b']}"
        )

    print("=" * 90)

def print_solution(solution):
    print("\n" + "=" * 90)
    print("HASIL CLARKE-WRIGHT SAVINGS DENGAN KAPASITAS BERBEDA")
    print("=" * 90)

```

```

for vehicle_index, route in enumerate(solution):
    vehicle = vehicles[vehicle_index]
    full_route = [DEPOT] + route + [DEPOT]

    print(f"\n{vehicle['nama']}")
    print(f"Tahun      : {vehicle['tahun']}")
    print(f"Kapasitas  : {vehicle['capacity']}")
    print(f"Koefisien  : a = {vehicle['a']}, b = {vehicle['b']}")

    if len(route) == 0:
        print("Rute      : Tidak digunakan")
        print("Demand    : 0")
        print("Jarak      : 0")
        print("Emisi      : 0")
    else:
        print("Rute      :", " -> ".join(map(str, full_route)))
        print(f"Demand    : {route_load(route)} / {vehicle['capacity']}")
        print(f"Jarak      : {route_distance(route)}")
        print(f"Emisi      : {route_emission(route, vehicle):.4f}")

    print("\n" + "-" * 90)
    print(f"Total kendaraan tersedia : {len(vehicles)}")
    print(f"Total kendaraan digunakan: {sum(1 for route in solution if len(route) >
0)})")
    print(f"Total Demand              : {sum(route_load(route) for route in solution)}")
    print(f"Total jarak                : {total_distance(solution)}")
    print(f"Total emisi                : {total_emission(solution):.4f}")
    print("-" * 90)

def print_validation(solution):
    print("\nVALIDASI SOLUSI CWS")

    if validate_solution(solution):
        print("Semua pelanggan dilayani tepat satu kali.")

        for vehicle_index, route in enumerate(solution):
            vehicle = vehicles[vehicle_index]

            if len(route) == 0:
                print(f"{vehicle['nama']}: tidak digunakan")
            else:
                print(
                    f"{vehicle['nama']}: kapasitas aman "
                    f"{(route_load(route)/vehicle['capacity'])}"
                )
        else:
            print("Solusi tidak feasible.")

# =====
# PLOT RUTE HASIL CWS
# =====

def plot_solution(solution, filename="plot_rute_cws_kapasitas_berbeda.png"):
    plt.figure(figsize=(11, 8))

    for node, (x, y) in coords.items():
        if node == DEPOT:
            plt.scatter(x, y, s=220, marker="s", label="Depot")
            plt.text(x + 1, y + 1, str(node), fontsize=11, fontweight="bold")
        else:
            plt.scatter(x, y, s=70)
            plt.text(x + 1, y + 1, str(node), fontsize=9)

```

```

for vehicle_index, route in enumerate(solution):
    if len(route) == 0:
        continue

    vehicle = vehicles[vehicle_index]
    full_route = [DEPOT] + route + [DEPOT]

    x_values = [coords[node][0] for node in full_route]
    y_values = [coords[node][1] for node in full_route]

    label_text = (
        f"{vehicle['nama']} | "
        f"Cap {vehicle['capacity']} | "
        f"Load {route_load(route)}"
    )

    plt.plot(
        x_values,
        y_values,
        marker="o",
        linewidth=2,
        label=label_text
    )

    for start, end in zip(full_route[:-1], full_route[1:]):
        x1, y1 = coords[start]
        x2, y2 = coords[end]

        plt.annotate(
            "",
            xy=(x2, y2),
            xytext=(x1, y1),
            arrowprops=dict(
                arrowstyle="->",
                lw=1.2,
                shrinkA=9,
                shrinkB=9
            )
        )

    plt.title(
        "Plot Rute Hasil CWS dengan Kapasitas Kendaraan Berbeda",
        fontsize=14,
        fontweight="bold"
    )
    plt.xlabel("Koordinat X")
    plt.ylabel("Koordinat Y")
    plt.grid(True, linestyle="--", alpha=0.5)
    plt.legend(fontsize=8)
    plt.tight_layout()
    plt.savefig(filename, dpi=300)
    plt.show()

# =====
# EKSEKUSI PROGRAM
# =====

print_vehicle_data()

routes_cws = clarke_wright_savings()
solution_cws = assign_routes_to_vehicles(routes_cws)

print_solution(solution_cws)
print_validation(solution_cws)

```

```
plot_solution(solution_cws)
```

VNS 3 Data 32 kapasitas dan koefisien emisi beda

```
import math
import random
import matplotlib.pyplot as plt

# =====
# DATA A-n32-k5
# =====

coords = {
    1: (82, 76), 2: (96, 44), 3: (50, 5), 4: (49, 8), 5: (13, 7),
    6: (29, 89), 7: (58, 30), 8: (84, 39), 9: (14, 24), 10: (2, 39),
    11: (3, 82), 12: (5, 10), 13: (98, 52), 14: (84, 25), 15: (61, 59),
    16: (1, 65), 17: (88, 51), 18: (91, 2), 19: (19, 32), 20: (93, 3),
    21: (50, 93), 22: (98, 14), 23: (5, 42), 24: (42, 9), 25: (61, 62),
    26: (9, 97), 27: (80, 55), 28: (57, 69), 29: (23, 15), 30: (20, 70),
    31: (85, 60), 32: (98, 5)
}

Demand = {
    1: 0, 2: 19, 3: 21, 4: 6, 5: 19, 6: 7, 7: 12, 8: 16,
    9: 6, 10: 16, 11: 8, 12: 14, 13: 21, 14: 16, 15: 3,
    16: 22, 17: 18, 18: 19, 19: 1, 20: 24, 21: 8, 22: 12,
    23: 4, 24: 8, 25: 24, 26: 24, 27: 2, 28: 20, 29: 15,
    30: 2, 31: 14, 32: 9
}

DEPOT = 1

# =====
# DATA KENDARAAN
# Kapasitas dan koefisien emisi berbeda
#  $E = (a + b \times \text{load}) \times d$ 
# =====
# Parameter VNS
MAX_ITER = 2500
K_MAX = 4
TOTAL_SEED = 3

vehicles = [
    {"nama": "Kendaraan 1", "tahun": 2024, "capacity": 100, "a": 0.150, "b": 0.0050},
    {"nama": "Kendaraan 2", "tahun": 2021, "capacity": 90, "a": 0.165, "b": 0.0055},
    {"nama": "Kendaraan 3", "tahun": 2018, "capacity": 90, "a": 0.185, "b": 0.0062},
    {"nama": "Kendaraan 4", "tahun": 2014, "capacity": 80, "a": 0.215, "b": 0.0072},
    {"nama": "Kendaraan 5", "tahun": 2010, "capacity": 70, "a": 0.250, "b": 0.0085}
]

# =====
# HASIL CWS KAPASITAS BERBEDA
# Digunakan sebagai solusi awal VNS
# =====

solution_cws = [
    [24, 4, 3, 18, 20, 32, 22],
    [7, 19, 29, 5, 12, 9, 10, 23, 15],
    [17, 8, 14, 2, 13],
    [21, 6, 26, 11, 16, 30],
    [28, 25, 27, 31]
]
```

```

# =====
# FUNGSI DASAR
# =====

def distance(i, j):
    xi, yi = coords[i]
    xj, yj = coords[j]
    return int(round(math.sqrt((xi - xj) ** 2 + (yi - yj) ** 2)))

def route_load(route):
    return sum(Demand[node] for node in route)

def route_distance(route):
    total = 0
    previous = DEPOT

    for node in route:
        total += distance(previous, node)
        previous = node

    total += distance(previous, DEPOT)
    return total

def route_emission(route, vehicle_index):
    vehicle = vehicles[vehicle_index]
    a = vehicle["a"]
    b = vehicle["b"]

    load = route_load(route)
    total = 0
    previous = DEPOT

    for node in route:
        d = distance(previous, node)
        total += (a + b * load) * d

        load -= Demand[node]
        previous = node

    d = distance(previous, DEPOT)
    total += (a + b * load) * d

    return total

def total_distance(solution):
    return sum(route_distance(route) for route in solution)

def total_emission(solution):
    return sum(route_emission(route, i) for i, route in enumerate(solution))

def copy_solution(solution):
    return [route[:] for route in solution]

def is_feasible(solution):
    all_customers = []

    for vehicle_index, route in enumerate(solution):
        if route_load(route) > vehicles[vehicle_index]["capacity"]:
            return False

        all_customers.extend(route)

    return sorted(all_customers) == list(range(2, 33))

```

```

def objective(solution):
    # Fokus utama VNS adalah meminimalkan emisi
    return total_emission(solution)

# =====
# NEIGHBORHOOD 1: 2-OPT
# =====

def best_two_opt(solution):
    best_solution = None
    best_obj = objective(solution)

    for vehicle_index, route in enumerate(solution):
        if len(route) < 2:
            continue

        for i in range(len(route) - 1):
            for j in range(i + 1, len(route)):
                candidate = copy_solution(solution)
                candidate[vehicle_index] = route[:i] + list(reversed(route[i:j+1])) +
route[j+1:]

                if is_feasible(candidate) and objective(candidate) < best_obj:
                    best_solution = candidate
                    best_obj = objective(candidate)

    return best_solution

# =====
# NEIGHBORHOOD 2: RELOCATE
# =====

def best_relocate(solution):
    best_solution = None
    best_obj = objective(solution)

    for r1 in range(len(solution)):
        for i in range(len(solution[r1])):
            node = solution[r1][i]

            for r2 in range(len(solution)):
                if r1 != r2:
                    if route_load(solution[r2]) + Demand[node] >
vehicles[r2]["capacity"]:
                        continue

                    for j in range(len(solution[r2]) + 1):
                        if r1 == r2 and (j == i or j == i + 1):
                            continue

                        candidate = copy_solution(solution)
                        moved = candidate[r1].pop(i)

                        if r1 == r2 and j > i:
                            j -= 1

                        candidate[r2].insert(j, moved)

                        if is_feasible(candidate) and objective(candidate) < best_obj:
                            best_solution = candidate
                            best_obj = objective(candidate)

    return best_solution

```

```

# =====
# NEIGHBORHOOD 3: SWAP
# =====

def best_swap(solution):
    best_solution = None
    best_obj = objective(solution)

    route_loads = [route_load(route) for route in solution]

    for r1 in range(len(solution)):
        for i in range(len(solution[r1])):
            node1 = solution[r1][i]

            for r2 in range(r1, len(solution)):
                start_j = i + 1 if r1 == r2 else 0

                for j in range(start_j, len(solution[r2])):
                    node2 = solution[r2][j]

                    if r1 != r2:
                        new_load_r1 = route_loads[r1] - Demand[node1] + Demand[node2]
                        new_load_r2 = route_loads[r2] - Demand[node2] + Demand[node1]

                        if new_load_r1 > vehicles[r1]["capacity"]:
                            continue

                        if new_load_r2 > vehicles[r2]["capacity"]:
                            continue

                    candidate = copy_solution(solution)
                    candidate[r1][i], candidate[r2][j] = candidate[r2][j],
candidate[r1][i]

                    if is_feasible(candidate) and objective(candidate) < best_obj:
                        best_solution = candidate
                        best_obj = objective(candidate)

    return best_solution

# =====
# NEIGHBORHOOD 4: ROUTE EXCHANGE
# Menukar seluruh rute antar kendaraan
# =====

def best_route_exchange(solution):
    best_solution = None
    best_obj = objective(solution)

    for r1 in range(len(solution) - 1):
        for r2 in range(r1 + 1, len(solution)):
            load_r1 = route_load(solution[r1])
            load_r2 = route_load(solution[r2])

            if load_r2 > vehicles[r1]["capacity"]:
                continue

            if load_r1 > vehicles[r2]["capacity"]:
                continue

            candidate = copy_solution(solution)
            candidate[r1], candidate[r2] = candidate[r2], candidate[r1]

```

```

        if is_feasible(candidate) and objective(candidate) < best_obj:
            best_solution = candidate
            best_obj = objective(candidate)

    return best_solution

# =====
# LOCAL SEARCH VNS
# =====

def local_search(solution):
    current = copy_solution(solution)

    neighborhoods = [
        best_route_exchange,
        best_two_opt,
        best_relocate,
        best_swap
    ]

    improved = True
    while improved:
        improved = False

        for neighborhood in neighborhoods:
            candidate = neighborhood(current)

            if candidate is not None and objective(candidate) < objective(current):
                current = candidate
                improved = True
                break

    return current

# =====
# SHAKING VNS
# =====

def random_two_opt(solution):
    candidate = copy_solution(solution)
    routes = [i for i, route in enumerate(candidate) if len(route) >= 2]

    if not routes:
        return candidate

    r = random.choice(routes)
    i, j = sorted(random.sample(range(len(candidate[r])), 2))

    candidate[r][i:j+1] = list(reversed(candidate[r][i:j+1]))

    if is_feasible(candidate):
        return candidate

    return copy_solution(solution)

def random_swap(solution):
    for _ in range(100):
        candidate = copy_solution(solution)

        r1, r2 = random.sample(range(len(candidate)), 2)

        if len(candidate[r1]) == 0 or len(candidate[r2]) == 0:
            continue

```

```

        i = random.randrange(len(candidate[r1]))
        j = random.randrange(len(candidate[r2]))

        candidate[r1][i], candidate[r2][j] = candidate[r2][j], candidate[r1][i]

        if is_feasible(candidate):
            return candidate

    return copy_solution(solution)

def random_relocate(solution):
    for _ in range(100):
        candidate = copy_solution(solution)

        r1 = random.choice([i for i, route in enumerate(candidate) if len(route) > 0])
        i = random.randrange(len(candidate[r1]))

        node = candidate[r1][i]
        r2 = random.randrange(len(candidate))

        if r1 != r2:
            if route_load(candidate[r2]) + Demand[node] > vehicles[r2]["capacity"]:
                continue

        moved = candidate[r1].pop(i)
        j = random.randrange(len(candidate[r2]) + 1)
        candidate[r2].insert(j, moved)

        if is_feasible(candidate):
            return candidate

    return copy_solution(solution)

def shake(solution, k):
    candidate = copy_solution(solution)

    moves = [
        random_two_opt,
        random_swap,
        random_relocate
    ]

    for _ in range(k + 1):
        move = random.choice(moves)
        candidate = move(candidate)

    return candidate

# =====
# METHODE VNS
# =====

def vns(initial_solution, max_iter=100, k_max=3, seed=1):
    random.seed(seed)

    best_solution = local_search(initial_solution)
    best_obj = objective(best_solution)

    history = [best_obj]

    for iteration in range(max_iter):
        k = 0

```

```

while k < k_max:
    shaken = shake(best_solution, k)
    local = local_search(shaken)
    local_obj = objective(local)

    if local_obj < best_obj:
        best_solution = local
        best_obj = local_obj
        k = 0
    else:
        k += 1

history.append(best_obj)

return best_solution, history

# =====
# CETAK HASIL
# =====

def print_solution(title, solution):
    print("\n" + "=" * 90)
    print(title)
    print("=" * 90)

    for vehicle_index, route in enumerate(solution):
        vehicle = vehicles[vehicle_index]
        full_route = [DEPOT] + route + [DEPOT]

        print(f"\n{vehicle['nama']}")
        print(f"Tahun      : {vehicle['tahun']}")
        print(f"Kapasitas : {vehicle['capacity']}")
        print(f"Route      : ", " -> ".join(map(str, full_route)))
        print(f"Demand     : {route_load(route)} / {vehicle['capacity']}")
        print(f"Jarak      : {route_distance(route)}")
        print(f"Emisi      : {route_emission(route, vehicle_index):.4f}")

    print("\n" + "-" * 90)
    print(f"Total Demand : {sum(route_load(route) for route in solution)}")
    print(f"Total jarak  : {total_distance(solution)}")
    print(f"Total emisi  : {total_emission(solution):.4f}")
    print("-" * 90)

def print_comparison(cws, vns_result):
    print("\n" + "=" * 90)
    print("PERBANDINGAN CWS DAN VNS")
    print("=" * 90)

    print(f"Total jarak CWS : {total_distance(cws)}")
    print(f"Total jarak VNS : {total_distance(vns_result)}")
    print(f"Selisih jarak   : {total_distance(cws) - total_distance(vns_result)}")

    print()

    print(f"Total emisi CWS : {total_emission(cws):.4f}")
    print(f"Total emisi VNS : {total_emission(vns_result):.4f}")
    print(f"Selisih emisi   : {total_emission(cws) - total_emission(vns_result):.4f}")

    if total_emission(cws) > 0:
        persen = ((total_emission(cws) - total_emission(vns_result)) /
total_emission(cws)) * 100
        print(f"Penurunan emisi : {persen:.2f}%")

    print("=" * 90)

```

```

# =====
# PLOT RUTE
# =====

def plot_solution(solution, title, filename):
    plt.figure(figsize=(11, 8))

    for node, (x, y) in coords.items():
        if node == DEPOT:
            plt.scatter(x, y, s=220, marker="s", label="Depot")
            plt.text(x + 1, y + 1, str(node), fontsize=11, fontweight="bold")
        else:
            plt.scatter(x, y, s=70)
            plt.text(x + 1, y + 1, str(node), fontsize=9)

    for vehicle_index, route in enumerate(solution):
        vehicle = vehicles[vehicle_index]
        full_route = [DEPOT] + route + [DEPOT]

        x_values = [coords[node][0] for node in full_route]
        y_values = [coords[node][1] for node in full_route]

        label_text = (
            f"{vehicle['nama']} | "
            f"Load {route_load(route)}/{vehicle['capacity']} | "
            f"E {route_emission(route, vehicle_index):.2f}"
        )

        plt.plot(
            x_values,
            y_values,
            marker="o",
            linewidth=2,
            label=label_text
        )

        for start, end in zip(full_route[:-1], full_route[1:]):
            x1, y1 = coords[start]
            x2, y2 = coords[end]

            plt.annotate(
                "",
                xy=(x2, y2),
                xytext=(x1, y1),
                arrowprops=dict(
                    arrowstyle="->",
                    lw=1.2,
                    shrinkA=9,
                    shrinkB=9
                )
            )

    plt.title(title, fontsize=14, fontweight="bold")
    plt.xlabel("Koordinat X")
    plt.ylabel("Koordinat Y")
    plt.grid(True, linestyle="--", alpha=0.5)
    plt.legend(fontsize=8)
    plt.tight_layout()
    plt.savefig(filename, dpi=300)
    plt.show()

def plot_convergence(history):
    plt.figure(figsize=(8, 5))

```

```

plt.plot(range(len(history)), history, marker="o", linewidth=2)
plt.title("Grafik Konvergensi VNS")
plt.xlabel("Iterasi")
plt.ylabel("Total Emisi")
plt.grid(True, linestyle="--", alpha=0.5)
plt.tight_layout()
plt.savefig("konvergensi_vns.png", dpi=300)
plt.show()

# =====
# EKSEKUSI PROGRAM
# =====

if not is_feasible(solution_cws):
    print("Solusi CWS tidak feasible. Periksa kapasitas kendaraan.")
else:
    print_solution("HASIL AWAL CWS", solution_cws)

    best_solution, history = vns(
        initial_solution=solution_cws,
        max_iter=100,
        k_max=3,
        seed=1
    )

    print_solution("HASIL OPTIMASI VNS", best_solution)
    print_comparison(solution_cws, best_solution)

    plot_solution(
        solution_cws,
        "Plot Rute Awal CWS",
        "plot_rute_cws.png"
    )

    plot_solution(
        best_solution,
        "Plot Rute Hasil VNS",
        "plot_rute_vns.png"
    )

    plot_convergence(history)

```